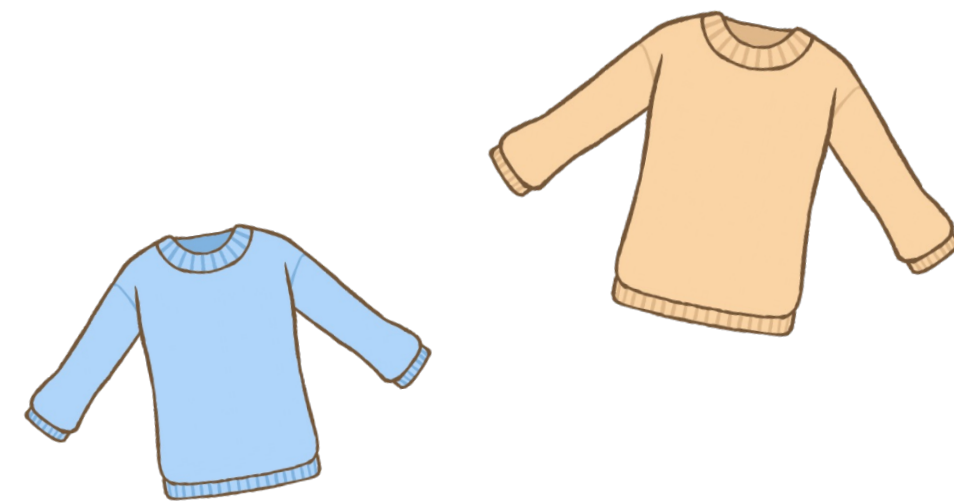


JOI 2023/2024 本選

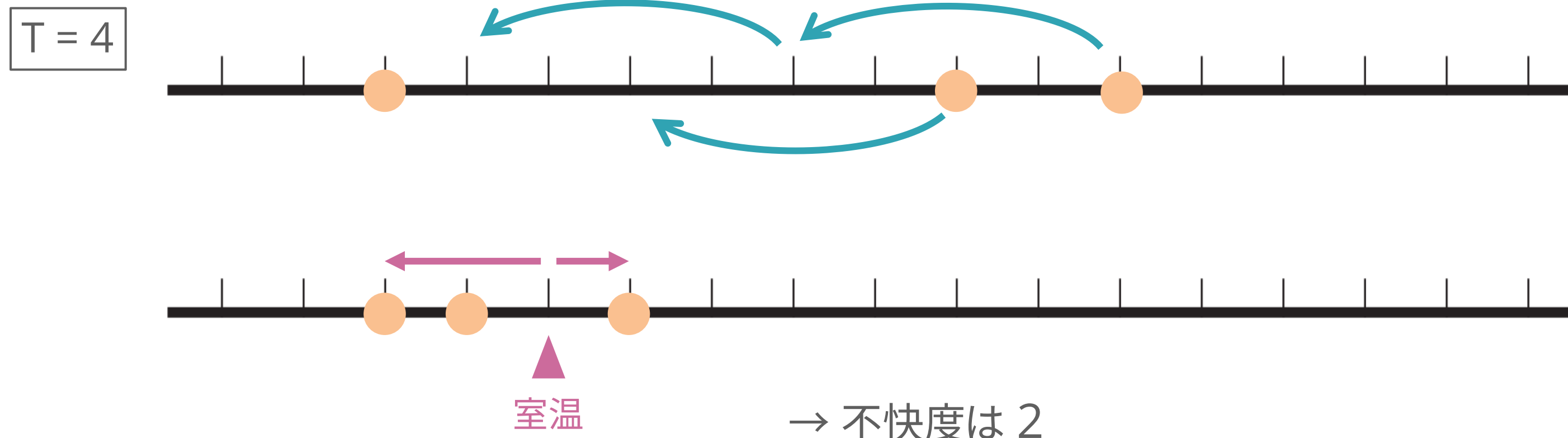
第1問「室温」解説

解説担当: 萩原 千晴 (Mendako)



問題概要

- 役員が N 人、適温は A_i
- 適温は上着で T 度ずつ下げられる
- 室温を適切に設定して、
室温と適温の差 (不快度) の最大値をできるだけ小さくしたい



配点

小課題 1 (15点)

$$N = 2$$

小課題 2 (5点)

$$N \leq 3000, T = 1$$

小課題 3 (30点)

$$N \leq 3000, T = 1, 2$$

小課題 4 (35点)

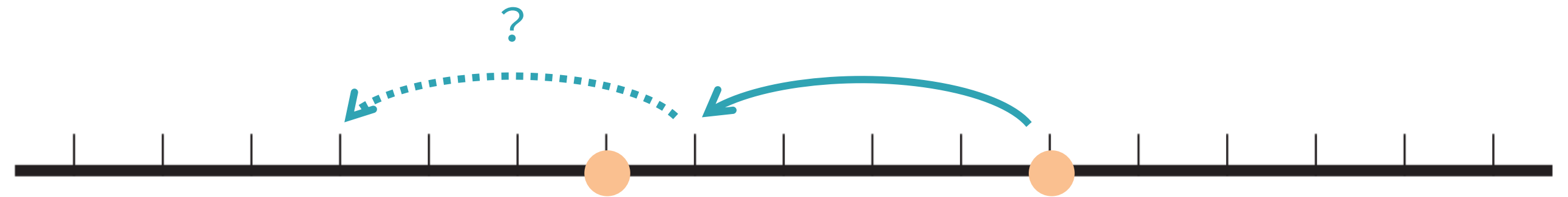
$$N \leq 3000, T \leq 3000$$

小課題 5 (15点)

追加制約なし

$$(2 \leq N \leq 500000, 1 \leq T \leq 10^9, 1 \leq A_i \leq 10^9)$$

小課題 1



$N = 2$

- 2 人の適温を近づけたい
- 2 人の適温の差を d として、
適温が高い方の役員が $\lfloor d / T \rfloor$ もしくは $\lceil d / T \rceil$ 枚の上着を着る
- 最終的な適温の差に 1 を足し 2 で割ったものが答え

小課題 2

$T = 1$

- 適温を 1 度刻みで調節できる
→ 全員の適温を一致させられる
- 室温をその温度にすれば不快値は 0
0 を出力すれば OK



小課題 3

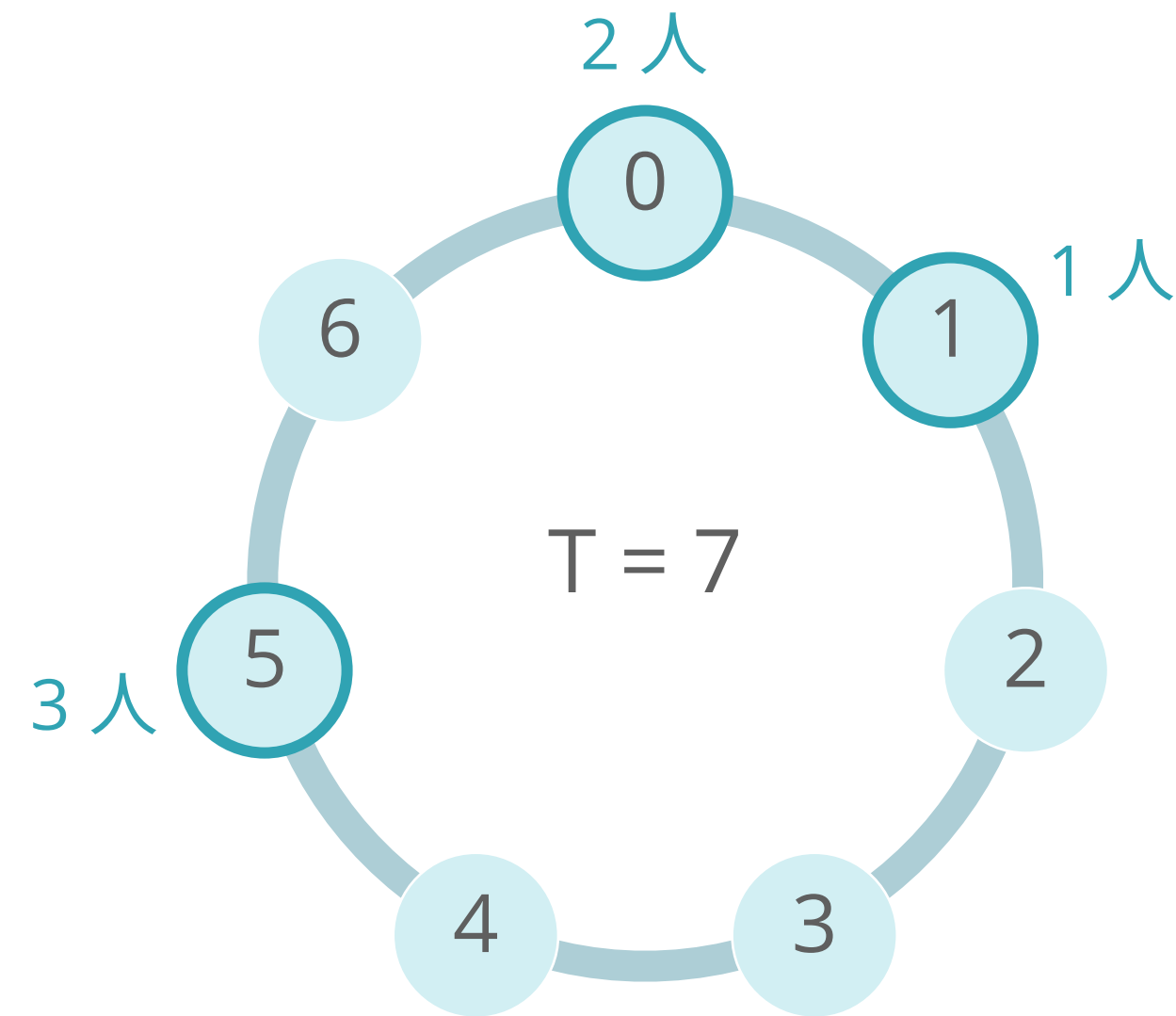
$N \leq 3000, T = 1, 2$

- $T = 1$ のときは小課題 2 の通り、 $T = 2$ について
 - 適温が奇数の役員どうし、偶数の役員どうしは適温を一致させられる
 - 全員の適温の偶奇が一致していれば適温を一致させられる
そうでなければ最後に 1 ずれる
- ➡ 全員の偶奇が一致していれば 0、していなければ 1

小課題 4

$N \leq 3000, T \leq 3000$

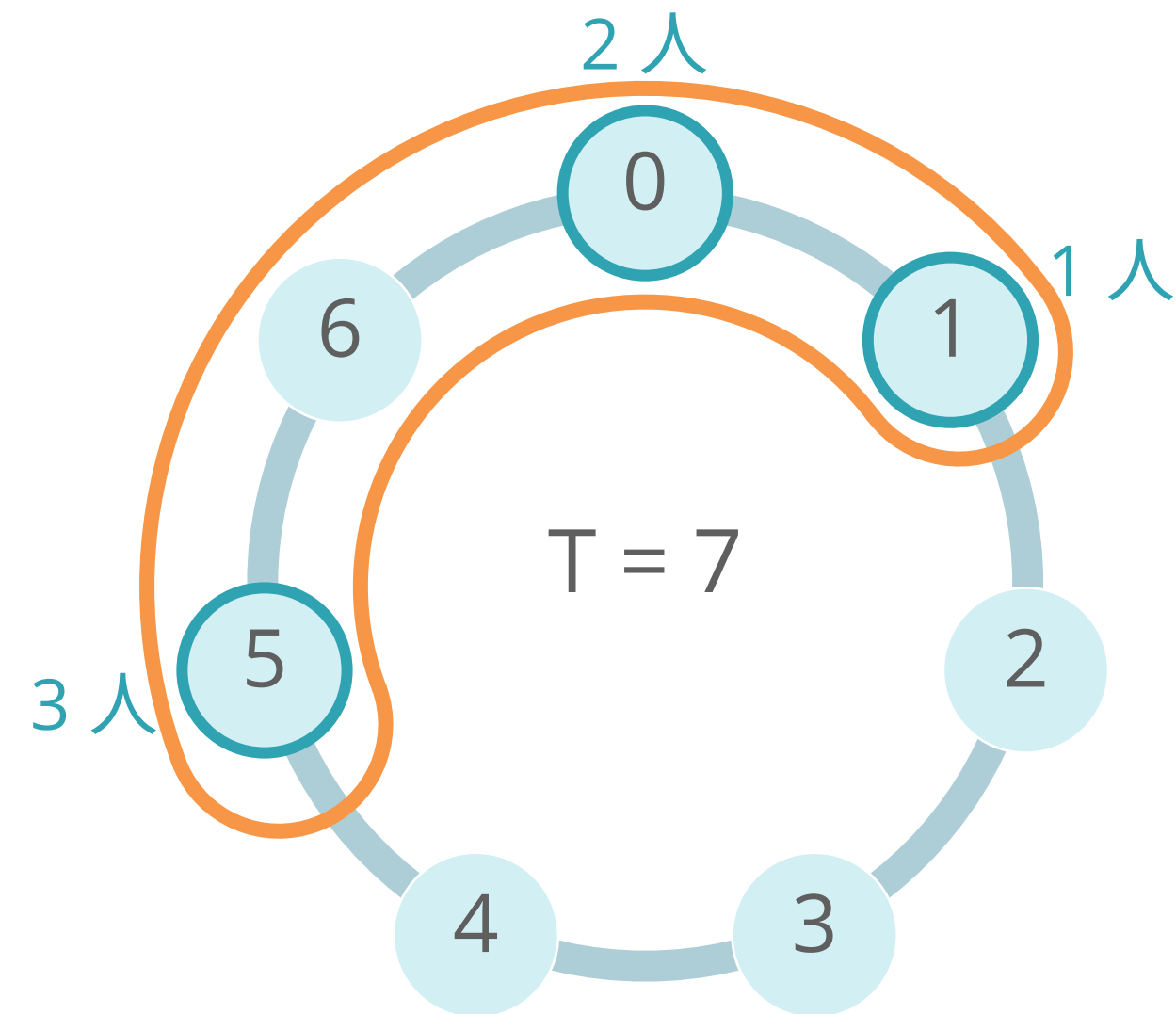
- 適温は T ずつ調節できる
→ $\text{mod } T$ に注目、円環状に考える
- 例えば $T = 7$ として、 $\text{mod } T$ で
 - 0 の役員が 2 人
 - 1 の役員が 1 人
 - 5 の役員が 3 人→ 全員を覆うような円環上の区間のうち
長さが最小なものを知りたい
→ 区間の端点を全探索 $O(T^2)$



小課題 4

$N \leq 3000, T \leq 3000$

- 適温は T ずつ調節できる
→ $\text{mod } T$ に注目、円環状に考える
- 例えば $T = 7$ として、 $\text{mod } T$ で
 - 0 の役員が 2 人
 - 1 の役員が 1 人
 - 5 の役員が 3 人
→ 全員を覆うような円環上の区間のうち
長さが最小なものを知りたい
→ 区間の端点を全探索 $O(T^2)$

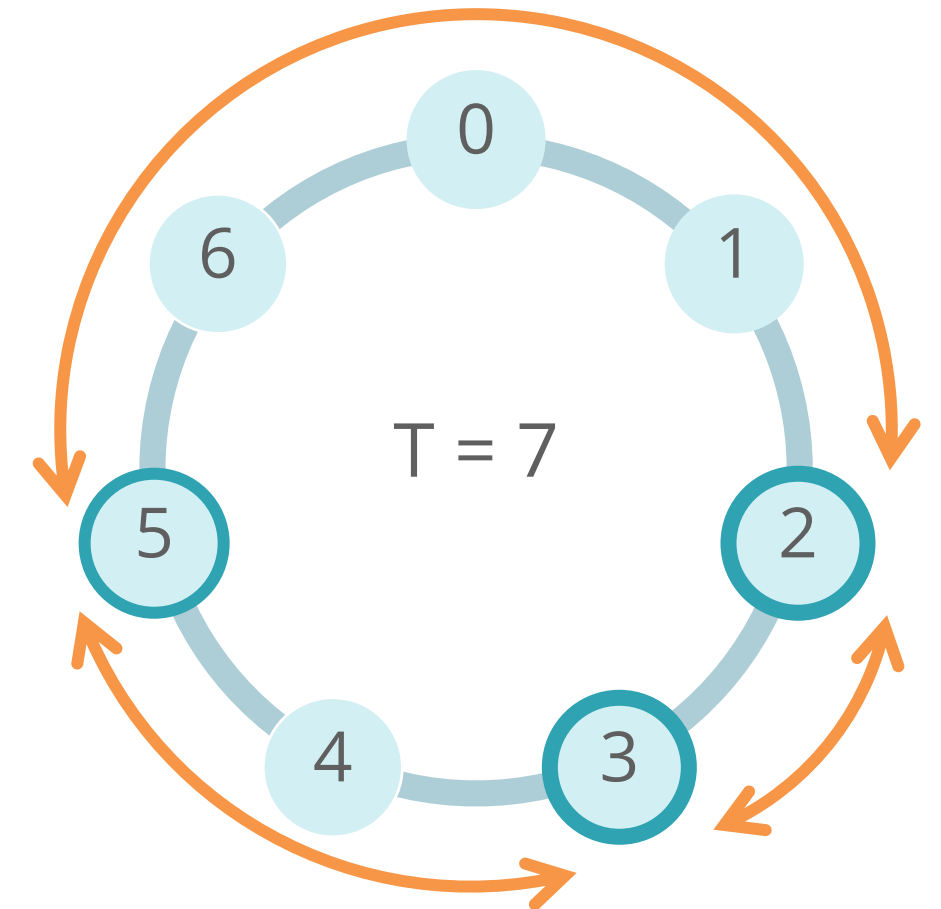


競プロ典型 90 問の
「円環を列にして 2 倍にする」が
実装に利用できる

小課題 5 (満点解法)

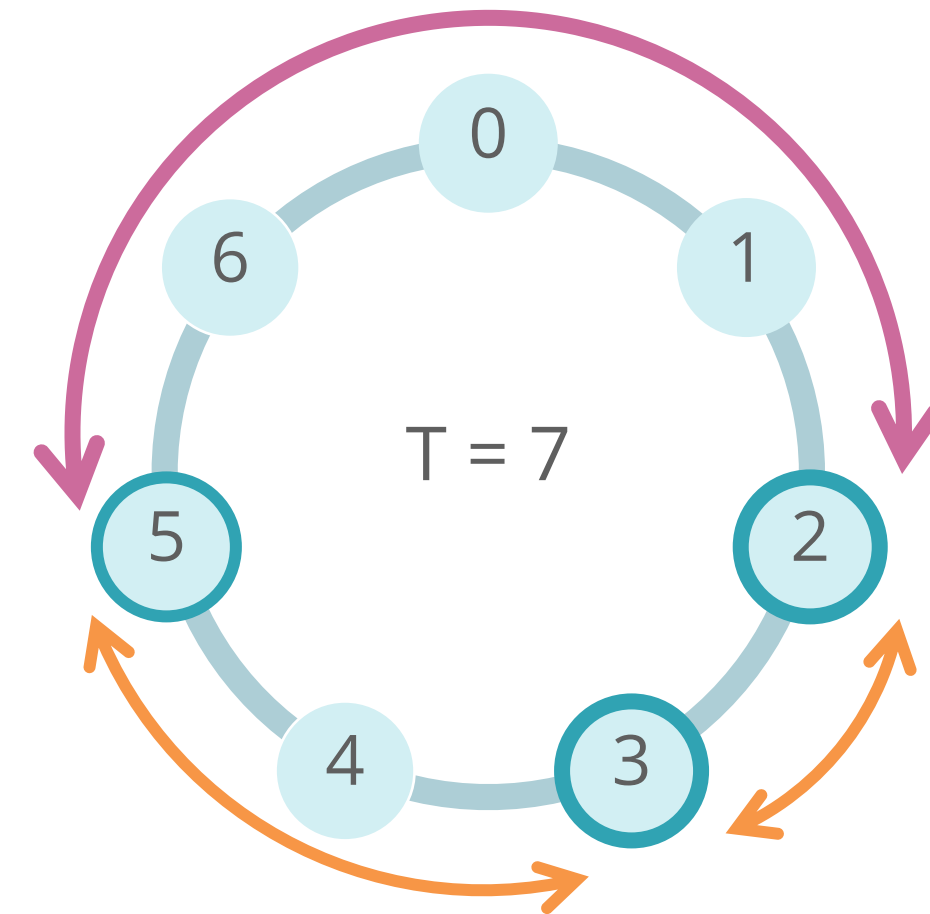
$$2 \leq N \leq 500000, 1 \leq T \leq 10^9, 1 \leq A_i \leq 10^9$$

- 全員を覆うような円環上の区間 を
効率良く求めたい
- この区間に **含まれない** のは隣り合う適温の間
→ 隣り合う適温の差を全探索し
円環長 T から引けば良い $O(N)$

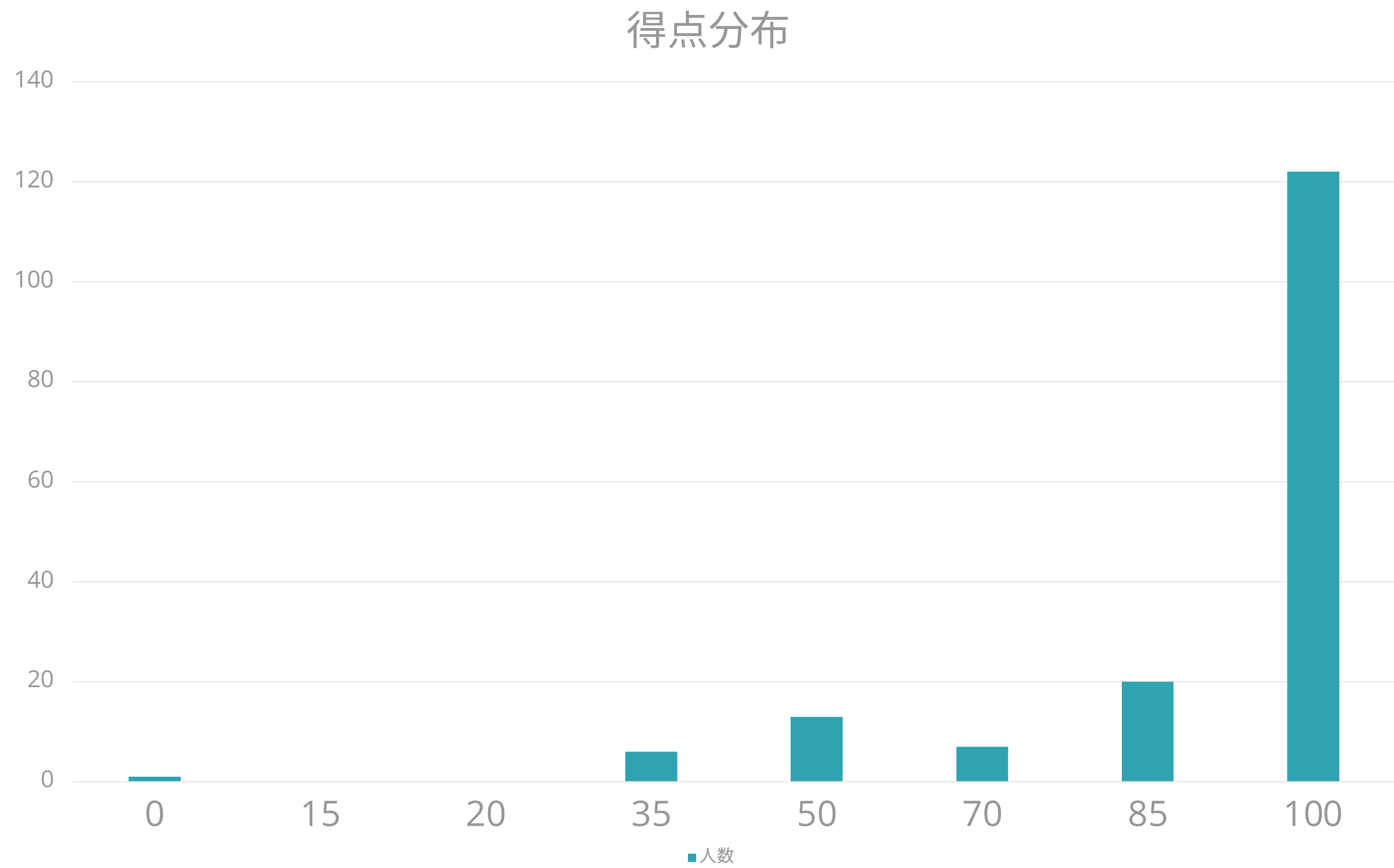


小課題 5 (満点解法) – 実装

- 適当の $\text{mod } T$ を取った B_i をソート
- $T - (B_{i+1} - B_i)$ が区間長なので、その最小値を求める
- 上だけだと **太線** のような 0 をまたぐ分が考慮できていないので区間長として $B_N - B_1$ も比較
- 最終的な区間長に 1 を足し 2 で割ったものが答え



得点分布





JOI2023/2024 本選 2 問目 建設事業 2 (Construction Project 2) 解説

解説: 蜂矢 倫久 (Mitsubachi)

Step 0

問題概要

0

問題概要

- N 頂点 M 辺のグラフがあります
- 辺は無向辺で、 i 番目の辺の長さは C_i です
- 長さ L の辺を 1 本追加して、条件「頂点 S から頂点 T までの最短距離が K 以下」を満たしたいです
- 追加して条件を満たせる辺は何通りありますか

0

制約

- $N, M \leq 200\,000$
- $L, C_i \leq 10^9$
- $K \leq 10^{15}$

0

小課題

- 小課題1 (8点) : $L = 1, K = 2, C_i = 1$



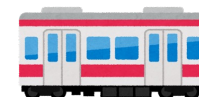
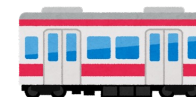
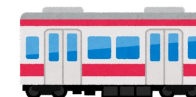
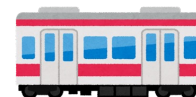
- 小課題2 (16点) : $N, M \leq 50$



- 小課題3 (29点) : $N, M \leq 3\,000$



- 小課題4 (47点) : 追加制約なし



Subtask 1

$$L = 1, K = 2, C_i = 1$$

1

はじめに

- そもそも最初から条件を満たされているなら辺をどう追加しても OK
- よって答えは $N(N-1)/2$ (オーバーフローに注意!)
- これはどの小課題においても成り立つ

1

最短距離を求めよう

- 最短距離をどうやって求める？
- 小課題 1 は 2 以下かを判定すれば OK
- 最短距離が $1 \Leftrightarrow S$ と T を直接結ぶ辺がある
- 最短距離が $2 \Leftrightarrow S$ と T の両方に繋がる頂点がある
- $O(N+M)$ で判定可能

1

最短距離 > 2 の場合

- 最短距離が 3 以上だったとする
- 最短距離を 1 か 2 にしたい
- 最短距離を 1 にする $\rightarrow$ S と T を結ぶ
- 最短距離を 2 にする $\rightarrow$ S に繋がってる頂点と T を結ぶ or T に繋がってる頂点と S を結ぶ

1

S に繋がってる頂点を数える

- S に繋がってる頂点の数を数えたい
- vector や set で $A_i = S$ のときの B_i や $B_i = S$ のときの A_i を管理すればいい
- T についても同様
- $O(N+M)$ など解けました

Subtask 2

$N, M \leq 50$

2

問題の言い換え

- 問題を言い換えると「 $N(N-1)/2$ 個のグラフがあるので、それら全てに対して S から T までの最短距離が K 以下を判定してください」となる
- 最短距離を求めるには？
- ダイクストラ法やワーシャルフロイド法を使おう

2

最短距離を求めよう

- 各グラフに対して、ダイクストラ法やワーシャルフロイド法を使おう
- それぞれの計算量は $O((N+M)\log N)$, $O(N^3)$
- よって $O(N^2(N+M)\log N)$ や $O(N^5)$ で解けました

Subtask 3

$N, M \leq 3\,000$

3 追加前の最短距離について

- 追加前の最短距離が K 以下 $\rightarrow$ 答えは $N(N-1)/2$
- 追加前の最短距離は K より大きいとしよう

3

視点を変えてみる

- 追加前の最短距離は K より大きいとする
- 新しく追加する辺に対して条件を満たせるか判定したい
- 実は、追加した辺は使うものとしていい ← なぜ？

3

言い換え

- もし、追加した辺を使わない方法で距離が K 以下となるなら最短距離は K 以下のはず $\rightarrow$ 矛盾
- u と v の間に辺を追加したとして考える行き方は「 $S \rightarrow u \rightarrow v \rightarrow T$ 」か「 $S \rightarrow v \rightarrow u \rightarrow T$ 」の2通り(先に u と v のどちらを経由するか)

3 解法

- $S \rightarrow u \rightarrow v \rightarrow T$ の行き方の最短距離は (S から u までの最短距離) + L + (v から T までの最短距離)
- $S \rightarrow v \rightarrow u \rightarrow T$ の行き方の最短距離は (S から v までの最短距離) + L + (u から T までの最短距離)
- (S から i までの最短距離) と (T から i までの最短距離) を全ての頂点 i について知りたい
- これはダイクストラ法

3 解法

- ① ダイクストラ法で (S から i までの最短距離) と (T から i までの最短距離) を全ての頂点 i について求める
- ② $1 \leq u < v \leq N$ を満たす (u, v) について
「(S から u までの最短距離) + L + (v から T までの最短距離) $\leq K$ 」か
「(S から v までの最短距離) + L + (u から T までの最短距離) $\leq K$ 」のうち 1 つ以上を満たすかを判定

3

解法

- ダイクストラ法で $O((N+M)\log N)$ 、判定部分で $O(N^2)$
- 全体で $O((N+M)\log N + N^2)$ で解けました

Subtask 4

追加制約なし

4

追加前の最短距離について

- 追加前の最短距離が K 以下 $\rightarrow$ 答えは $N(N-1)/2$
- 追加前の最短距離は K より大きいとしよう

4

小課題 3 をおさらい

- 条件P「(S から u までの最短距離) + L + (v から T までの最短距離) $\leq$ K」
- 条件Q「(S から v までの最短距離) + L + (u から T までの最短距離) $\leq$ K」
- P か Q のうち 1 つ以上を満たす (u, v) の数を数えたい
- これは「P を満たす (u, v) の数」 + 「Q を満たす (u, v) の数」 - 「P と Q を両方満たす (u, v) の数」と言い換えられる

4

重要な性質

- 実は「P と Q を両方満たす (u, v) の数」 = 0 ← なぜ？
- (S から u までの最短距離) = a ,
- (v から T までの最短距離) = b ,
- (S から v までの最短距離) = c ,
- (u から T までの最短距離) = d とする
- 条件 P $\Leftrightarrow a + L + b \leq K \Leftrightarrow a + b \leq K - L$
- 条件 Q $\Leftrightarrow c + L + d \leq K \Leftrightarrow c + d \leq K - L$

4

証明

- u を経由する方法を考えると**追加した辺を使わずに**距離 $a + d$ で S から T へ行ける
- v を経由する方法を考えると**追加した辺を使わずに**距離 $b + c$ で S から T へ行ける
- よって追加前の最短距離は $\min(a + d, b + c)$ 以下
- ここで追加前の最短距離は K より大きいので
 $a + d > K, b + c > K$

4

証明

- $a + d > K, b + c > K$
- 足し合わせると $a + b + c + d > 2K$
- ところで、2 ページ前のスライドでは以下を示した
- 条件 P $\Leftrightarrow a + L + b \leq K \Leftrightarrow a + b \leq K - L$
- 条件 Q $\Leftrightarrow c + L + d \leq K \Leftrightarrow c + d \leq K - L$
- 足し合わせると $a + b + c + d \leq 2K - 2L$

4

証明

- よって $2K < 2K - 2L$ となり $L < 0$ となる $\rightarrow$ 矛盾！
- これより示されました
- よって答え = 「P を満たす (u, v) の数」 + 「Q を満たす (u, v) の数」 となる
- ここで $u < v$ が成り立っている(次ページのために明記)

4

求めるものの言い換え

- 「 $u < v$ かつ P を満たす (u, v) の数」 + 「 $u < v$ かつ Q を満たす (u, v) の数」を求めたい
- 実はこれは「 P を満たす (u, v) の数」に等しい ← なぜ？
- $u = v$ となることはない(重要な性質の証明より成り立つ)
- $u > v$ かつ P を満たす (u, v) は u と v を入れ替えれば $u < v$ かつ Q を満たす (u, v) と一対一対応できる
- よって「 $u < v$ かつ Q を満たす (u, v) の数」 = 「 $u > v$ かつ P を満たす (u, v) の数」となり、示されました

4

解法

- 「P を満たす (u, v) の数」を求める
- u を固定する
- 「 $(v \text{ から } T \text{ までの最短距離}) \leq K - (S \text{ から } u \text{ までの最短距離}) - L$ 」となる v の個数を求めたい
- あらかじめ $(v \text{ から } T \text{ までの最短距離})$ を sort しておけば二分探索で $O(\log N)$ で v の個数を求めることができる

4

解法

- u を固定したときに $O(\log N)$ で v の個数を求められる
- よってダイクストラ法後は $O(N \log N)$ で数えられる
- ダイクストラ法は $O((N+M) \log N)$ なので全体で $O((N+M) \log N)$ で解けました

4

別解

- 「 $u < v$ かつ P を満たす (u, v) の数」を直接求められる
- 各頂点からの距離を座標圧縮して BIT や Segment Tree などの一点更新区間和取得が可能なデータ構造を使う
- $v = 1, 2, \dots$ の順に $O(\log N)$ で処理できるので全体で $O((N+M)\log N)$ で解けました

Step 5

得点分布

5

得点分布

- 100点：?人
- 53点：?人
- 45点：?人
- 24点：?人
- 16点：?人
- 8点：?人

5

得点分布

- 100点 : 111人
- 53点: 6人
- 45点: 11人
- 24点: 6人
- 16点: 7人
- 8点: 4人





マラソン大会 2

JOI 2023/2024 本選 問題 3 解説

解説担当：米田 寛峻（よねだ ひろたか / square1001）

数直線上の座標 $X_1, X_2, \dots, X_N$ にボールが落ちています

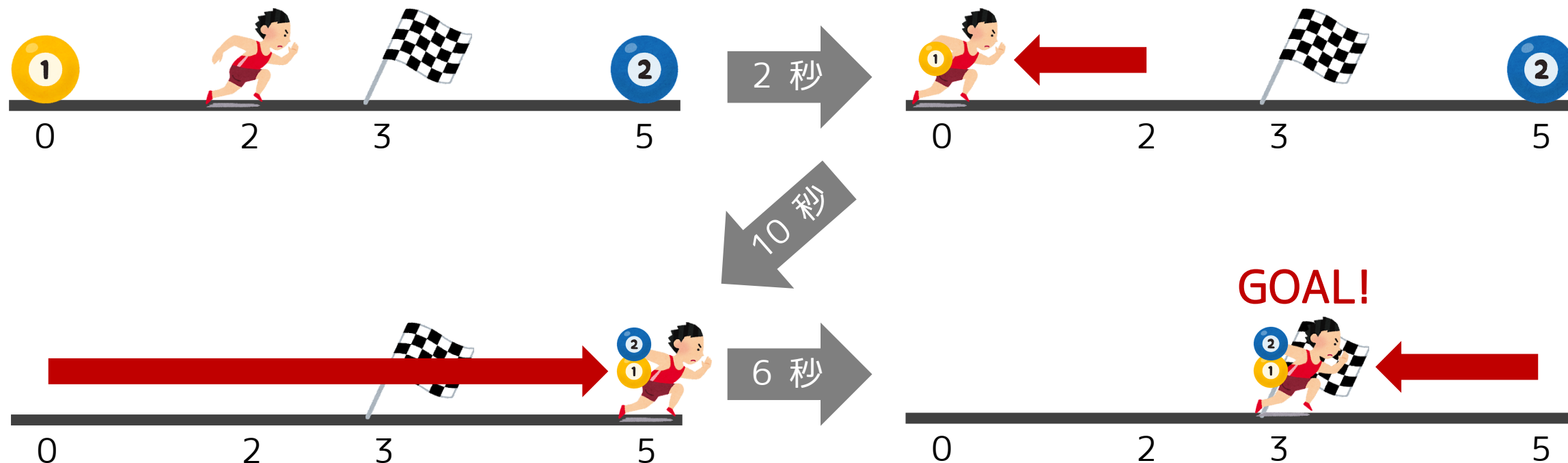
距離 1 の移動に（運んでいるボールの個数）+ 1 秒かかります

以下の q 個の質問に答えましょう：

- スタート地点 s_j 、ゴール地点 G_j 、制限時間 T_j 秒
- このとき、 N 個のボールを回収して（落とさずに）ゴールは可能か？



具体例で考えてみよう：制限時間 20 秒でゴール可能か？



	累計得点	N	Q	追加制約
1	7 点	7	10	$S_j = 0, G_j = 0$
2	14 点			
3	24 点	14		
4	52 点	100		
5	62 点	2000	500000	
6	81 点			
7	100 点	500000		



1 小課題 1 [累計 7 点]

5 / 41

小課題 1 では、スタート地点とゴール地点は 道路の左端

Q. ボールをどの順番で取るのが最適か？（最短でゴールできるか？）



1 小課題 1 [累計 7 点]

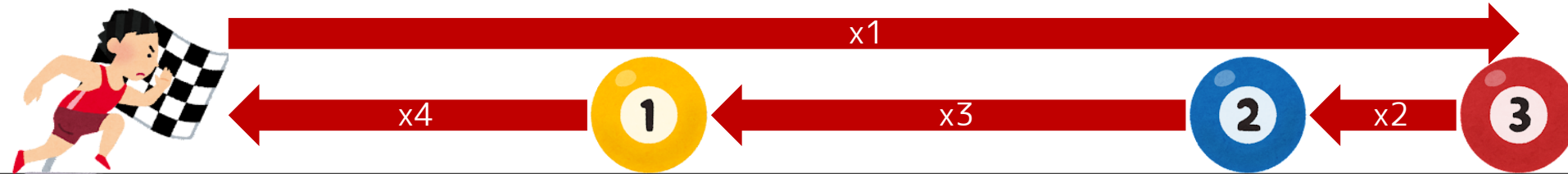
6 / 41

小課題 1 では、スタート地点とゴール地点は 道路の左端

Q. ボールをどの順番で取るのが最適か？（最短でゴールできるか？）

A. 一番右のボールから取る！

最短時間はソートで計算量 $O(N \log N)$ で求められる

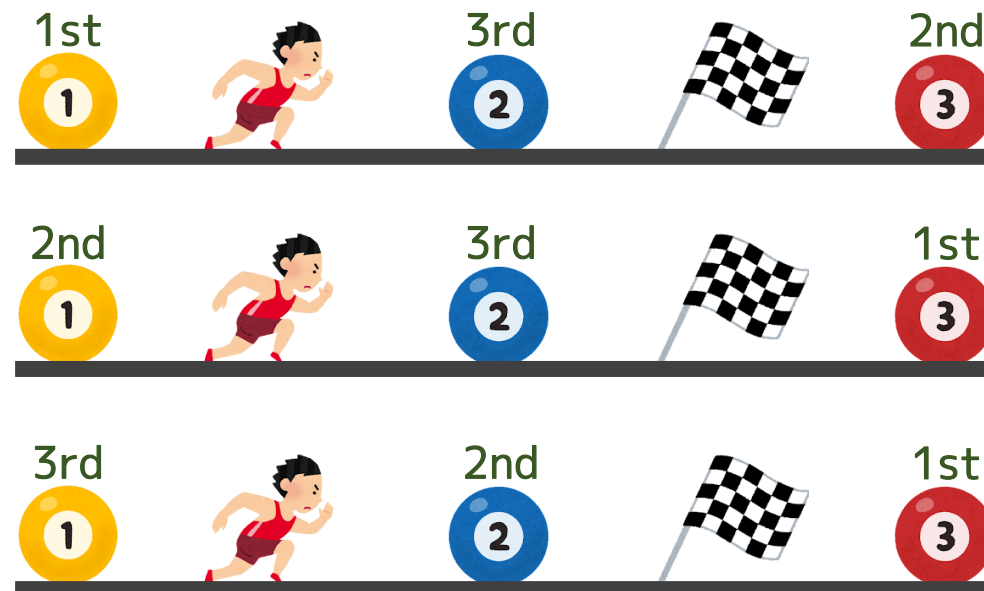
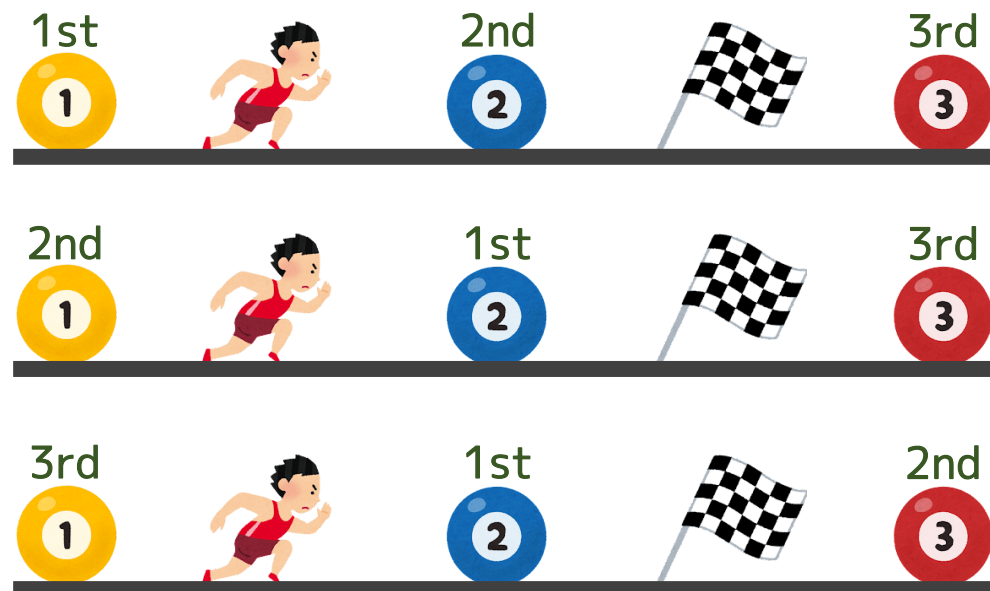


2 小課題 2 [累計 14 点]

7 / 41

小課題 2 では、N が 7 以下

ボールを取る順番を $N! = 1 \times 2 \times \dots \times N$ 通り全探索できる



2 小課題 2 [累計 14 点]

8

41

順列の全探索

C++ では、`next_permutation` を使って順列の全探索を実装できる

```
int mintime = INF;
vector<int> p(N);
for (int i = 0; i < N; i++) {
    p[i] = i;
}
do {
    // t は消費時間、pos は現在位置
    int t = 0, pos = S;
    for (int i = 0; i < N; i++) {
        t += abs(X[p[i]] - pos) * (i + 1);
        pos = X[p[i]];
    }
    t += abs(G - pos) * (N + 1);
    mintime = min(mintime, t);
} while (next_permutation(p.begin(), p.end()));
```

最短時間の計算プログラム

3 小課題 3 [累計 24 点]

9 / 41

小課題 3 では、 N が 14 以下

ビット DP を使って解くことができる

- $dp[S][pos]$ を「現在回収したボールの集合が S であり、現在位置が x_{pos} であるときの、ここまでの最短時間」とする
- 全体の最短時間が計算量 $O(2^N \times N^2)$ で求まる

※ 現在のボールの個数は S から一意に定まるので、DP の状態としてボールの個数を持つ必要がないことに注意

3 小課題 3 [累計 24 点]

10 / 41

ビット DP

動的計画法 (DP) の一種で、状態として「既にとったボールの集合」などの 2^n 通りの部分集合を持つ DP のことを指す

- 2 進数を用いて実装されるので、ビット DP と呼ばれる
- 典型問題：巡回セールスマン問題 (Held-Karp 法)

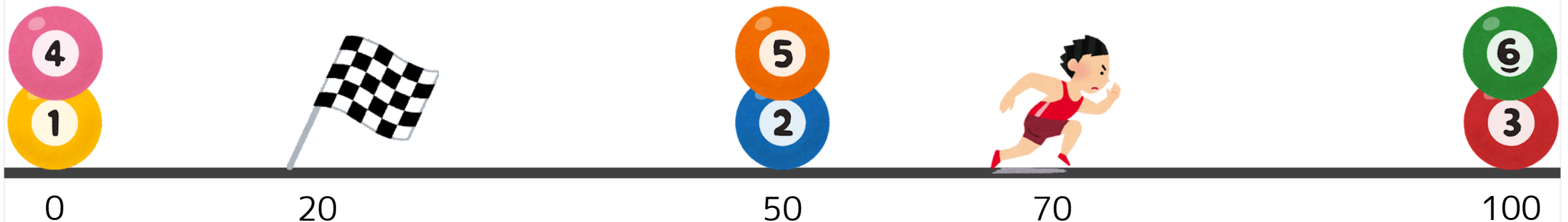
4

最適な経路について

11 / 41

ところで、最適解はどんな経路なのか？ 考えてみよう

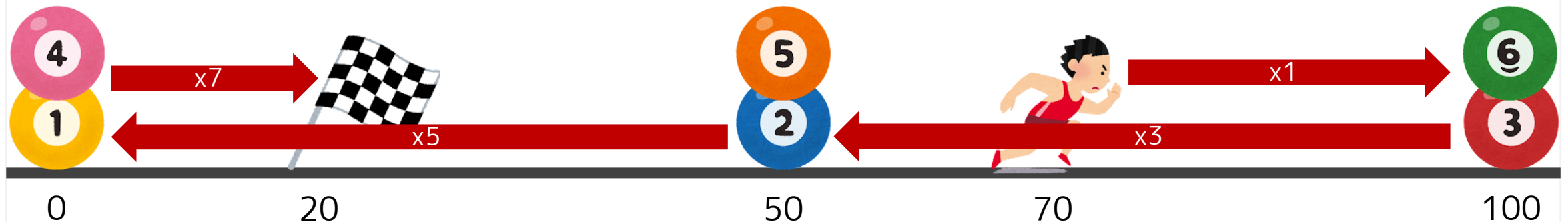
入力例 3: $N = 6$, $X = [0, 50, 100, 0, 50, 100]$, $S = 70$, $G = 20$



ところで、最適解はどんな経路なのか？ 考えてみよう

入力例 3: $N = 6$, $X = [0, 50, 100, 0, 50, 100]$, $S = 70$, $G = 20$

最適解は「スタート → 一番右 → 一番左 → ゴール」(576 秒)



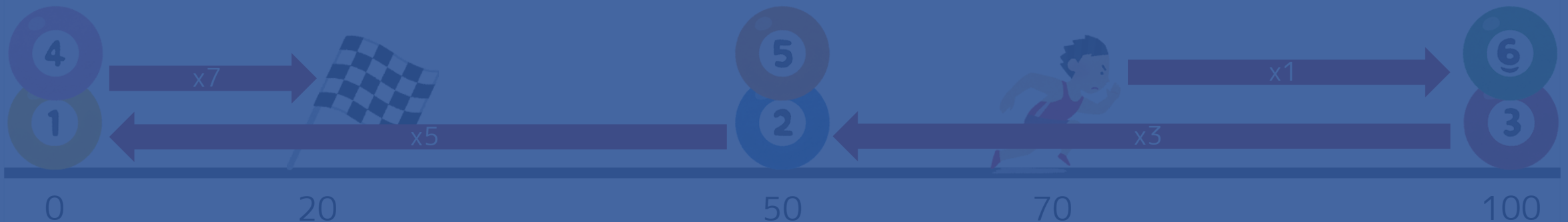
4

最適な経路について

ところで、最適解はどんな経路なのか？ 考えてみよう

入力例: $N = 6, X = [0, 50, 100, 0, 50, 100], S = 70, C = 20$
予想 1: 左右の端で折り返すのが
最適解は「スタート → 一番右 → 一番左 → ゴール」(576 秒)

最適解？



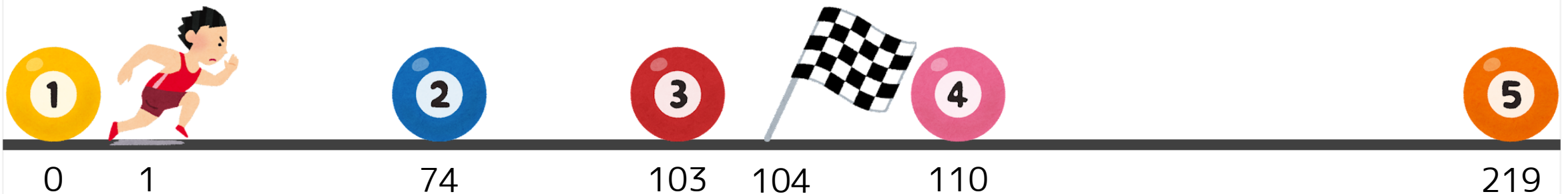
4

最適な経路について

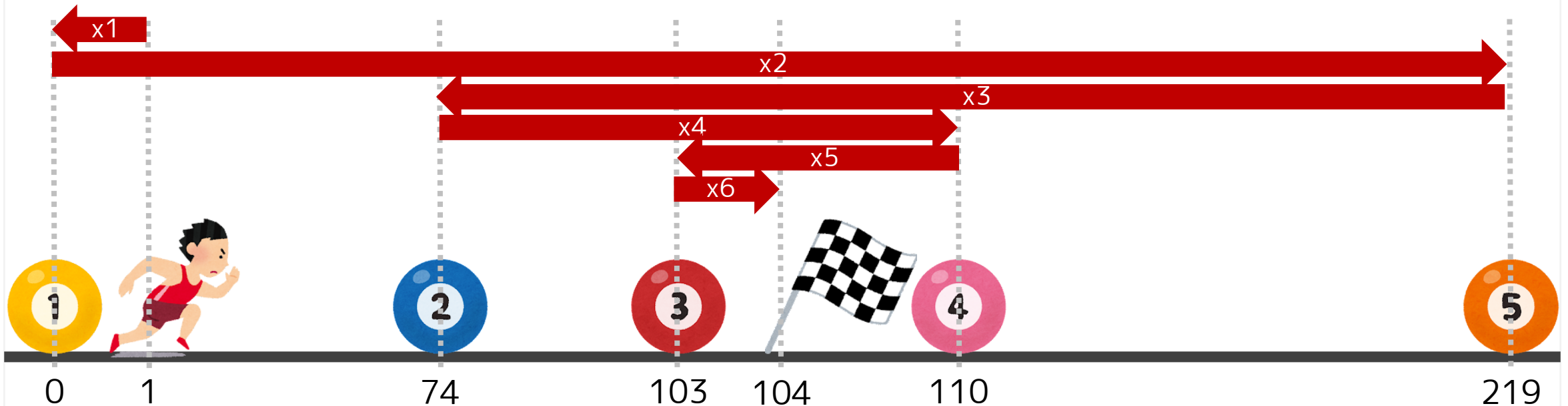
14 / 41

残念ながらこの予想は正しくない

例: $N = 5$, $X = [0, 74, 103, 110, 219]$, $S = 1$, $G = 104$



N 個のボールがあって N 回曲がるのが最適なケースもある！



4

最適な経路について

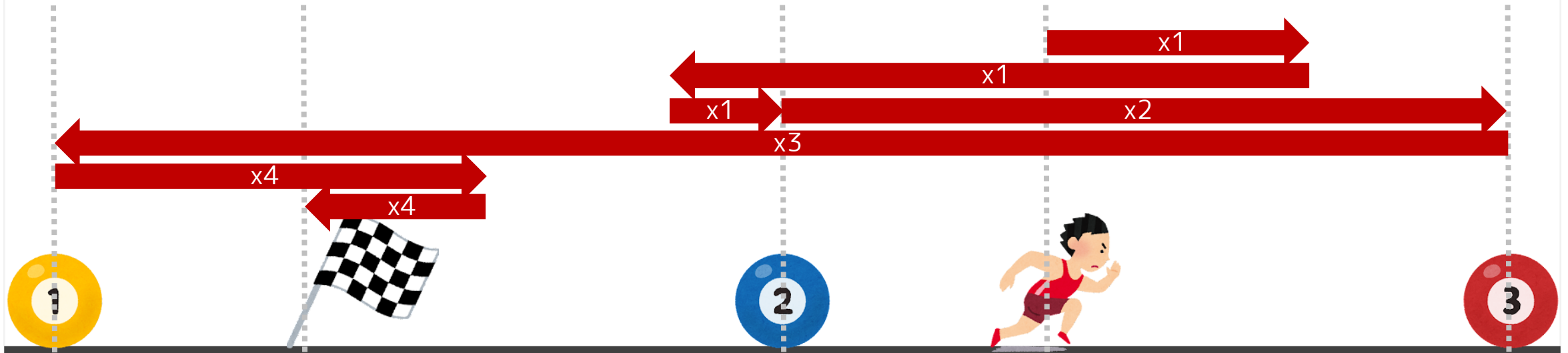
N 個のボールがあって N 回曲がるのが最適なケースもある！

本当にどんな経路でもアリなのか？



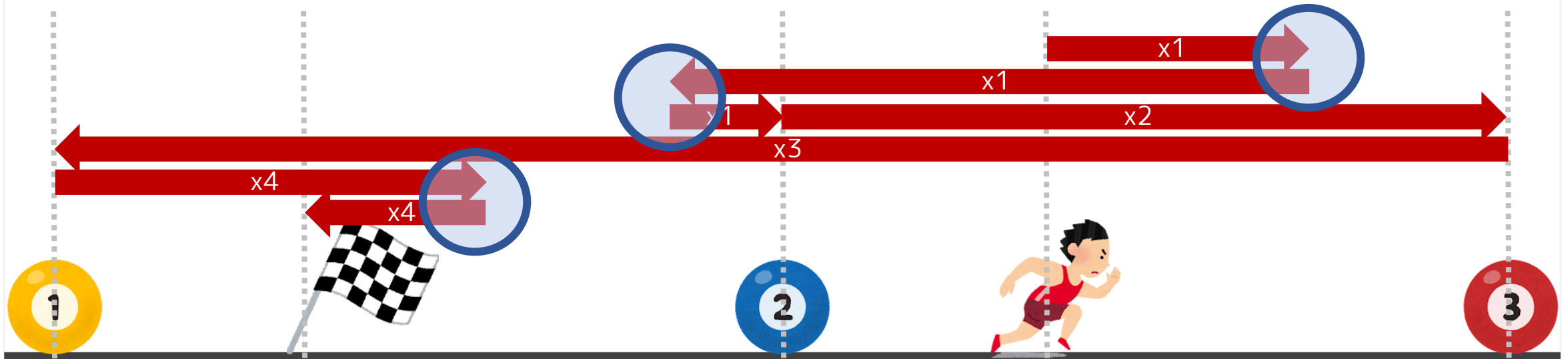
本当にどんな経路でもアリなのか？

適当な経路を考えてみよう：こんな経路はアリ？

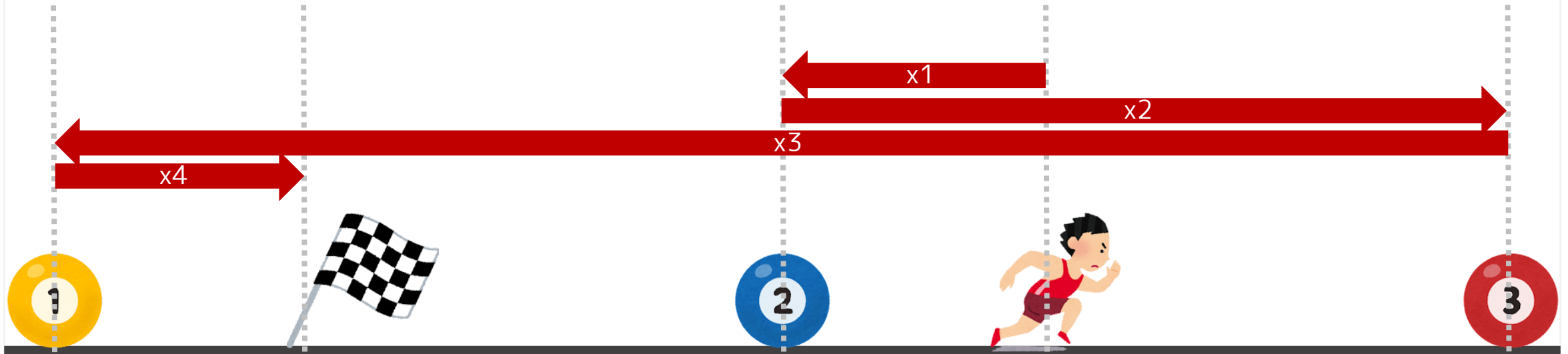


明らかに無駄になっている部分がある（丸の部分）

ポイント 1: ボールに向かって直接進むのが最適

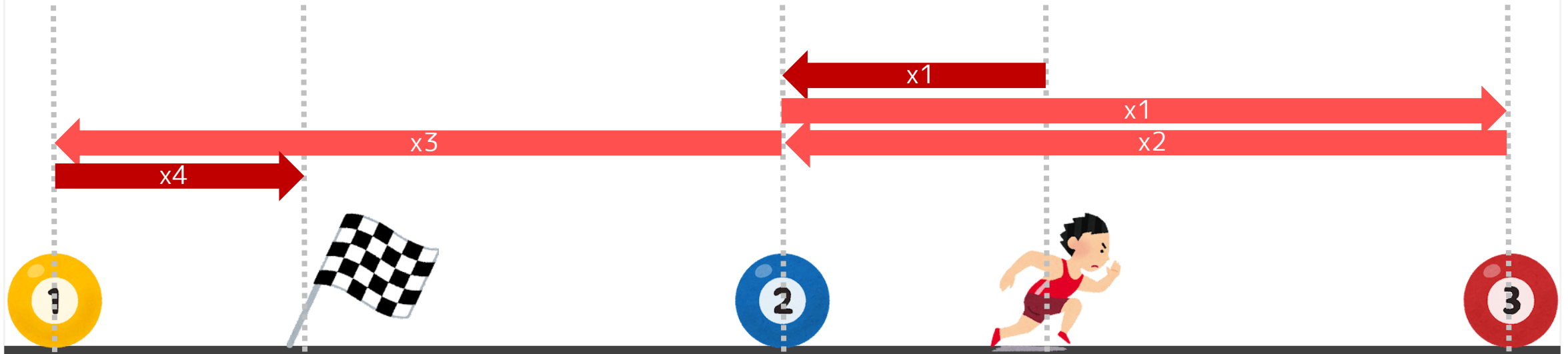


さっきの経路を改善したこんな経路はアリ？



「2」のボールを先に取りよりも後に取りの方が効率的！

ポイント 2: ボールは「最後に通る」ときに回収するのが最適



4

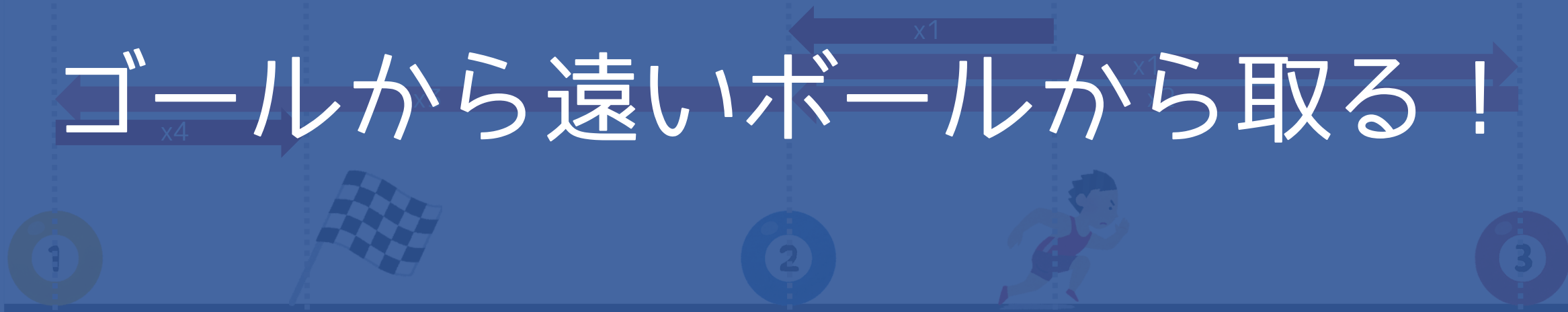
最適な経路について

重要な性質

「2」のボールを先に取るよりも先にゴールを通るのが決定的！
ポイント 2: ボールは「最後に通る」ときに回収するのが最適

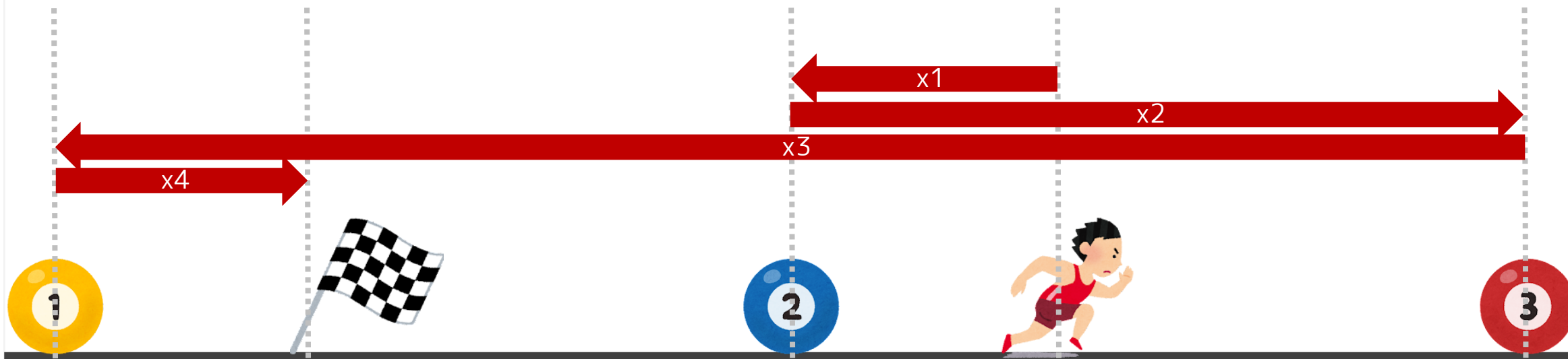
同じサイドであれば最適解では

ゴールから遠いボールから取る！



例：なぜ「2」を「3」より先に取ってはいけないのか？

理由：最後に通るタイミングは「3」よりも「2」の方が必ず遅いから



小課題 3 のビット DP を思い出してみよう

- $dp[S][pos]$ を「現在回収したボールの集合が S であり、現在位置が x_{pos} であるときの、ここまでの最短時間」とする

Q. 本当に $2^N \times N$ 通り全部の (S, pos) の組み合わせを考える必要があるか？

小課題 3 のビット DP を思い出してみよう

- $dp[S][pos]$ を「現在回収したボールの集合が S であり、現在位置が X_{pos} であるときの、ここまでの最短時間」とする

Q. 本当に $2^N \times N$ 通り全部の (S, pos) の組み合わせを考える必要があるか？

A. $S = \{1, 2, \dots, l, r, r+1, \dots, N\}$ で pos が「 l または r 」

だけ考えればよい！（理由：同じサイドならゴールから遠い順に取る）

※ ただし $X_1 \leq X_2 \leq \dots \leq X_N$ とソートされているものとする

以下の DP を考える

- $dp[l][r][d]$: 既にボール $1, 2, \dots, l$ およびボール $r, r+1, \dots, N$ を回収し、現在位置が X_l or X_r (それぞれ $d = 0, 1$) の状態になるまでの最短時間
- 計算量はクエリ当たり $O(N^2)$



小課題 6 では、N が 2000 以下だが、クエリがたくさん

目標

DP を適切に前計算 → 各クエリ計算量 $O(1)$

戦略

元々の DP には S, G の情報が入っているので、そうではない形にするために、 S, G による解の変化を考える

6 小課題 6 [累計 81 点]

27 / 41

以下の 4 パターンの経路が考えられる (S, G によってどれかは変わる)



左端のボールを最初に取り、左からゴールする



左端のボールを最初に取り、右からゴールする



右端のボールを最初に取り、左からゴールする



右端のボールを最初に取り、右からゴールする

以下の DP を前計算することを考える。すると…？

- $dp_L[l][r][d]$: 左端のボールを取ってスタートするときの最短時間
- $dp_R[l][r][d]$: 右端のボールを取ってスタートするときの最短時間

4 パターンの最短時間が以下のように求まる！

- $dp_L[l][r][d]$: 左端のボールを取ってスタートするときの最短時間
- $dp_R[l][r][d]$: 右端のボールを取ってスタートするときの最短時間



最短時間は $dp_L[2][3][0] + |S - X_1| + 5|G - X_2|$



最短時間は $dp_R[2][3][0] + |S - X_4| + 5|G - X_2|$



最短時間は $dp_L[2][3][1] + |S - X_1| + 5|G - X_3|$



最短時間は $dp_R[2][3][1] + |S - X_4| + 5|G - X_3|$

6 小課題 6 [累計 81 点]

4 パターンの最短時間が以下のように求まる！

全体計算量 $O(N^2 + Q \log N)$ で解ける

- $dp_L[l][r][d]$: 左端のボールを取ってスタートするときの最短時間
 - $dp_R[l][r][d]$: 右端のボールを取ってスタートするときの最短時間
- 前計算の DP ゴールの位置の二分探索

※ 二分探索の代わりに、各場所につき「どのボールの間か」を前計算すると、計算量 $O(N^2 + Q + L)$ になる

最短時間は $dp_L[2][3][0] + |S - X_1| + 5|G - X_2|$

最短時間は $dp_L[2][3][1] + |S - X_1| + 5|G - X_3|$



最短時間は $dp_R[2][3][0] + |S - X_4| + 5|G - X_2|$



最短時間は $dp_R[2][3][1] + |S - X_4| + 5|G - X_3|$

小課題 7 では、N が 500000 以下

制約から考えれば、例えば計算量 $O(N \log N)$ とかで解く必要がある

7

小課題 7 [累計 100 点]

小課題 6 では、 N が 500000 以下

残念ながら、 $O(N \log N)$ で

制約から考えれば、計算量 $O(N \log N)$ とかで解く必要がある

解くのは難しい

※ 解けるかもしれないが、仮にそうだとした場合も相当難易度が高いと思われる

本当に全ての制約を使い果たしたのか？

制約

- $1 \leq N \leq 500\,000$.
- $1 \leq L \leq 500\,000$.
- $0 \leq X_i \leq L$ ($1 \leq i \leq N$).
- $1 \leq Q \leq 500\,000$.
- $0 \leq S_j \leq L$ ($1 \leq j \leq Q$).
- $0 \leq G_j \leq L$ ($1 \leq j \leq Q$).
- $1 \leq T_j \leq 500\,000$ ($1 \leq j \leq Q$).
- 入力される値はすべて整数である.

7

小課題 7 [累計 100 点]

本当に全ての制約を使い果たしたのか？

制約

- $1 \leq N \leq 500\,000$.
- $1 \leq L \leq 500\,000$.
- $0 \leq X_i \leq L$ ($1 \leq i \leq N$).
- $1 \leq Q \leq 500\,000$.
- $0 \leq S_j \leq L$ ($1 \leq j \leq Q$).
- $0 \leq G_j \leq L$ ($1 \leq j \leq Q$).
- $1 \leq T_j \leq 500\,000$ ($1 \leq j \leq Q$).
- 入力される値はすべて整数である.

Q. 「T が 500000 以下」だと何か変わるのか？



Q. 「T が 500000 以下」だと何か変わるのか？

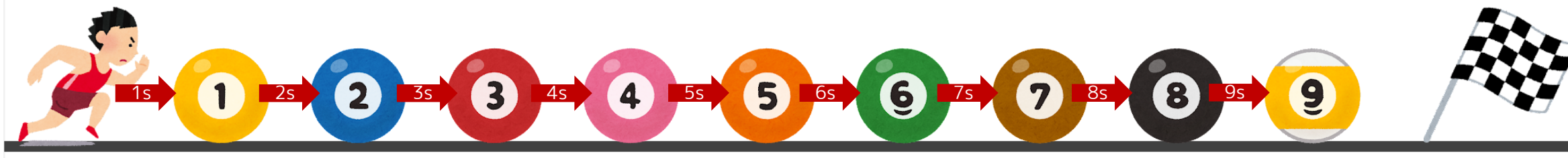
A. たくさんの種類の座標にボールが落ちていた場合、それを T 秒以内に全部回収するのは不可能になる！



例えば、下図の場合、9 個のボールを回収するには

$$2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 = 44$$

秒が必要



一般に、K 種類の場所にボールが落ちていた場合…

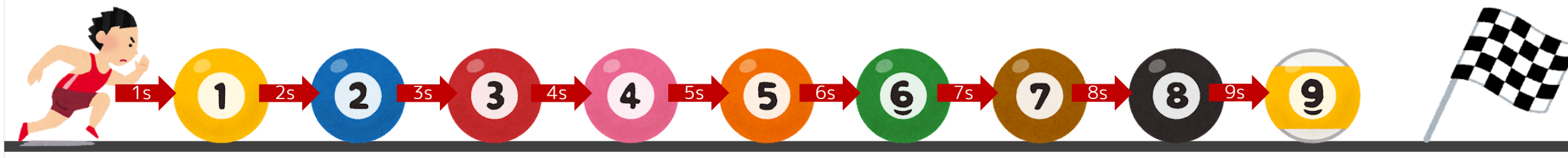
$$1 + (2+1) + (3+1) + \dots + (K+1)$$

最初のボールを
取る時間

次のボール
まで動く時間

ボールを
取る時間

秒が必要



7

小課題 7 [累計 100 点]

一般に、 K 種類の場所にボールが落ちていた場合...

ボールの場所が 999 種類以上なら
秒が必要

答えは全部「No」！！！！



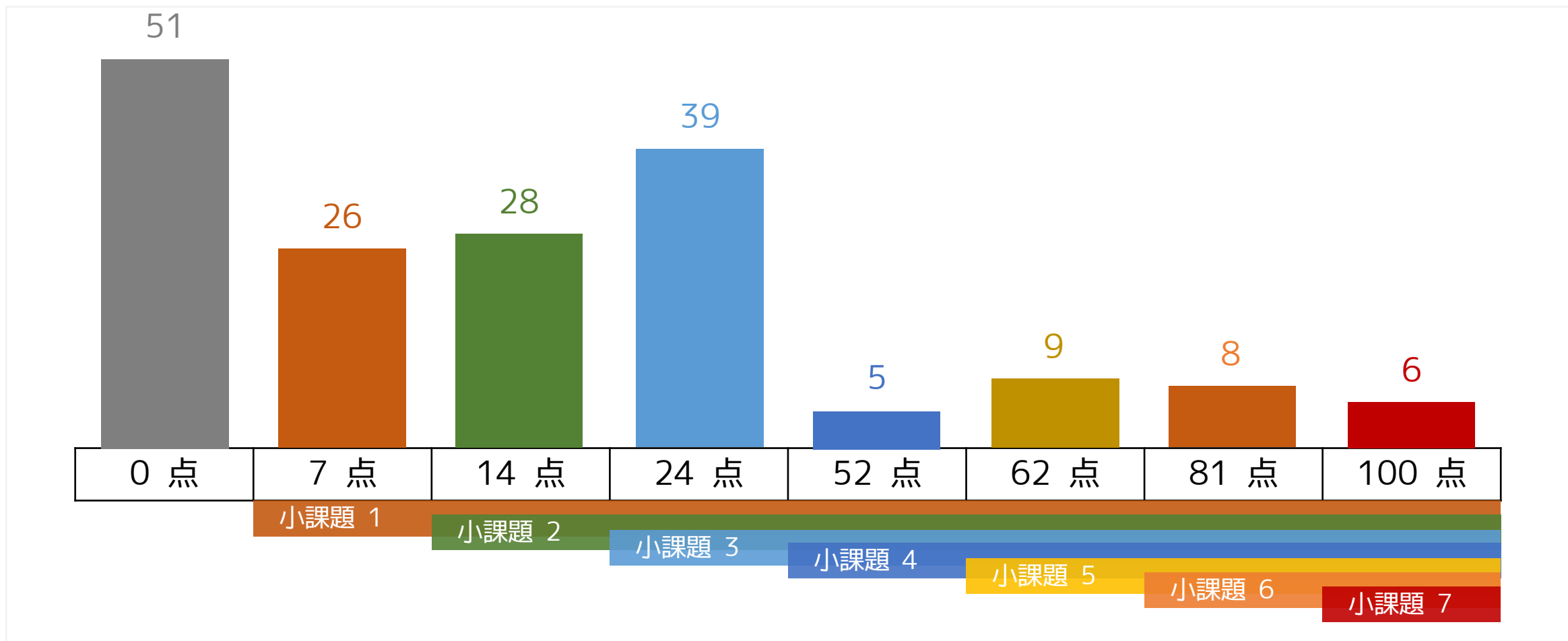
小課題 6 の解法を、同じ場所にあるものをまとめて考えた「重み付き」の問題として実装する（同じ座標なら一気に取るのが最適なことに注意）

ボールの場所が K 種類ある場合、計算量は $O(N + Q + K^2 + L)$

ここで $K = O(\sqrt{\max T})$ なので、全体計算量は...

$$O(N + Q + \max T)$$

※ ここで $L \leq \max T$ とみなしてよい。理由は、最左から最右のボールまで行くのに T 分かかるため。





December

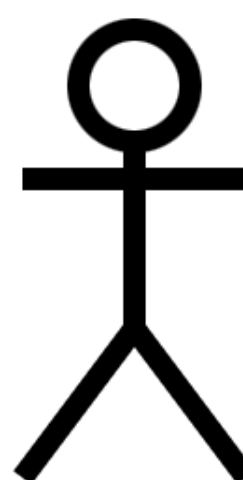
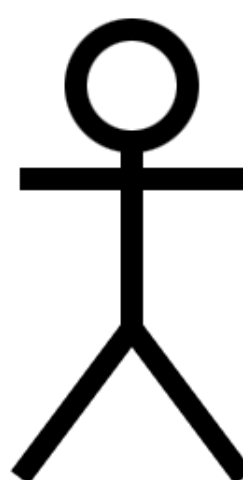
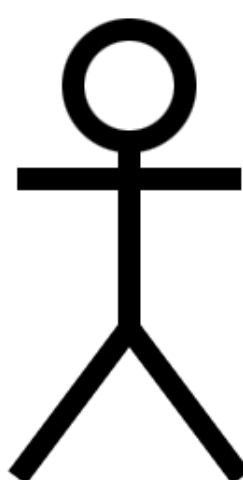
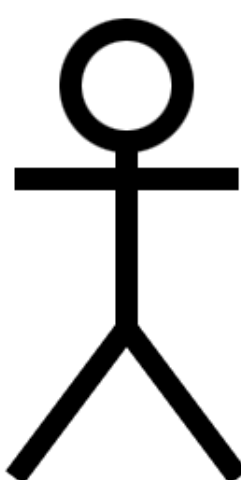
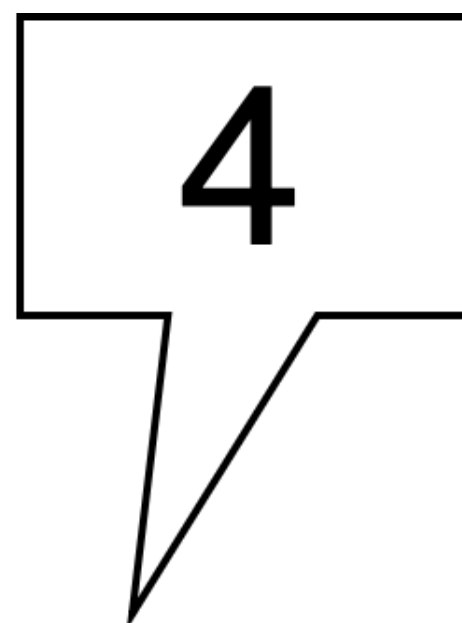
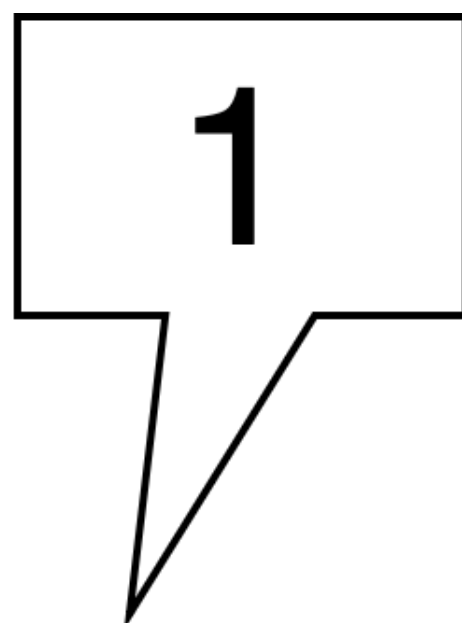
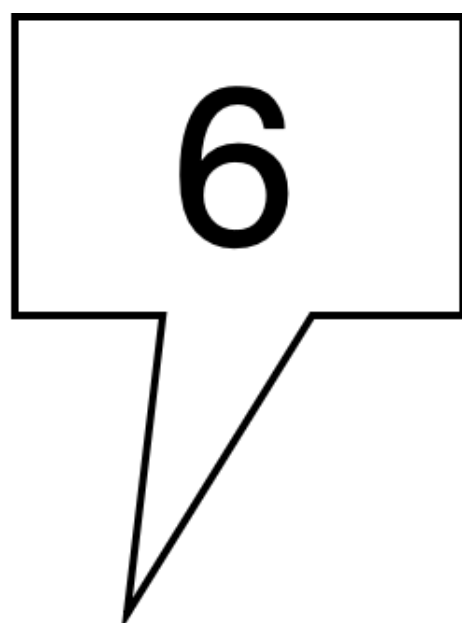
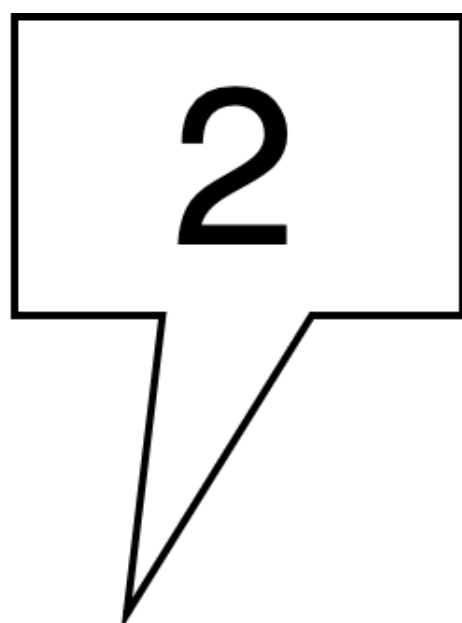
プレゼント交換 解説

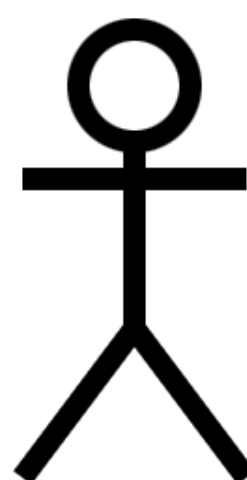
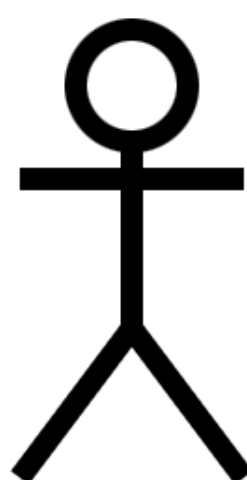
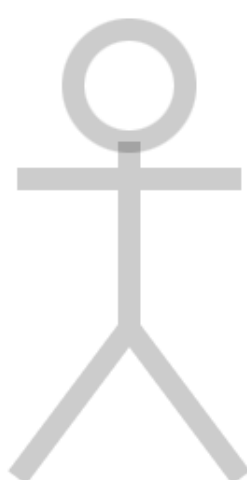
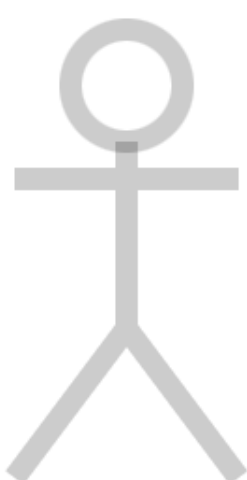
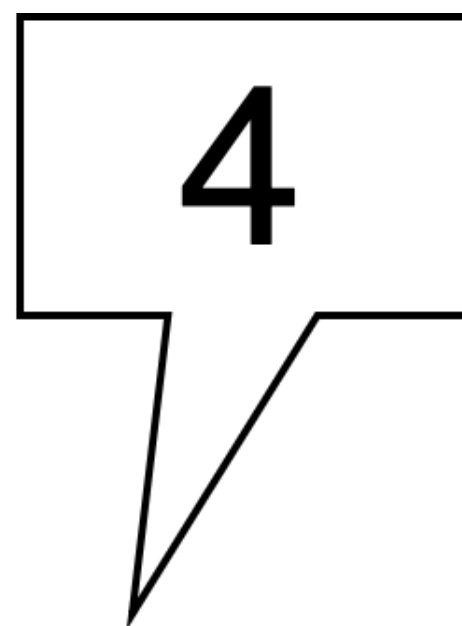
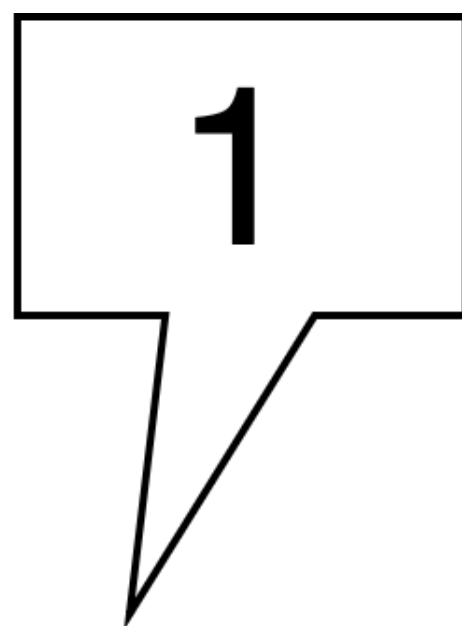
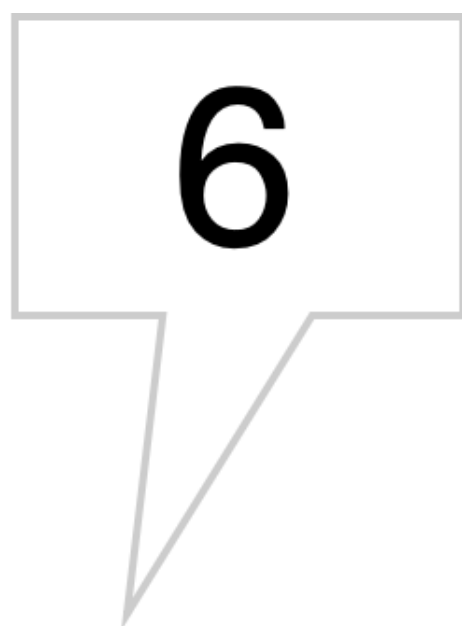
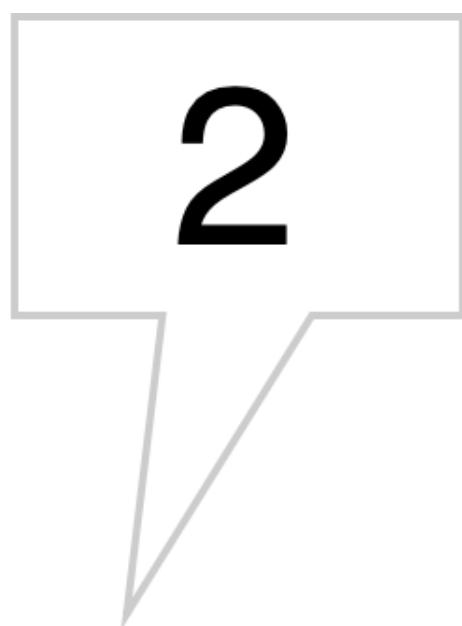
JOI'23 本選 4

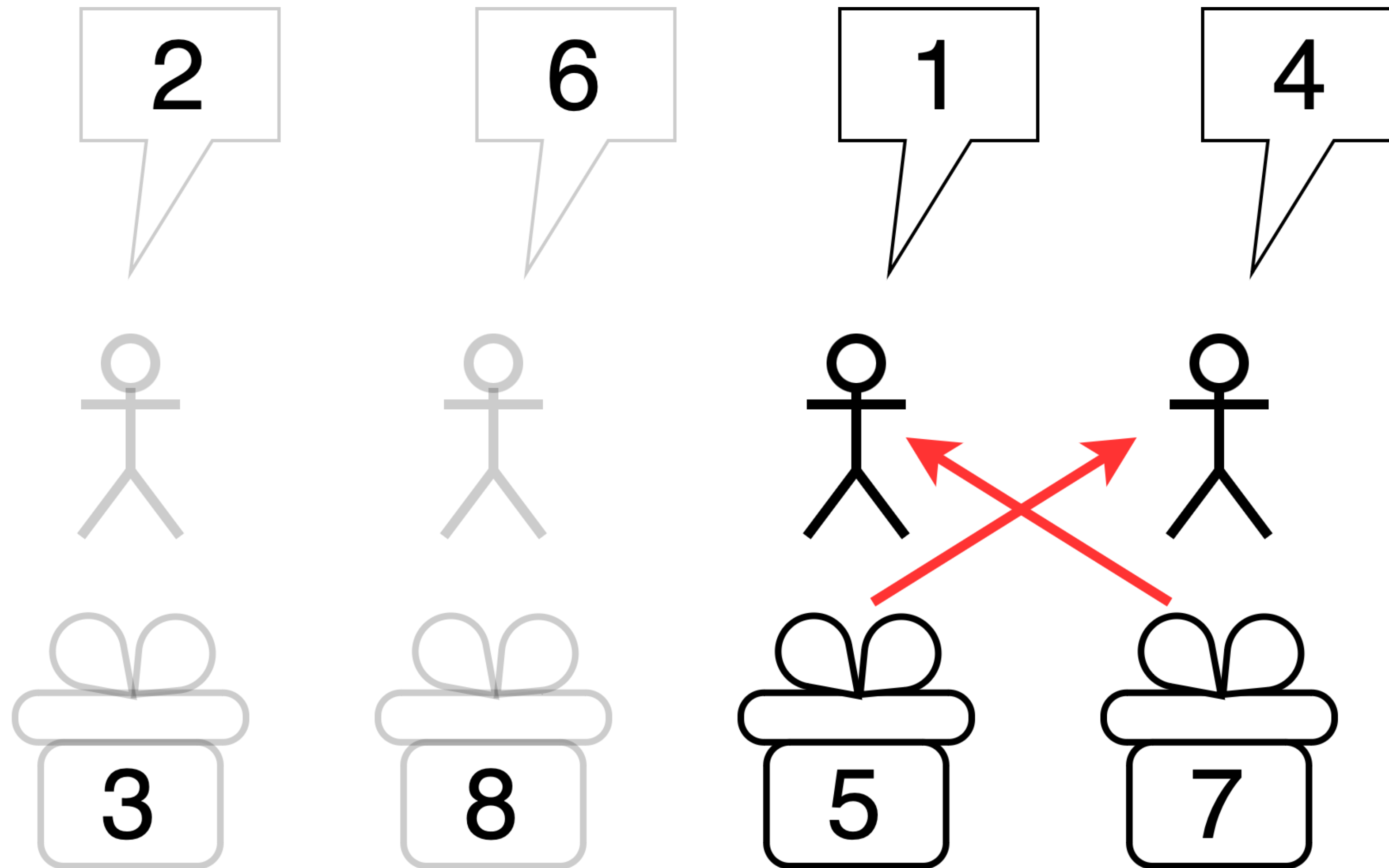
渡邊 雄斗 (yuto1115)

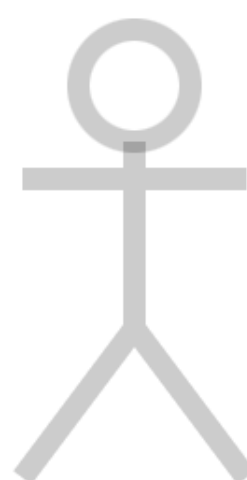
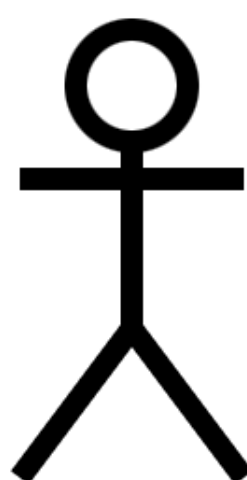
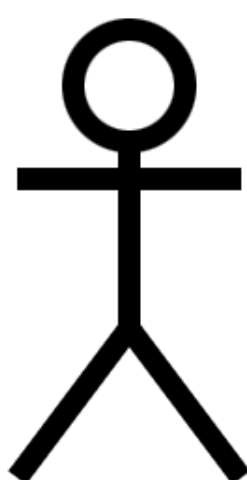
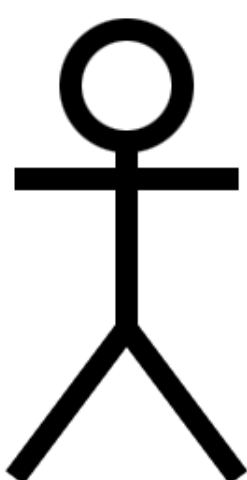
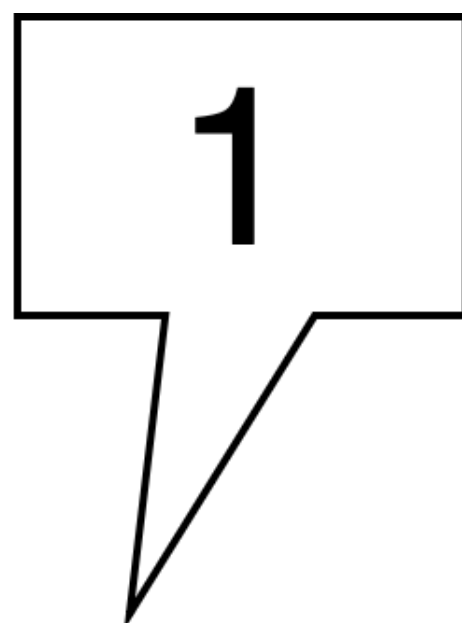
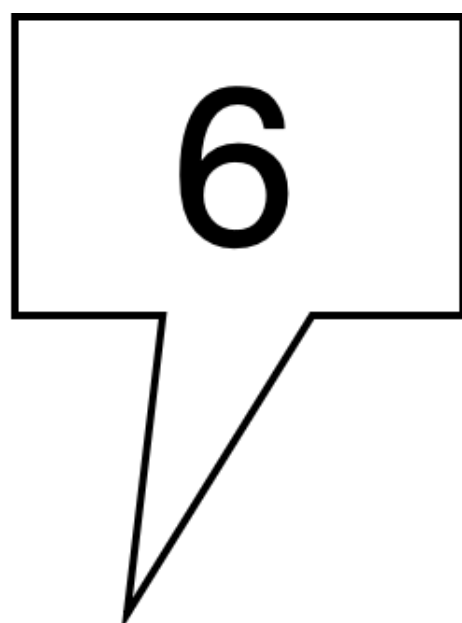
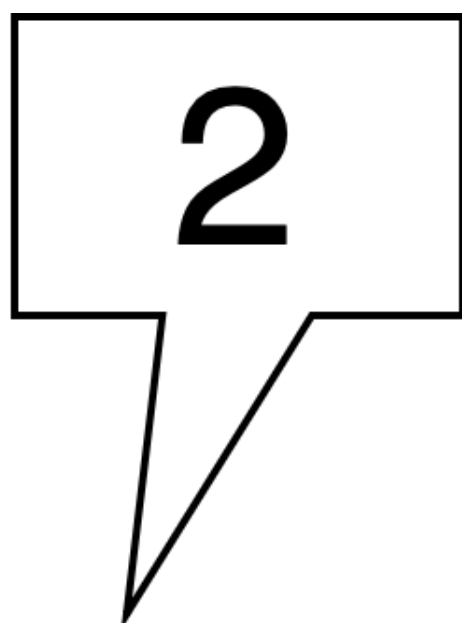
問題概要

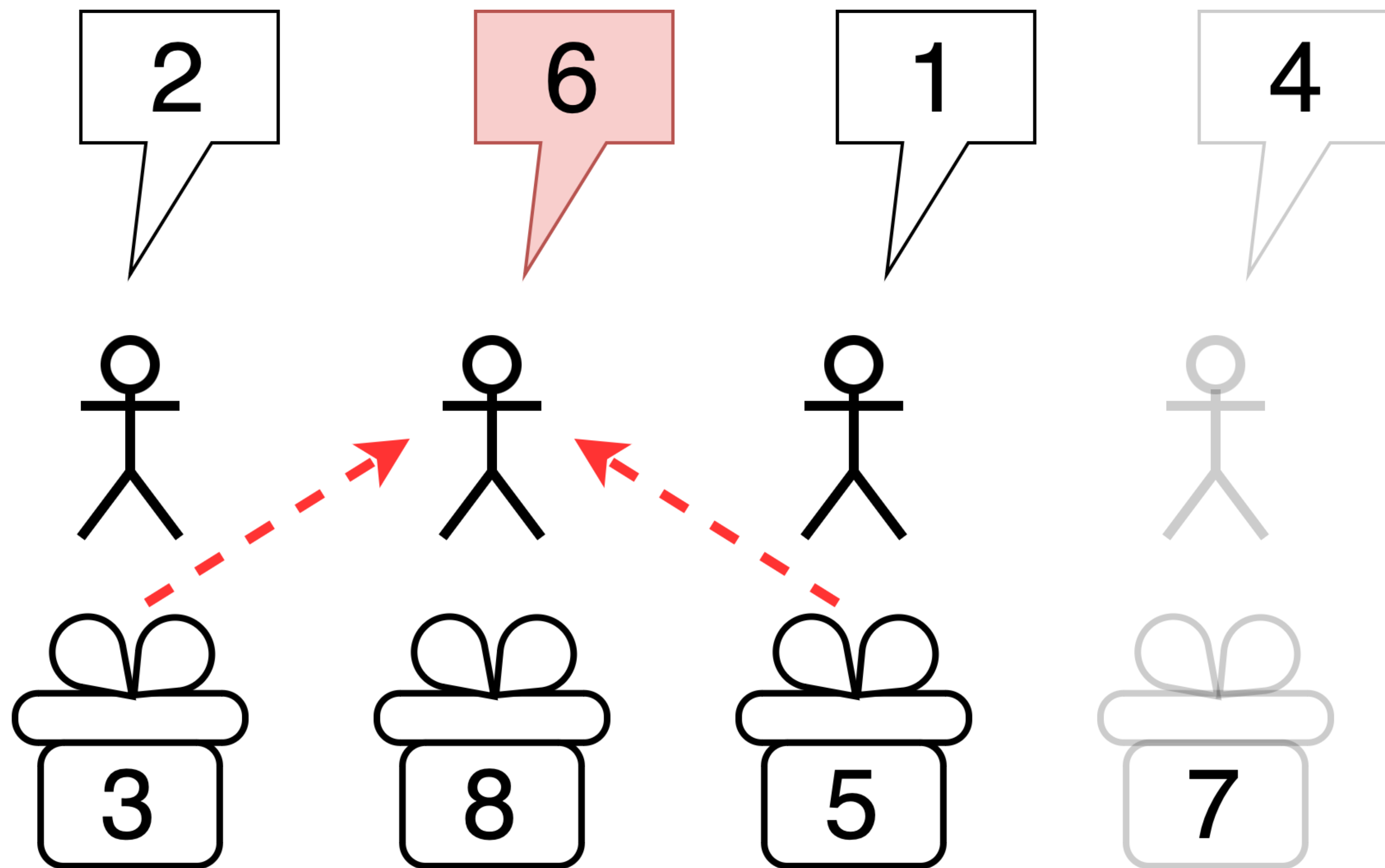
- N 人の生徒がプレゼントを持っている. 生徒 i のプレゼントの価値は A_i .
- 生徒 i は価値 B_i ($< A_i$) 以上のプレゼントしか受け取りたくない.
- Q 個のグループがある. j 個目のグループは生徒 $L_j, L_j + 1, \dots, R_j$ からなる.
- それぞれのグループについて, そのグループ内でプレゼント交換できるか判定せよ.
 - 価値 B_i 未満のプレゼントや, 自分自身のプレゼントは受け取れない.

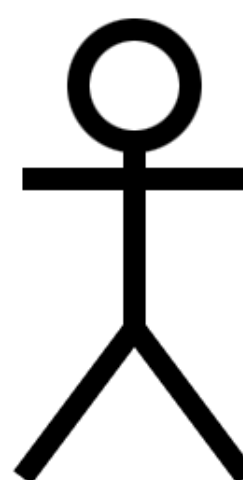
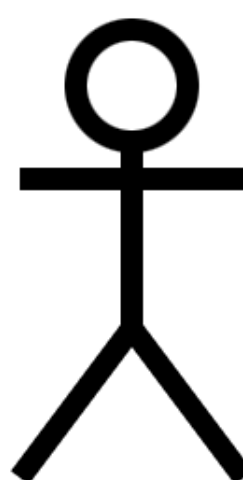
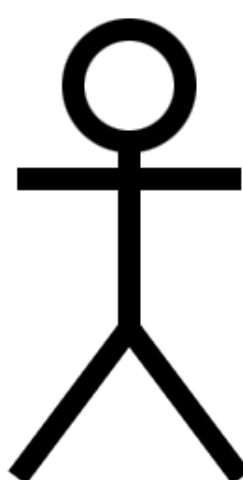
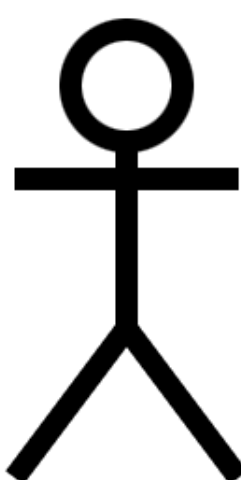
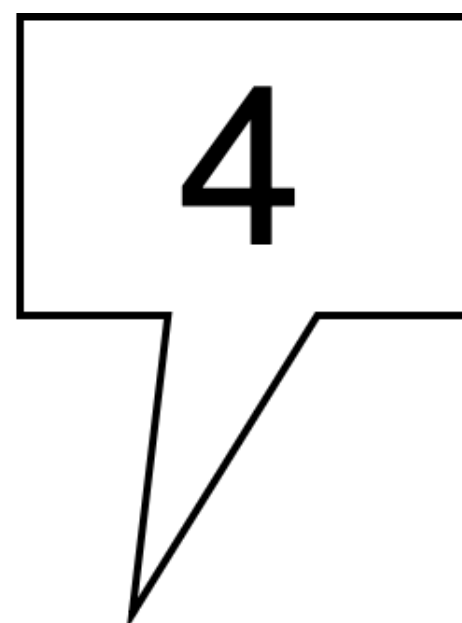
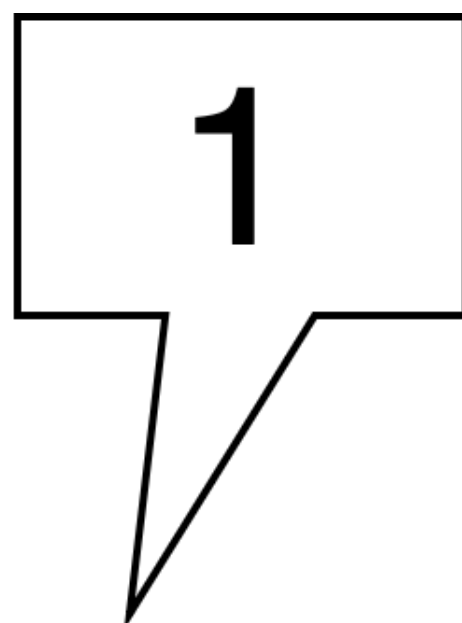
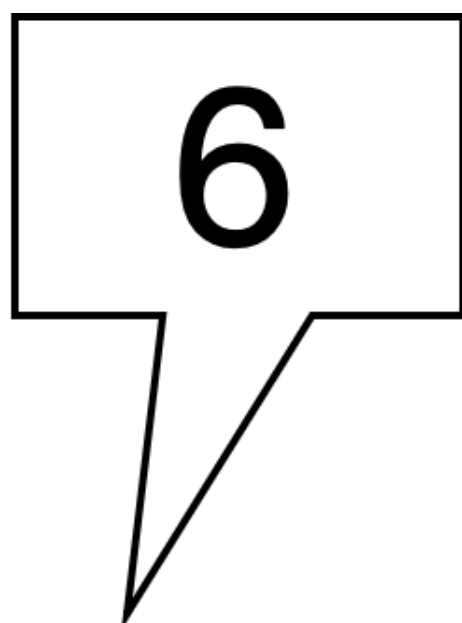
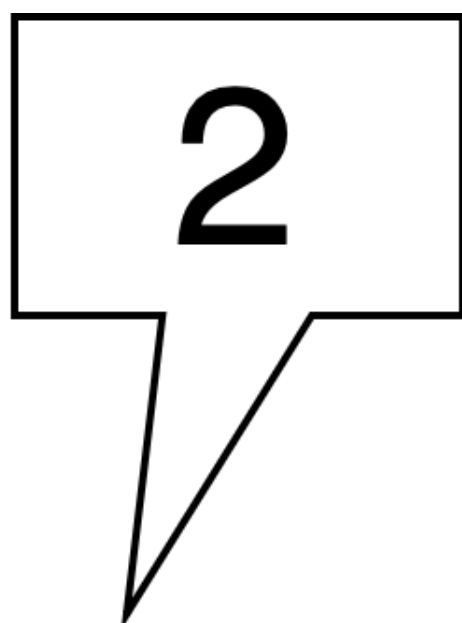


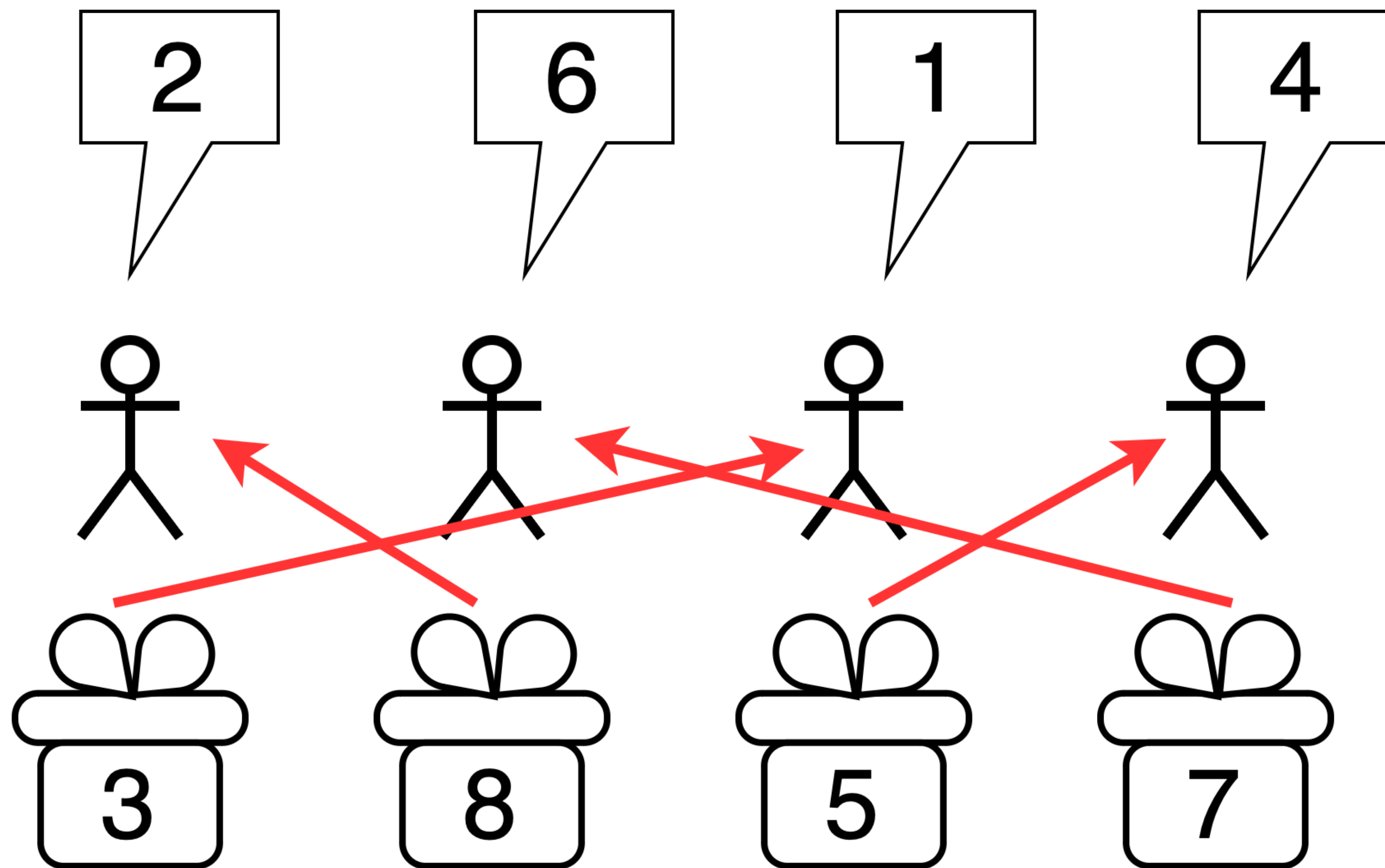












小課題 1 (4 点)

- $N \leq 10, Q \leq 10$
- 各グループについて, (グループの人数)! 通りある交換方法を全部試し, 条件を満たすものがあるか判定する.
- 計算量 $O(QN!)$

小課題 2 (5 点, 累計 9 点)

- $N \leq 18, Q \leq 10$
- 各グループについて, bit DP を用いて交換方法を効率的に全探索する.
- 計算量 $O(Q2^N)$

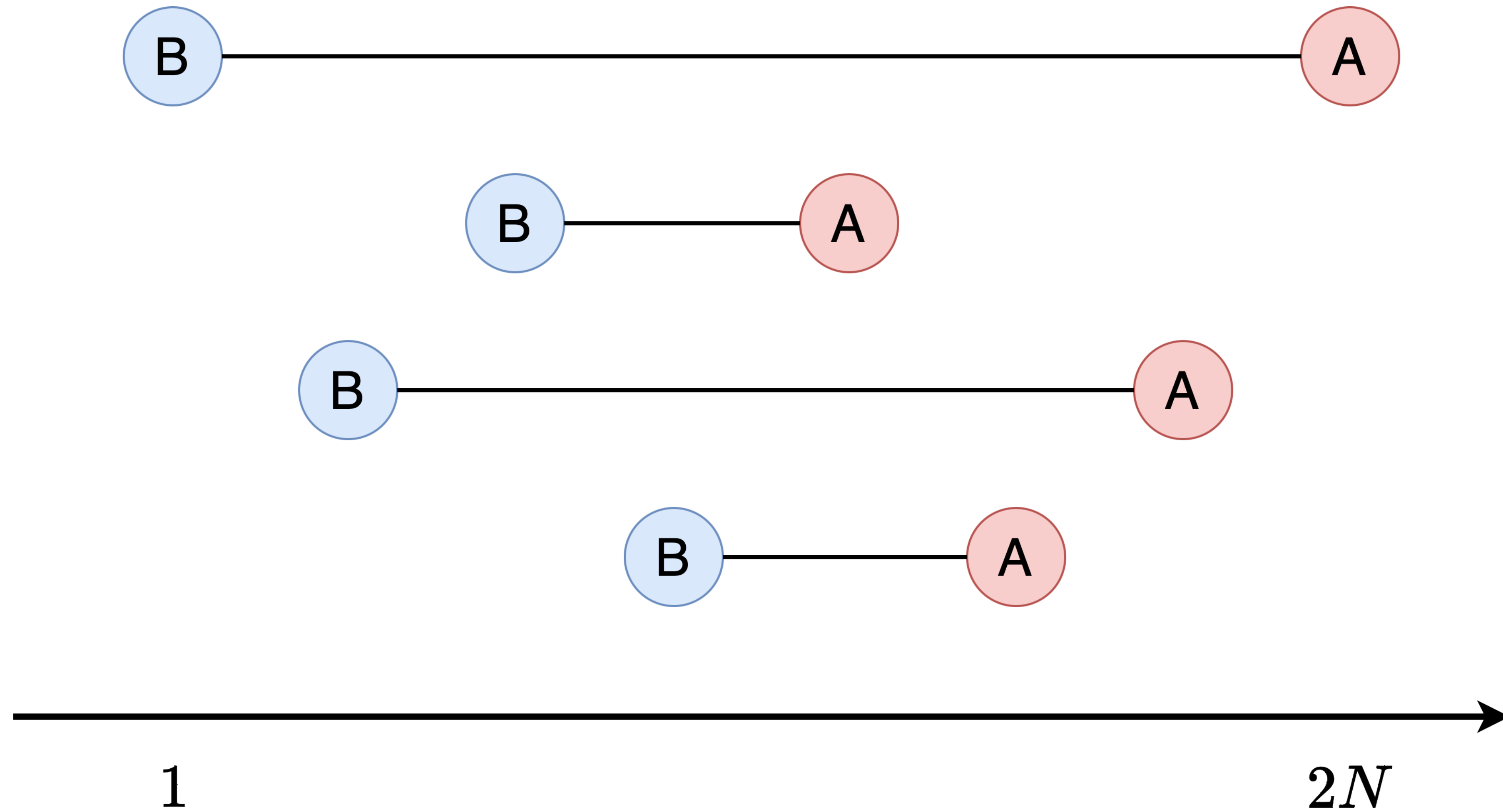
小課題 3 (10 点)

- $N \leq 10^5$, $A_1 \geq 2N - 2$, $B_1 = 1$, $Q = 1$, $L_1 = 1$, $R_1 = N$
- グループは, N 人全員からなる 1 つだけ
- N 人全員でプレゼント交換できるかを $O(N)$ くらいで判定したい
- $A_1 \geq 2N - 2$, $B_1 = 1$ ← だから何？

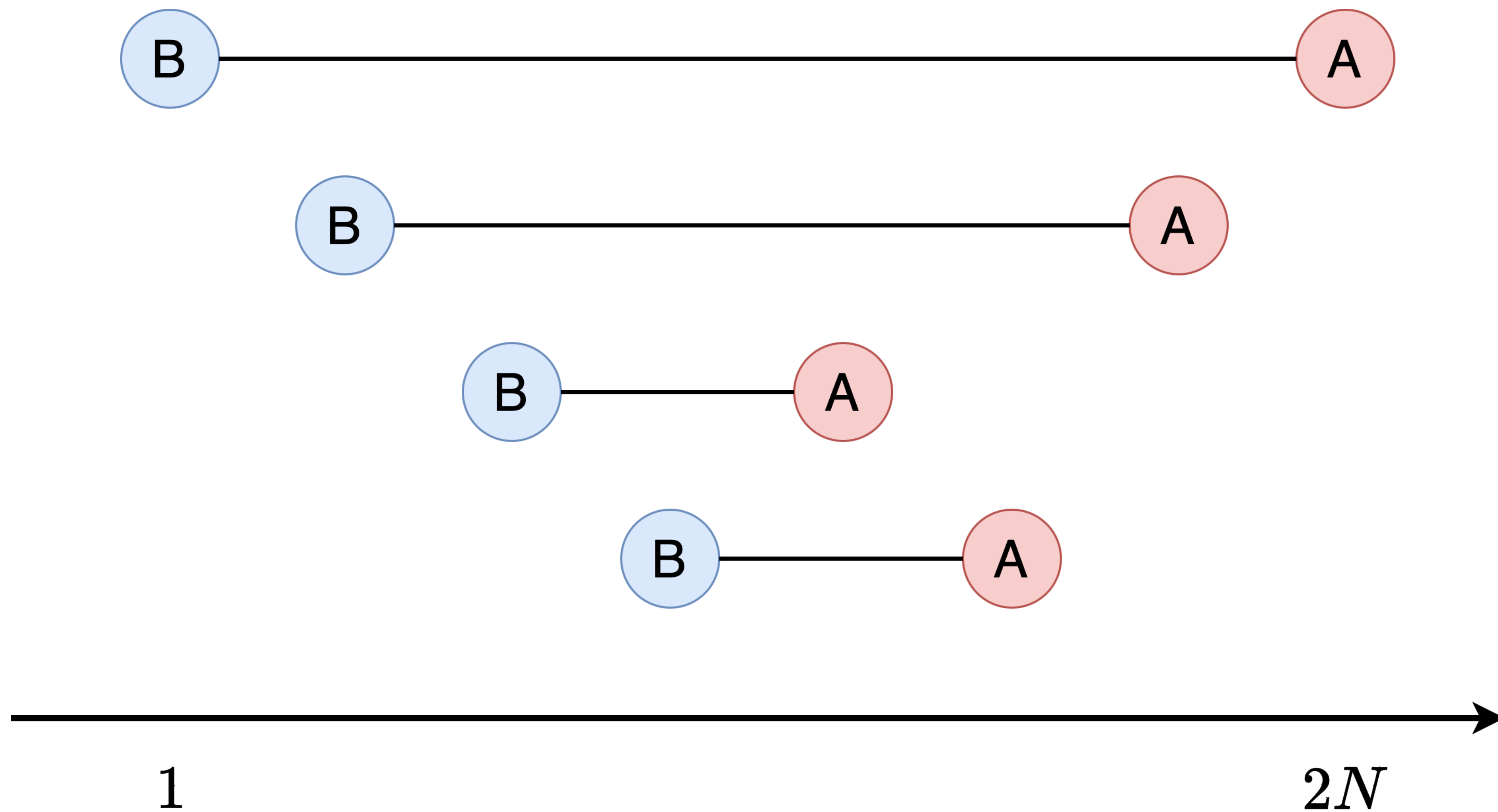
小課題3 (10 点)

- $A_1 \geq 2N - 2$, $B_1 = 1 \leftarrow$ だから何？
- A_1 としてあり得る値は $2N - 2$, $2N - 1$, $2N$ のみ
- まずは $A_1 = 2N$ の場合を考えてみる

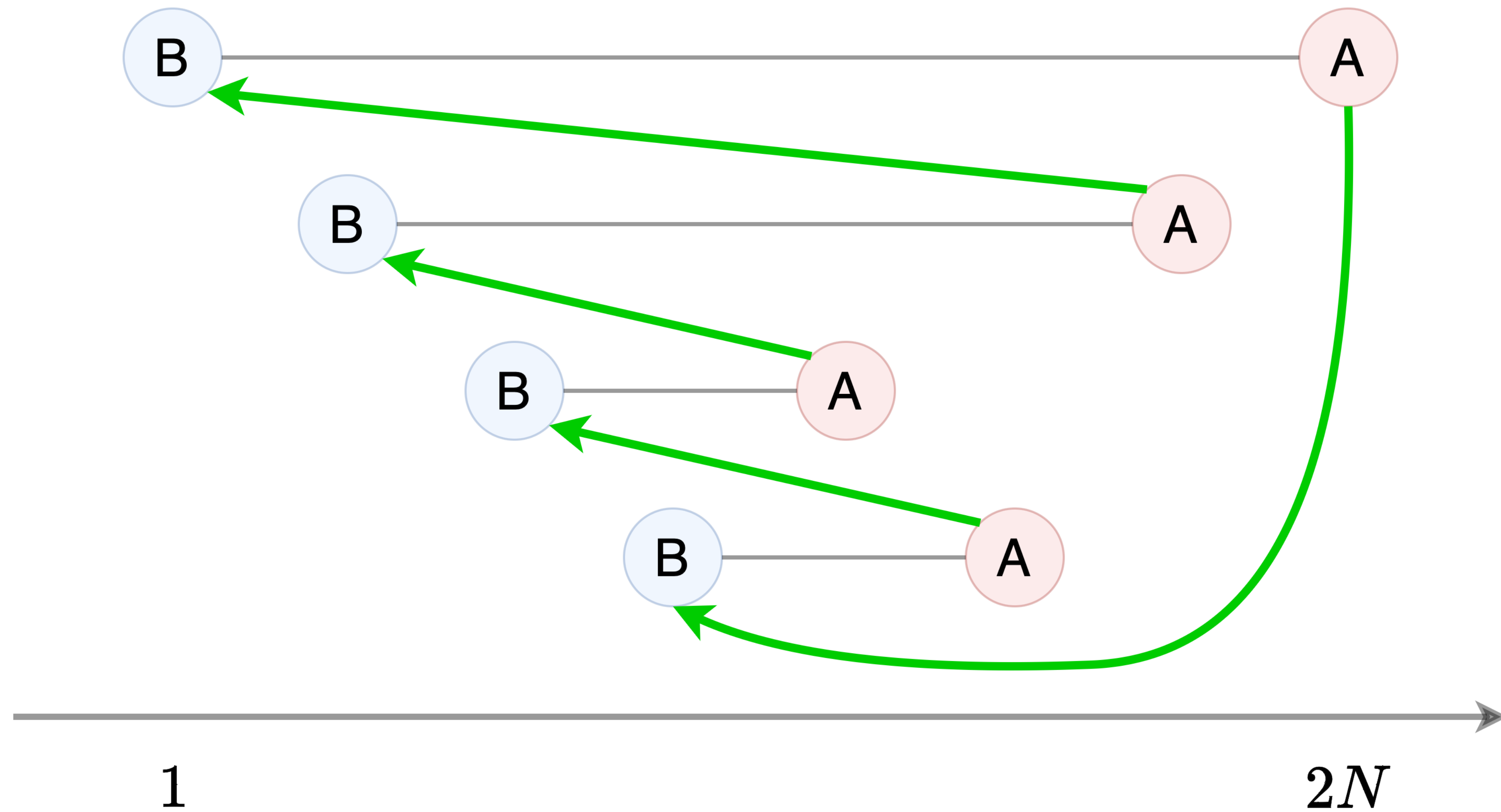
図の導入



B の昇順に並び替え



1 個ずつずらして渡す



小課題 3 (10 点)

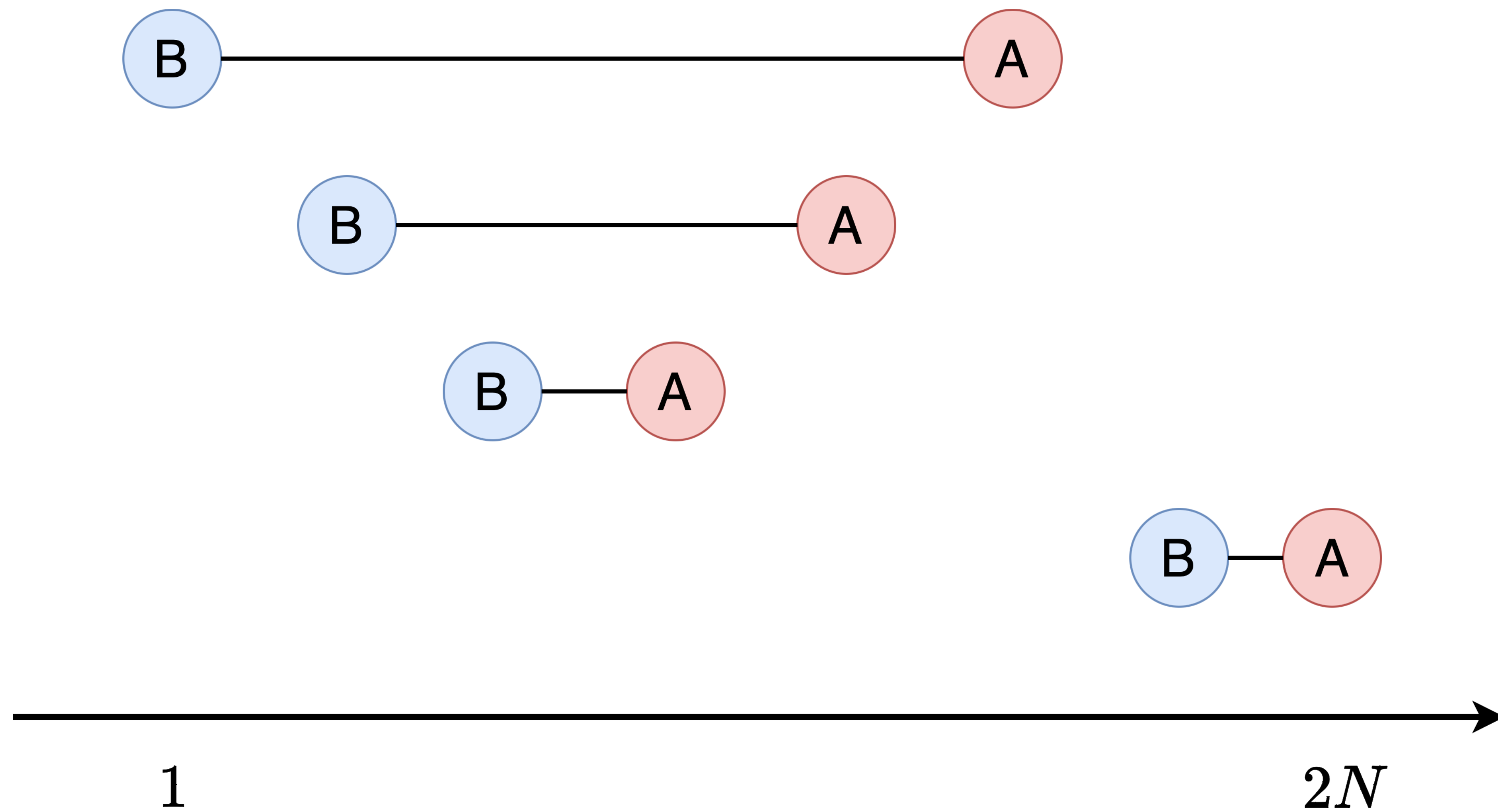
- 「 B_i の昇順にソートして, 1 個ずつずらして渡す」戦略をとれば,
 B_i の最大値が A_1 より小さいとき, 必ずプレゼント交換可能
- これは $A_1 \geq 2N - 1$ ならば常に成立する
- $A_1 = 2N - 2$ のときは?
 - $(A, B) = (2N, 2N - 1)$ なるペアが存在するときに限り不成立
 - このとき, 明らかにプレゼント交換不可能

小課題 3 (10 点)

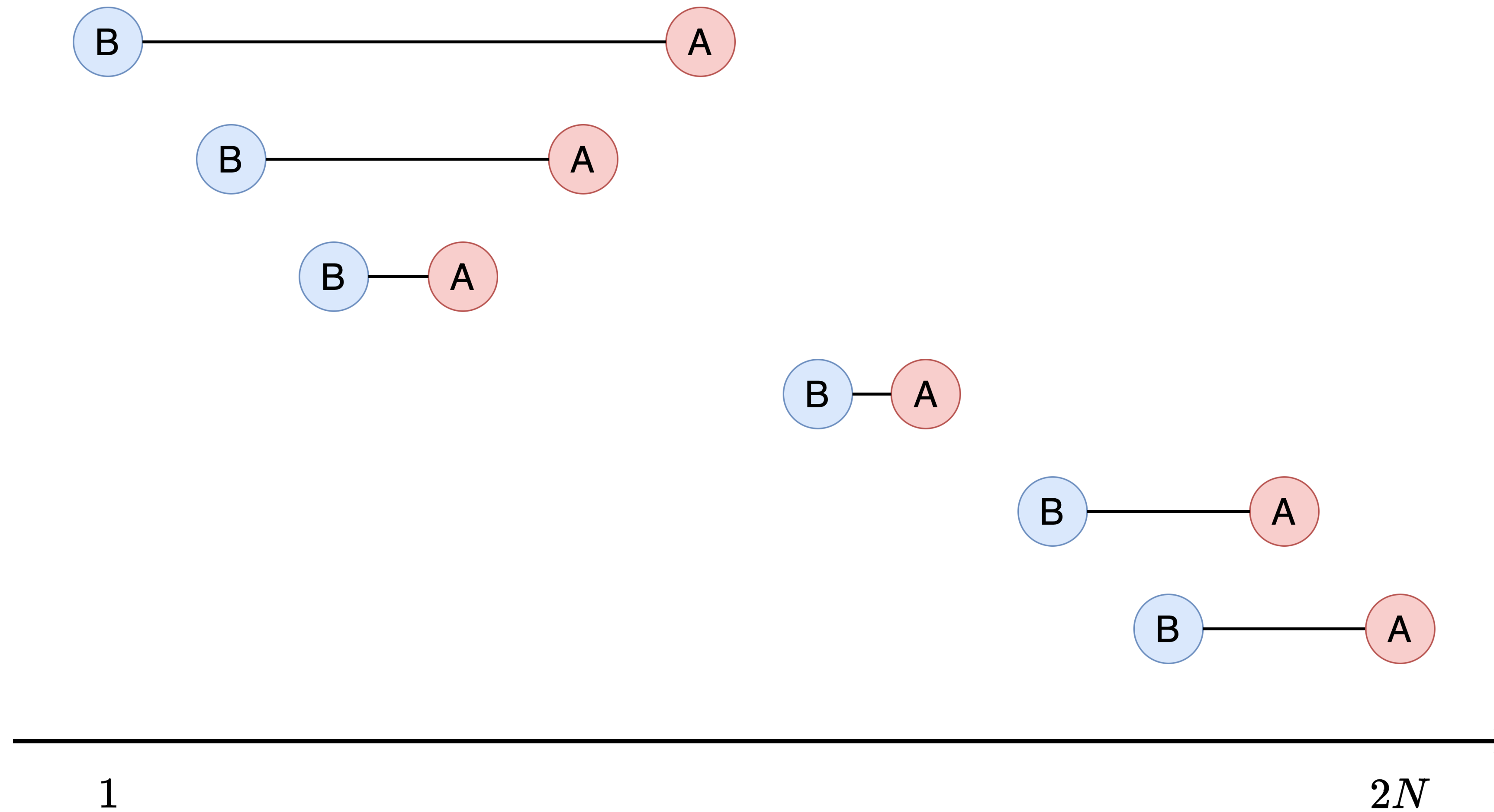
- よって, $(A, B) = (2N, 2N - 1)$ なるペアが存在するか探せばいい
- 計算量 $O(N)$

もっと一般化できないか？

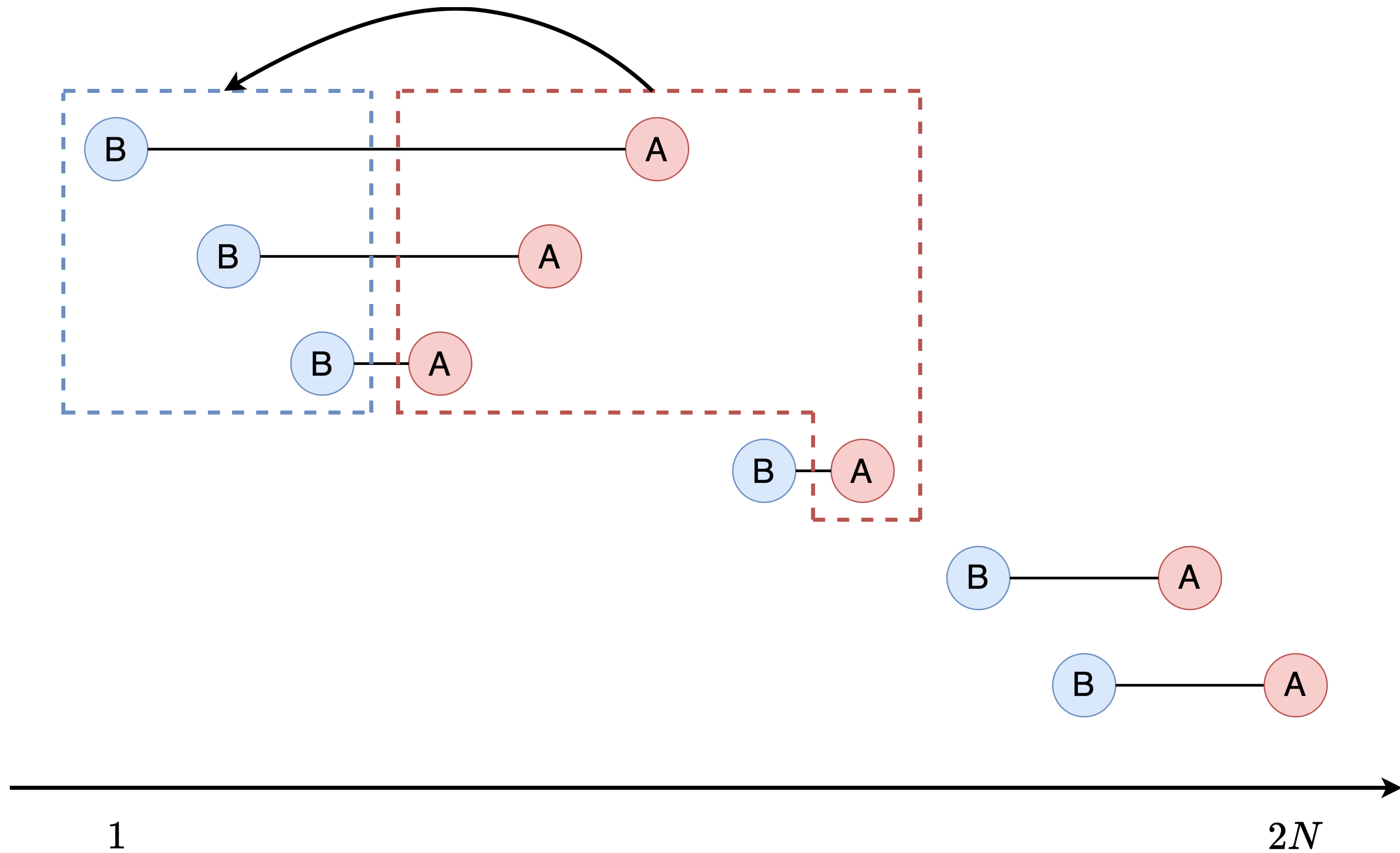
ダメなパターン



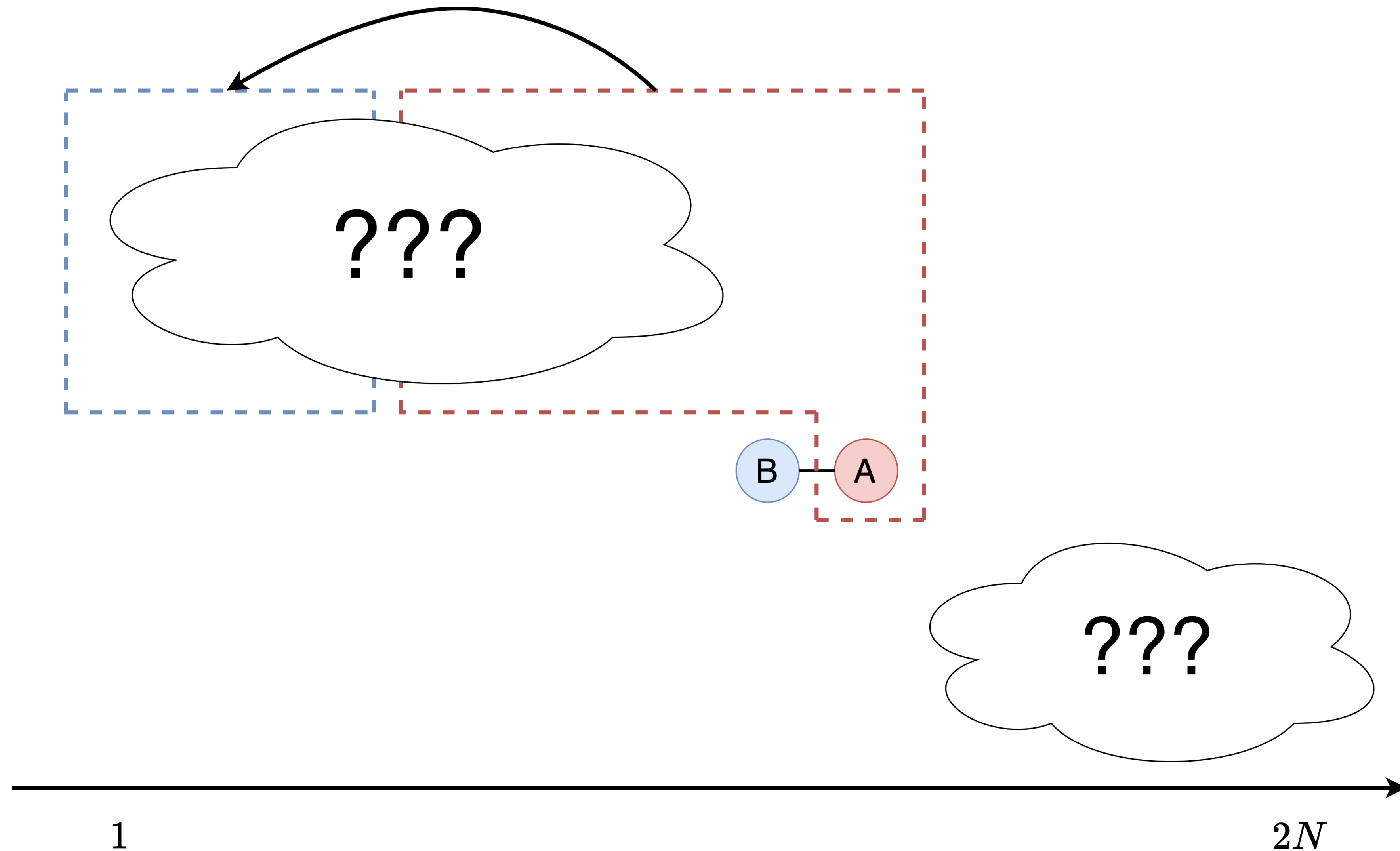
これはどうか？



やっぱりダメ



もっと一般的にこれはダメ



つまり

- ある区間を境に「それより完全に左にある区間」「完全に右にある区間」に二分されてしまう場合、プレゼント交換は不可能
- 言い換えると、**どの区間も、他のどれかしらの区間と共通部分を持つ必要がある**
- 実は十分

証明

1. マッチングと捉えて Hall の結婚定理を適用

- 知識が必要なので, 詳細略
- 証明に使えるだけでなく, 最初から Hall で考えると自然にこの条件が出てくる

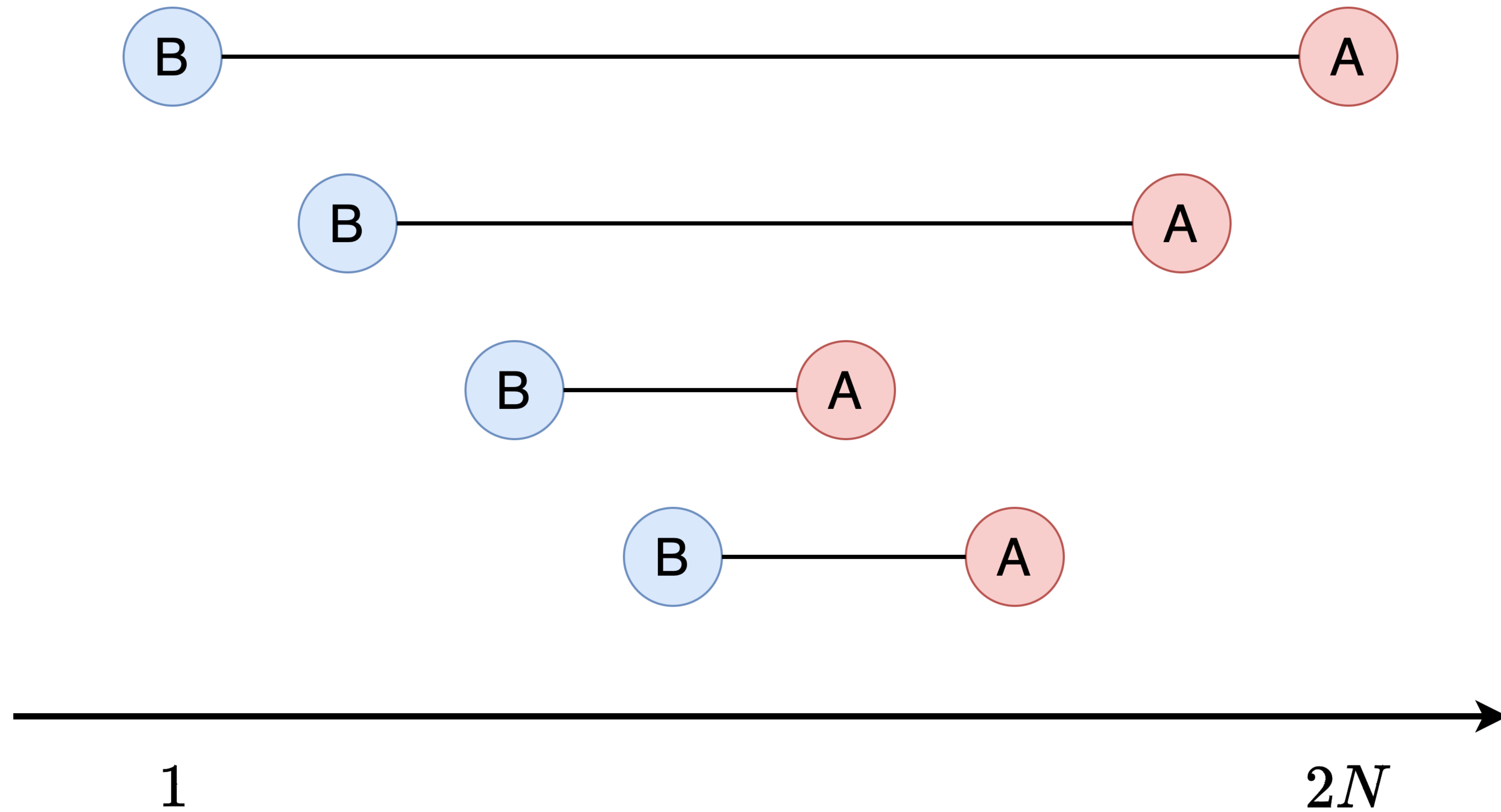
2. 貪欲法を用いる

- どの区間も他のどれかしらの区間と共通部分を持つとき, 実際にプレゼント交換をする方法を構築できればよい.
- 正しい貪欲法を用いれば, 可能

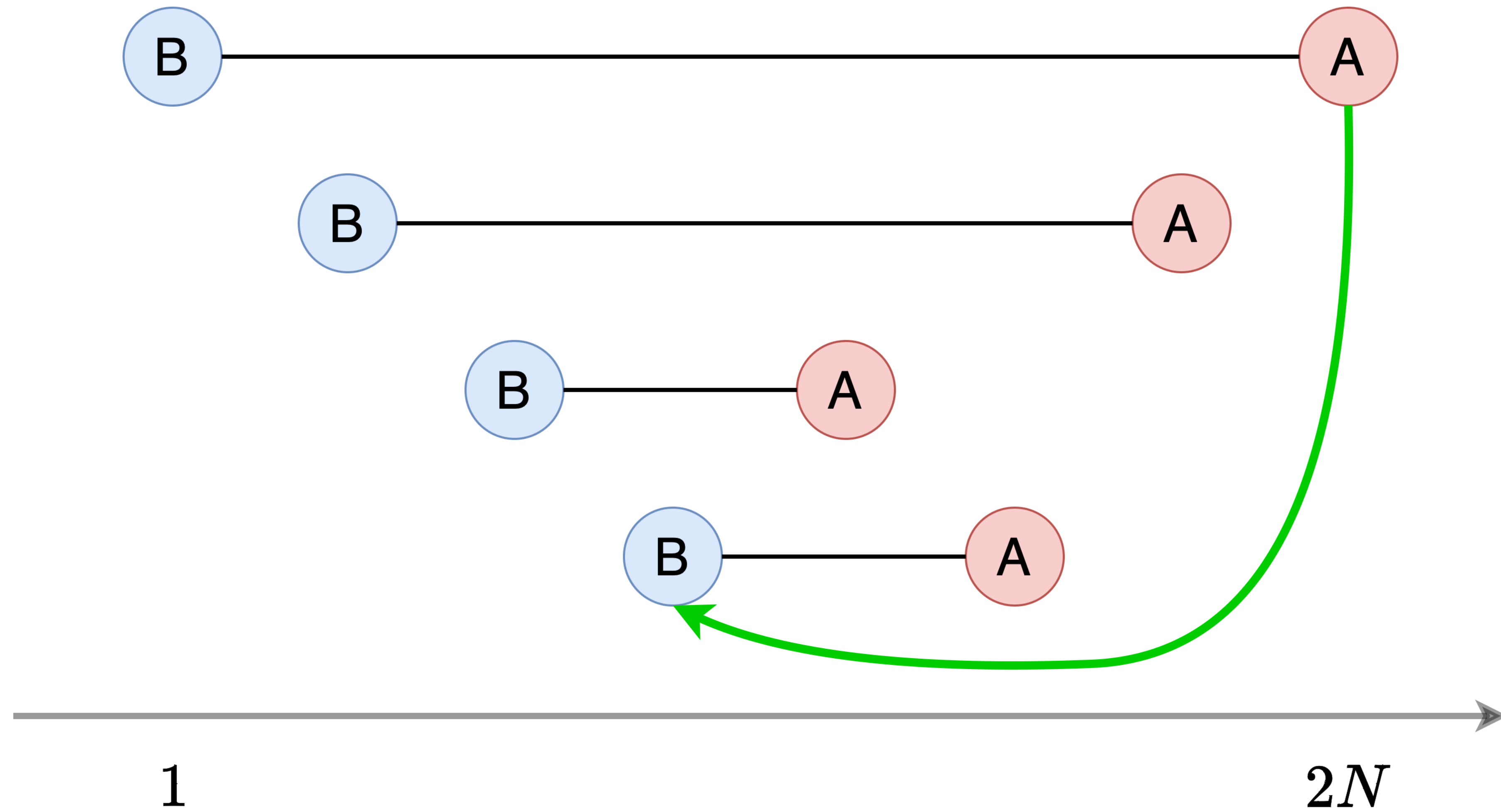
正しい貪欲法

- グループは N 人の生徒全員からなり, $B_1 < B_2 < \dots < B_N$ が成り立つと仮定する.
 - そうでなければソート等によって適切に変形する.
- $i = 1, 2, \dots, N$ の順に以下を行う:
 - 生徒 i のプレゼントを受け取ることができ, まだ誰のプレゼントを受け取っていない人のうち, B_i が最も大きい人を選んで生徒 i のプレゼントを渡す.
 - そもそもそんな人が 1 人もいないなら, プレゼント交換は不可能.

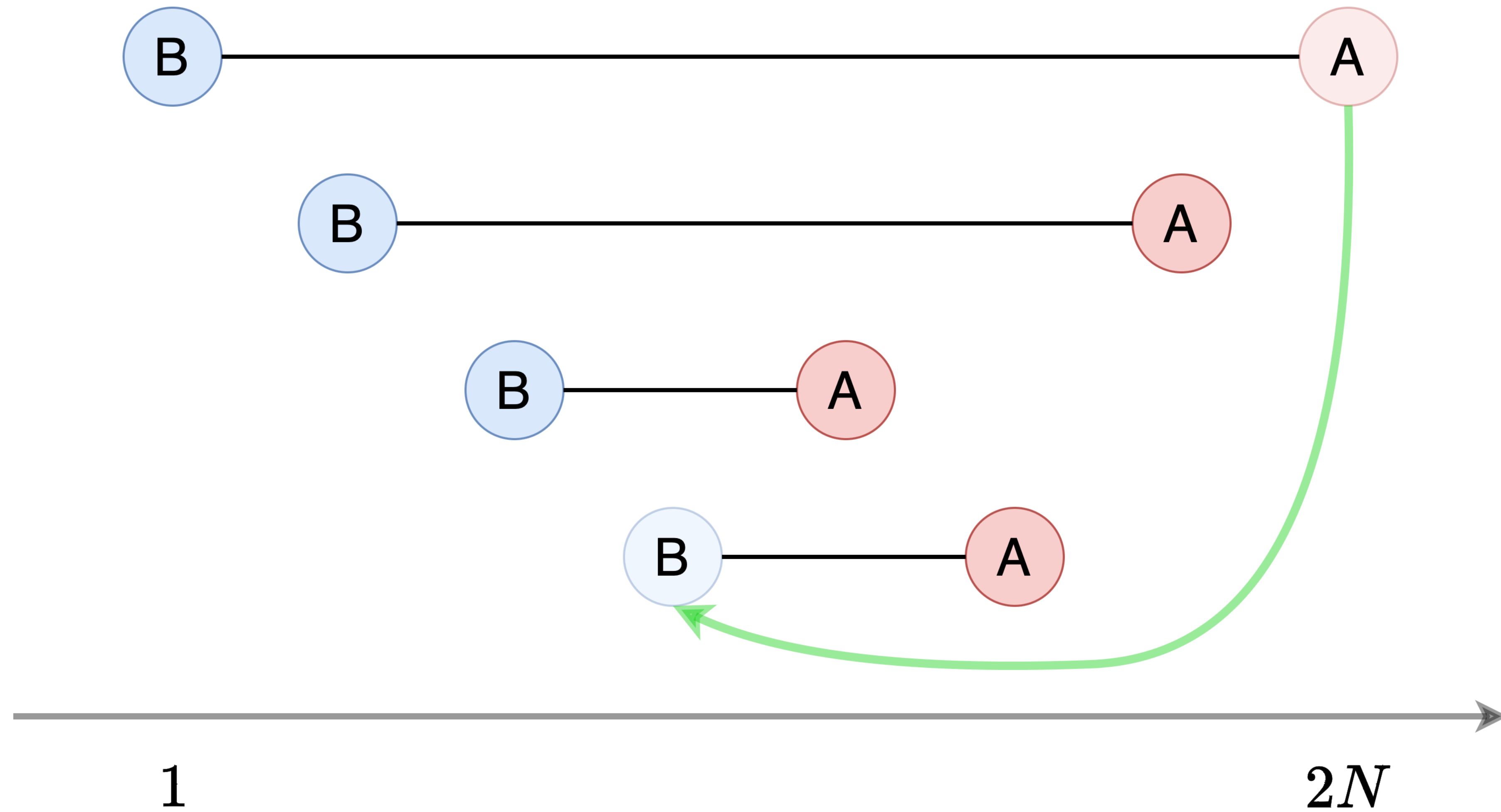
正しい貪欲法



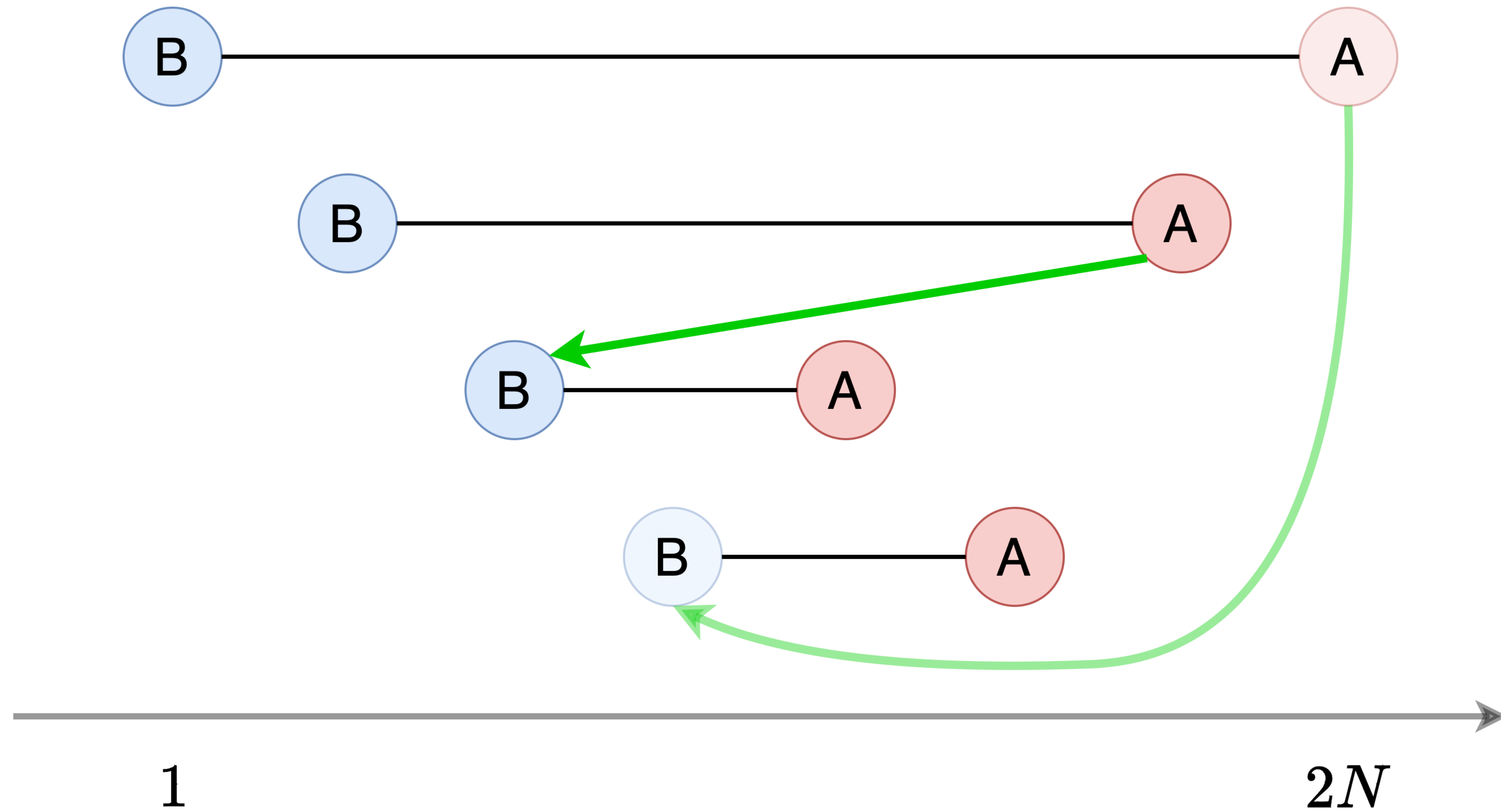
正しい貪欲法



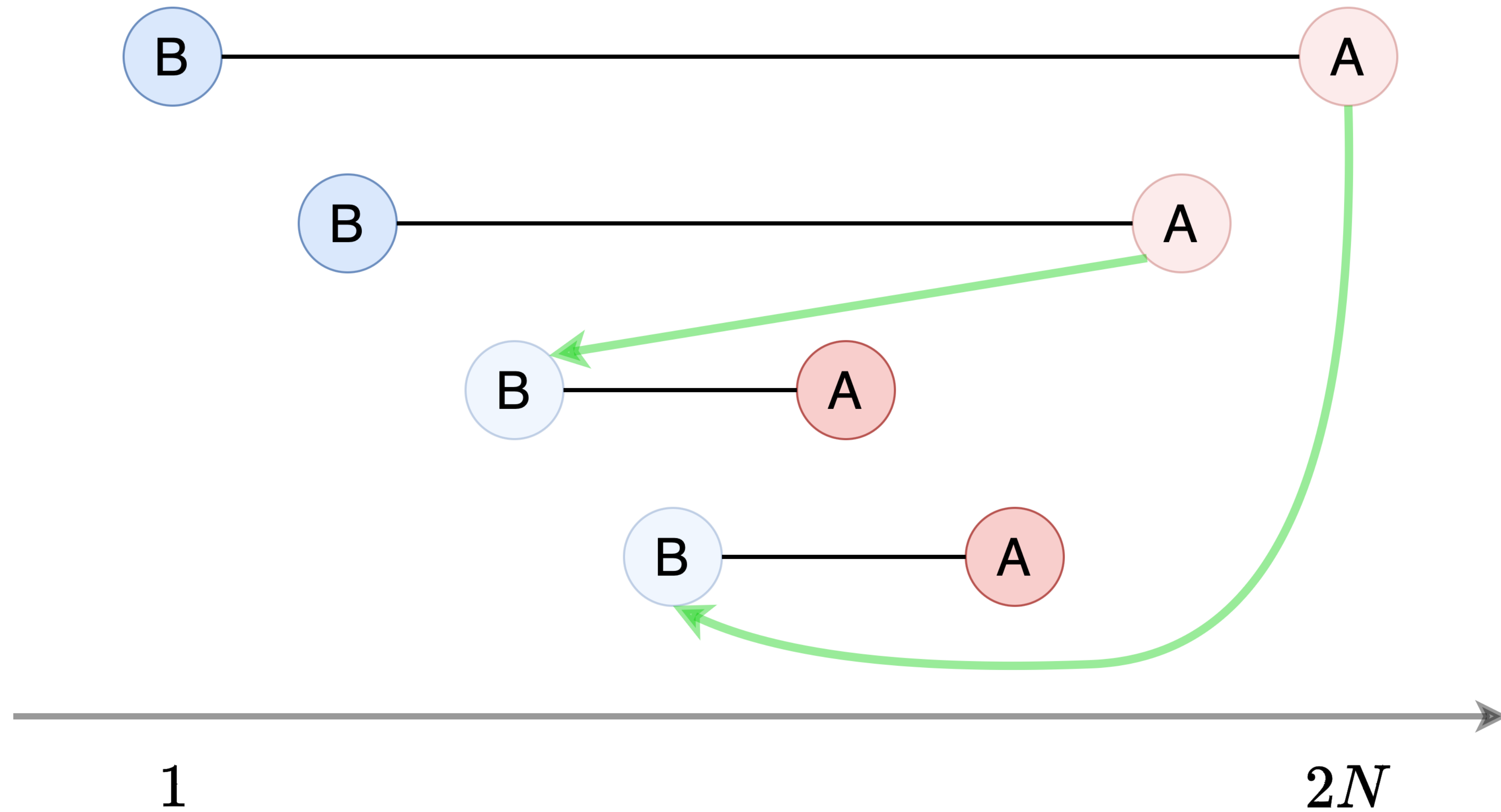
正しい貪欲法



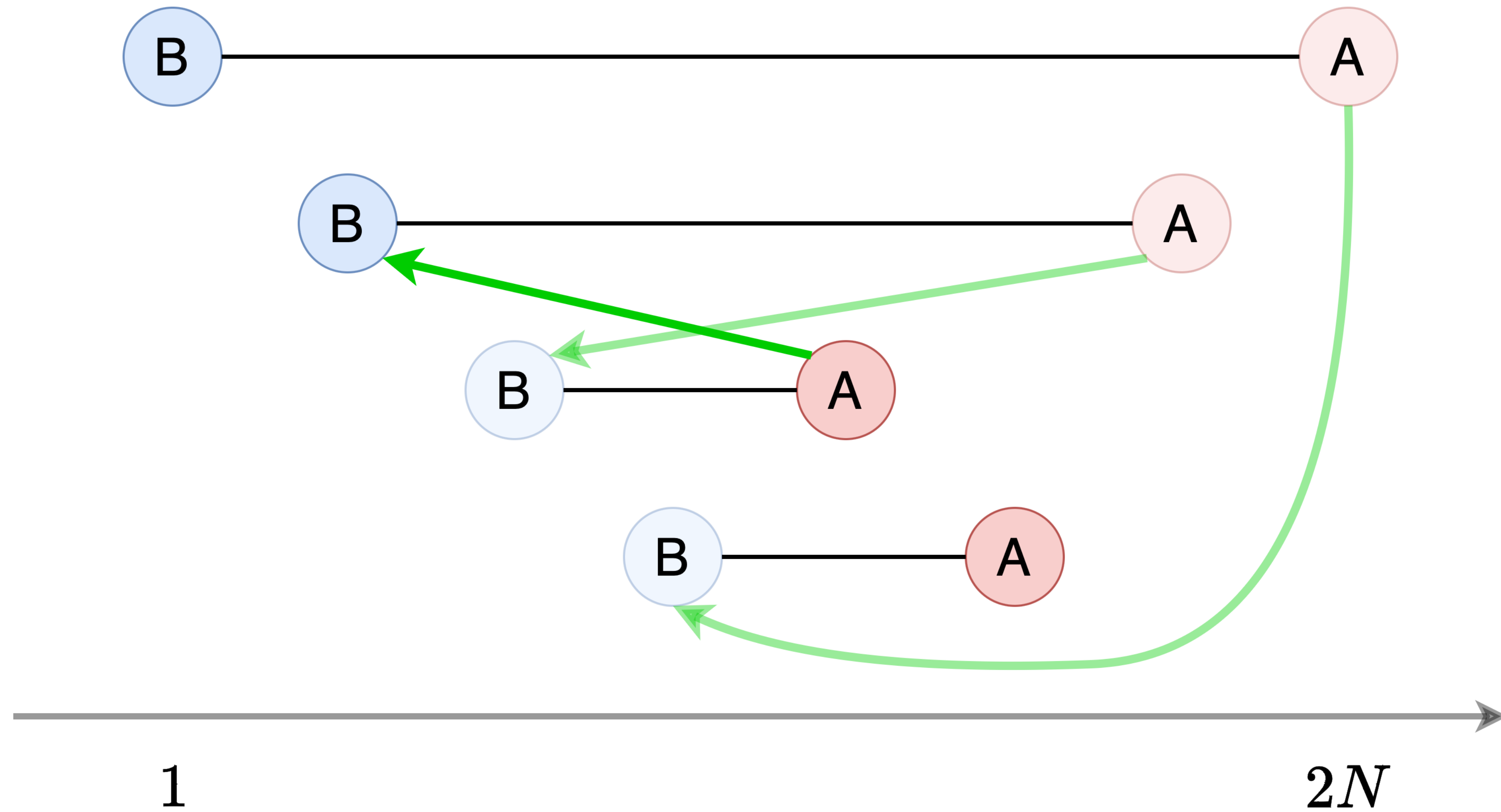
正しい貪欲法



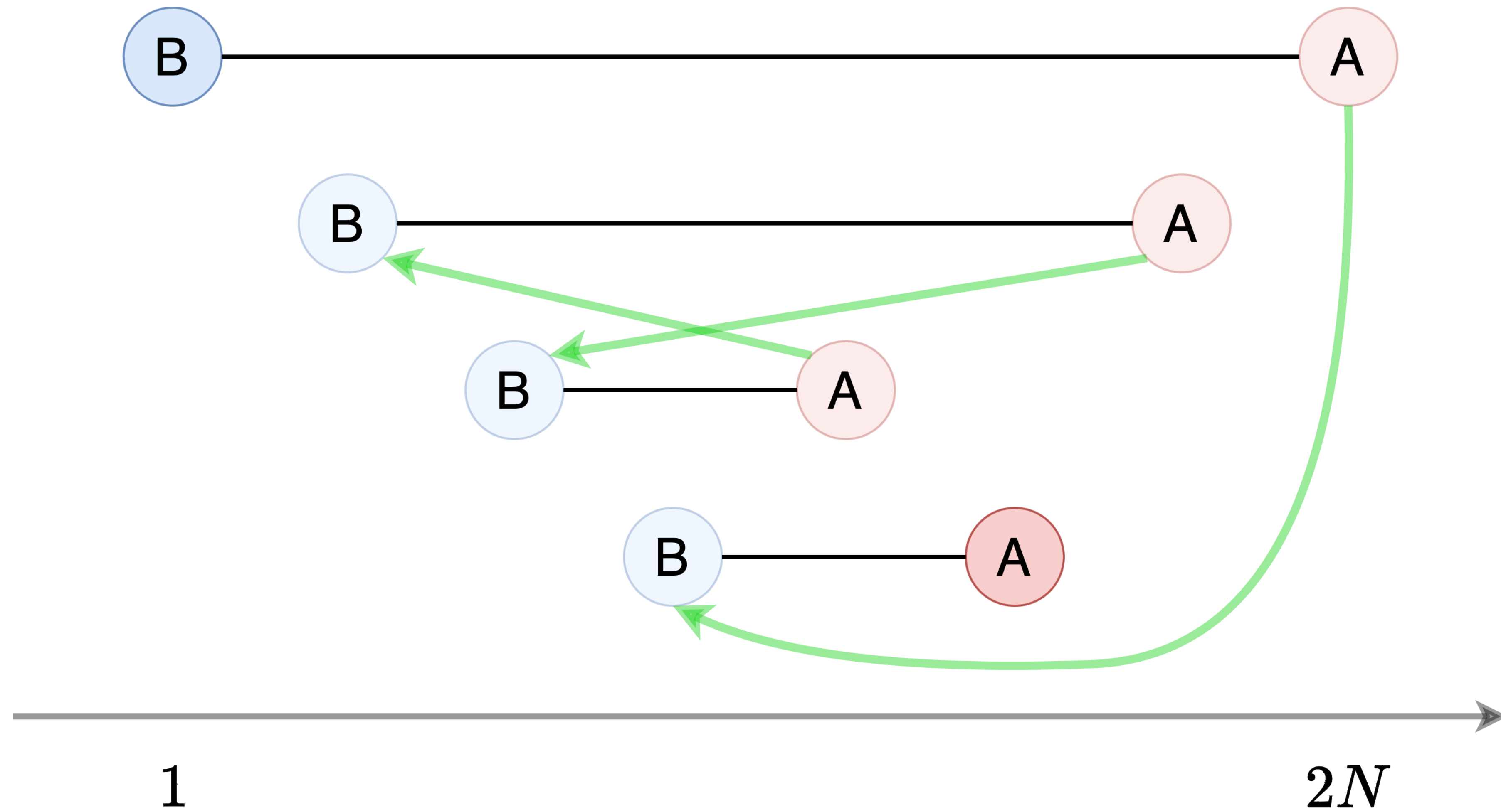
正しい貪欲法



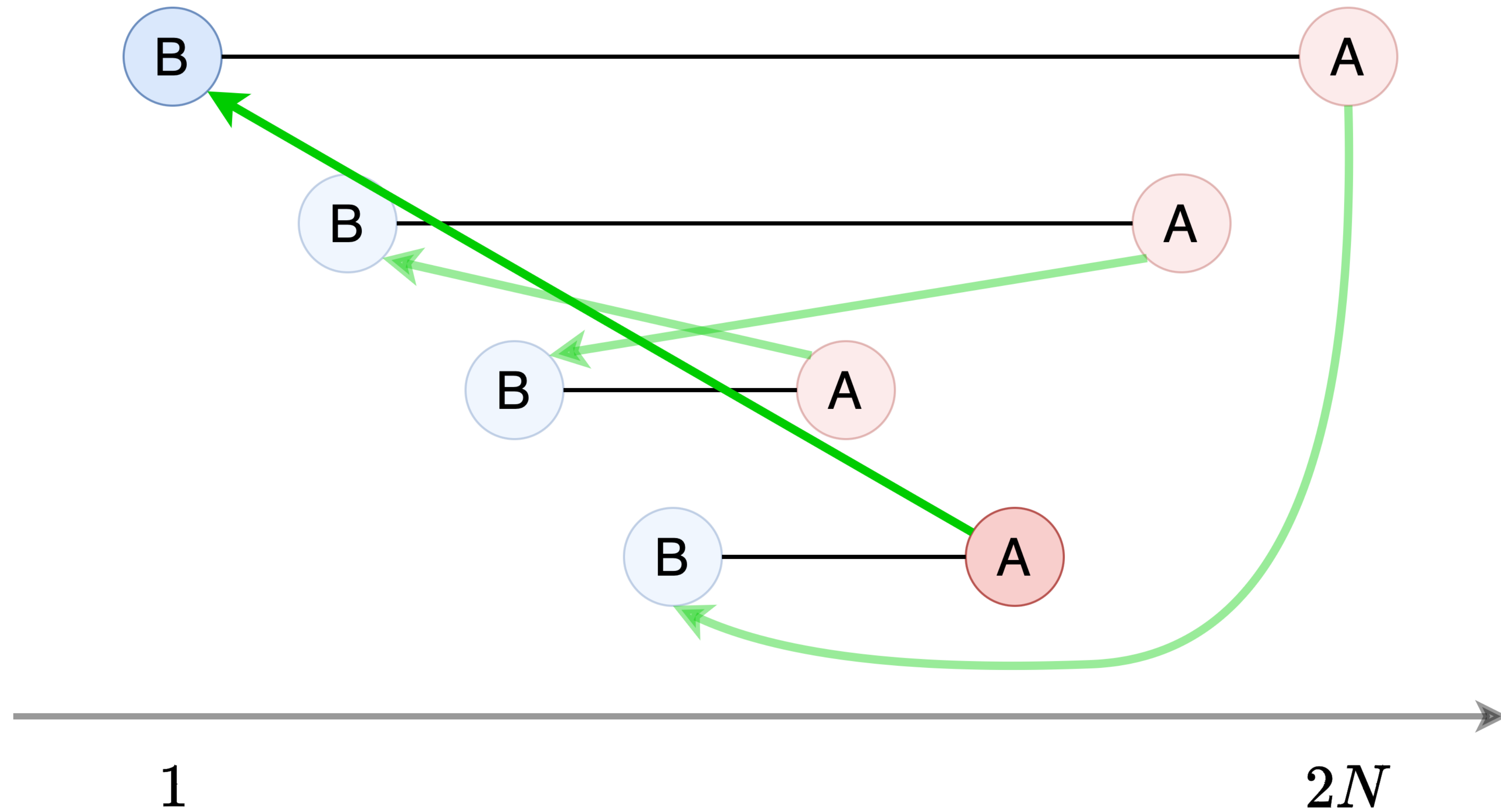
正しい貪欲法



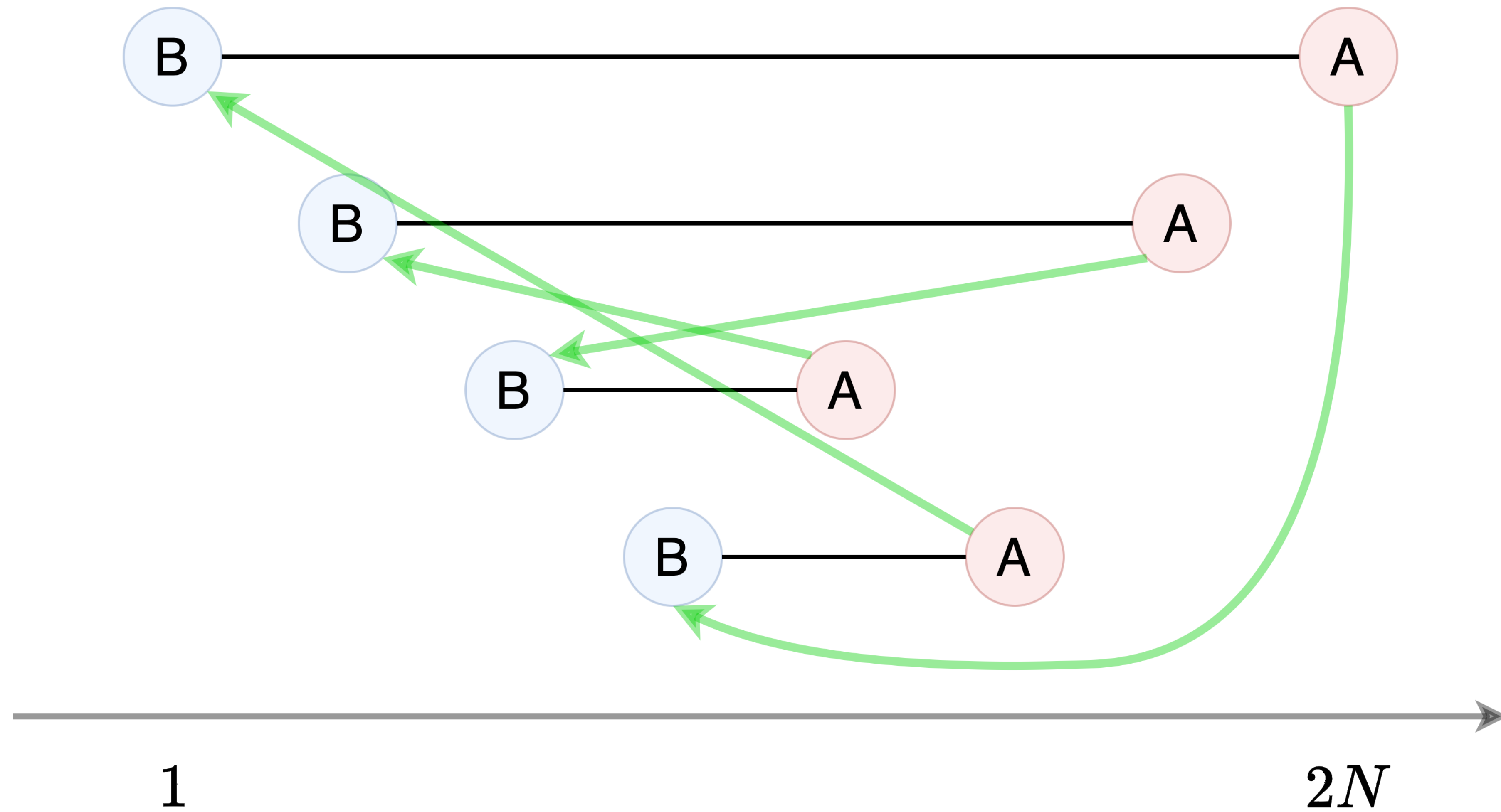
正しい貪欲法



正しい貪欲法



正しい貪欲法



正しい貪欲法

- この貪欲法が途中で失敗する i.e. ある j について生徒 j のプレゼントを受け取れる人が誰もいない場合, 区間 j は他のどの区間とも共通部分を持たないことが示せる.
- 対偶: どの区間も他のどれかしらの区間と共通部分を持つとき, この貪欲法は必ず成功する. つまり, 実際にプレゼント交換をする方法が構築できる. 証明完了.

小課題 4 (31 点, 累計 50 点)

- $N \leq 10^5$, $Q \leq 10$
- グループ内の各区間について, その区間との共通部分を持つような他の区間がグループ内に存在するか判定すれば良い.
ソート等を用いて, クエリあたり $O(N \log N)$
- 条件を見つけていなくても, 正しい貪欲がわかっているならば, `std::set` 等を用いて実際にシミュレーションし, 貪欲が途中で失敗するかどうか調べればよい.
- 計算量 $O(QN \log N)$
- 嘘貪欲に気をつけましょう

小課題 5 (8 点, 累計 58 点)

- $N \leq 10^5$, $A_1 < A_2 < \dots < A_N$, $B_1 < B_2 < \dots < B_N$
- 小課題 4 を貪欲法で解いた人が条件を見つけるのを助ける用の小課題
 - 限定された条件下での貪欲法の動作をよく観察し, 貪欲法が失敗するのはどんなときか考察する.

小課題 5 (8 点, 累計 58 点)

- $N \leq 10^5$, $A_1 < A_2 < \dots < A_N$, $B_1 < B_2 < \dots < B_N$
- 条件をすでに見つけている人にとっては簡単
 - ある区間が他の区間と共通部分を持つかは, その区間と両隣の区間との位置関係のみによって定まる.
 - $A_{i-1} < B_i < A_i < B_{i+1}$ を満たすような区間 (つまり, 両隣と共通部分を持たないような区間) の番号の集合 S を前計算する.
 - クエリでは, $[L_j, R_j]$ 内に S の要素があるか見ればいい. **両端の扱いに注意.**

小課題 6 ～満点

- 各区間 i に対して以下の値を定義する：
 - S_i : 区間 i と共通部分を持つ区間の番号のうち, i 未満で最大のもの (ないならば 0)
 - T_i : 区間 i と共通部分を持つ区間の番号のうち, i より大きくて最小のもの (ないならば $N + 1$)
- あるグループが生徒 i を含み, かつそのグループを成す生徒の番号の範囲が $[S_i + 1, T_i - 1]$ に含まれるとき, 区間 i はそのグループ内で他と共通部分を持たない
- $\Rightarrow L_j \in [S_i + 1, i]$ かつ $R_j \in [i, T_i - 1]$ なら, グループ j はプレゼント交換不可能

小課題 6 ～満点

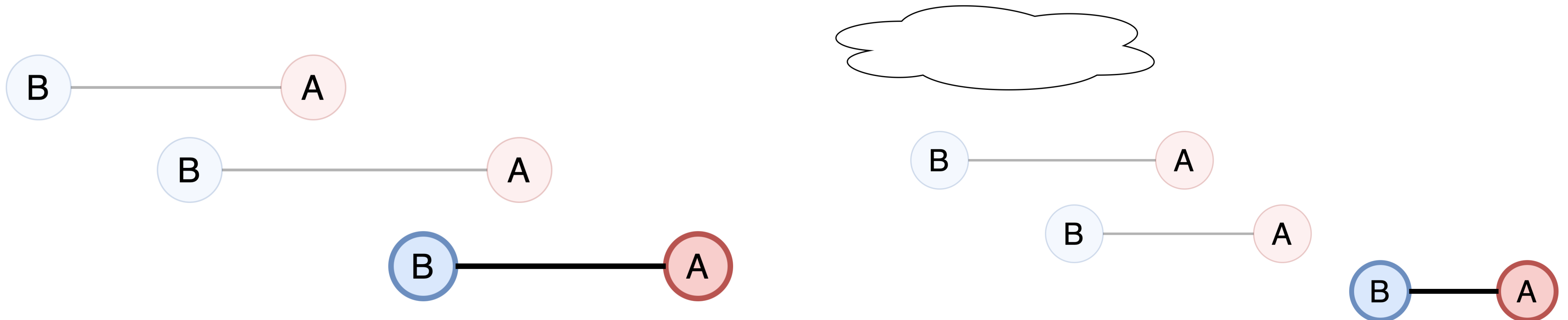
- 従って, S_i, T_i が各区間 i について求まっているとき, 以下の問題に帰着される:
 - $N \times N$ の二次元平面上に N 個の長方形が存在している.
 - 平面上の点が Q 個与えられる.
それぞれの点について, その点を内部に含む長方形が存在するか判定せよ.
- Fenwick tree (BIT) や Segment tree を用いた平面走査によって $O(Q \log N)$

小課題 6 ～満点

- 各区間 i に対して以下の値を定義する（再掲）：
 - S_i : 区間 i と共通部分を持つ区間の番号のうち, i 未満で最大のもの（ないならば 0）
 - T_i : 区間 i と共通部分を持つ区間の番号のうち, i より大きくて最小のもの（ないならば $N + 1$ ）
- これを高速に求めるのが最後の課題（満点まで共通）.

小課題 6 (12 点, 累計 70 点)

- 各区間 i に対して以下の値を定義する（再掲）：
 - S_i : 区間 i と共通部分を持つ区間の番号のうち, i 未満で最大のもの（ないならば 0）
 - T_i : 区間 i と共通部分を持つ区間の番号のうち, i より大きくて最小のもの（ないならば $N + 1$ ）
- $N \leq 10^5, A_1 < A_2 < \dots < A_N$
- S_i については簡単 : $A_{i-1} > B_i$ ならば $i - 1$, $A_{i-1} < B_i$ ならば 0



小課題 6 (12 点, 累計 70 点)

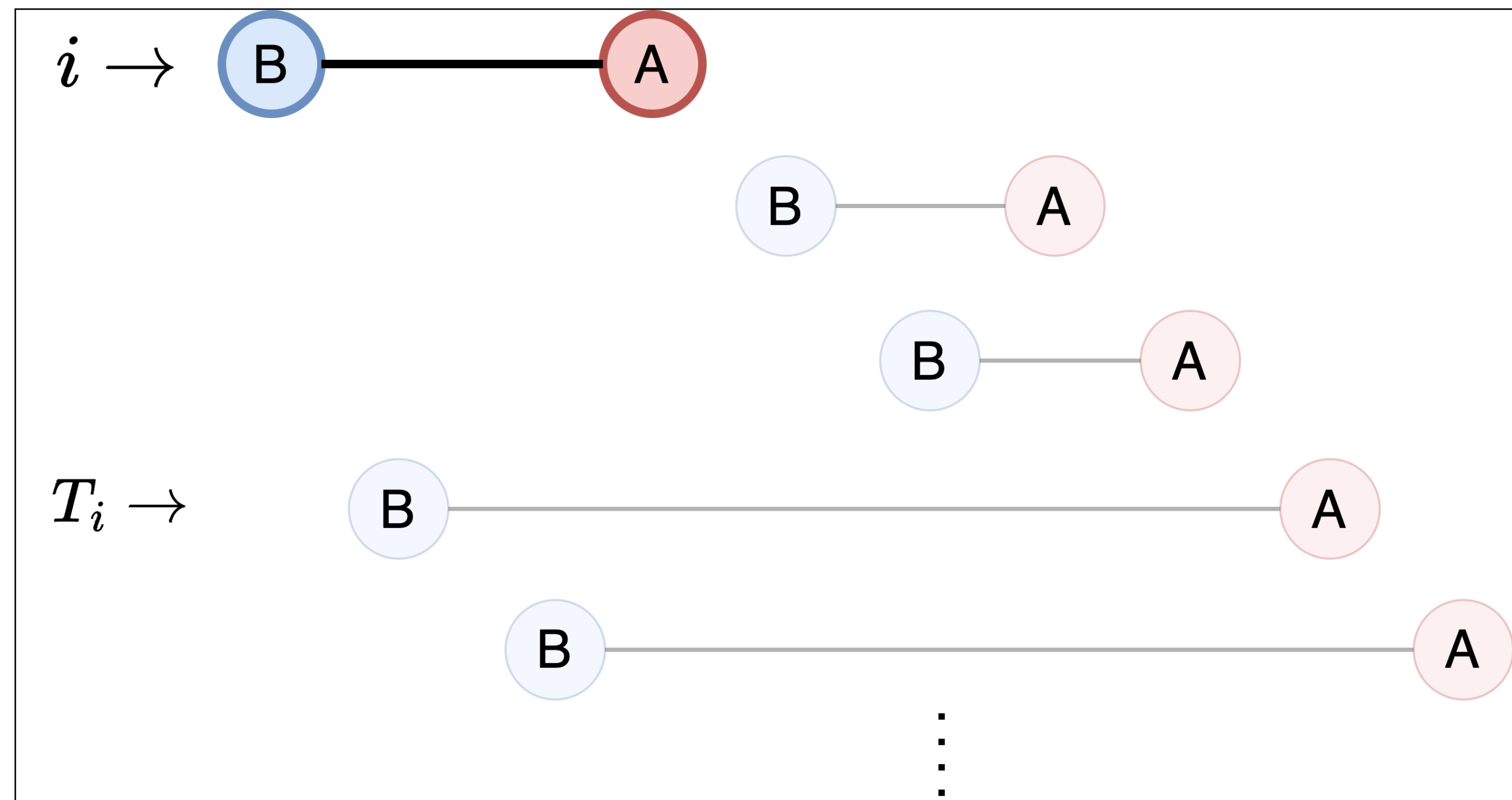
- 各区間 i に対して以下の値を定義する（再掲）：

- S_i ：区間 i と共通部分を持つ区間の番号のうち， i 未満で最大のもの（ないならば 0）
- T_i ：区間 i と共通部分を持つ区間の番号のうち， i より大きくて最小のもの（ないならば $N + 1$ ）

- $N \leq 10^5$, $A_1 < A_2 < \dots < A_N$

- T_i については？

- $A_i > B_j$ を満たす最小の j ($> i$)



小課題 6 (12 点, 累計 70 点)

- 各区間 i に対して以下の値を定義する（再掲）：
 - S_i : 区間 i と共通部分を持つ区間の番号のうち, i 未満で最大のもの（ないならば 0）
 - T_i : 区間 i と共通部分を持つ区間の番号のうち, i より大きくて最小のもの（ないならば $N + 1$ ）
- $N \leq 10^5, A_1 < A_2 < \dots < A_N$
- T_i については？
 - $A_i > B_j$ を満たす最小の j ($> i$)
 - segment tree や stack などのデータ構造上で二分探索
- 計算量 $O(N \log N + Q(\log N + \log Q))$

小課題 7 (18 点, 累計 88 点)

- 各区間 i に対して以下の値を定義する（再掲）：
 - S_i : 区間 i と共通部分を持つ区間の番号のうち, i 未満で最大のもの（ないならば 0）
 - T_i : 区間 i と共通部分を持つ区間の番号のうち, i より大きくて最小のもの（ないならば $N + 1$ ）
- $N \leq 10^5$
- 2D segment tree (point update rectangle max) などを用いる（詳細略）
 - ほぼ ”貼るだけ” だが, 余分 $\log$ が付くので満点は取れない
- 計算量 $O(N \log^2 N + Q(\log N + \log Q))$ など

小課題 8 (12 点, 累計 100 点)

- 各区間 i に対して以下の値を定義する (再掲) :
 - S_i : 区間 i と共通部分を持つ区間の番号のうち, i 未満で最大のもの (ないならば 0)
 - T_i : 区間 i と共通部分を持つ区間の番号のうち, i より大きくて最小のもの (ないならば $N + 1$)
- $N \leq 5 \times 10^5$
- S_i についてだけ考える (T_i も同様)
- 長さ $2N$ の配列 $v = (0, 0, \dots, 0)$ を用意する. $i = 1, 2, \dots, N$ の順に以下を行う :
 - $v[B_i], v[B_i + 1], \dots, v[A_i]$ の最大値を求め, S_i とする.
 - $v[B_i], v[B_i + 1], \dots, v[A_i]$ の値をすべて i に更新する.

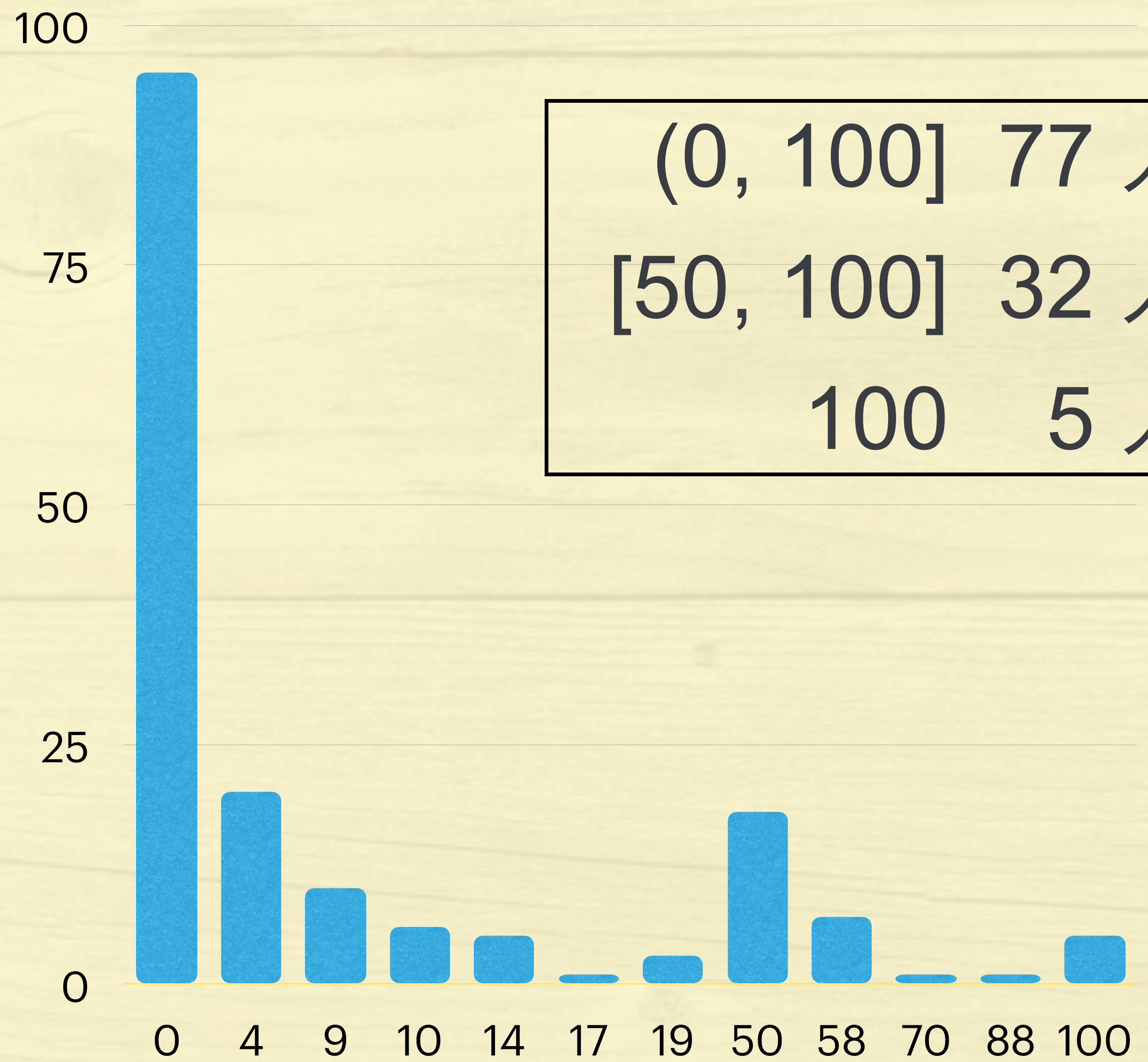
小課題 8 (12 点, 累計 100 点)

- 各区間 i に対して以下の値を定義する (再掲) :
 - S_i : 区間 i と共通部分を持つ区間の番号のうち, i 未満で最大のもの (ないならば 0)
 - T_i : 区間 i と共通部分を持つ区間の番号のうち, i より大きくて最小のもの (ないならば $N + 1$)
- 長さ $2N$ の配列 $v = (0, 0, \dots, 0)$ を用意する. $i = 1, 2, \dots, N$ の順に以下を行う :
 - $v[B_i], v[B_i + 1], \dots, v[A_i]$ の最大値を求め, S_i とする.
 - $v[B_i], v[B_i + 1], \dots, v[A_i]$ の値をすべて i に更新する.
- lazy segment tree (range update range max) が使える.
- 計算量 $O(N \log N + Q(\log N + \log Q))$





得点分布



(0, 100] 77 人

[50, 100] 32 人

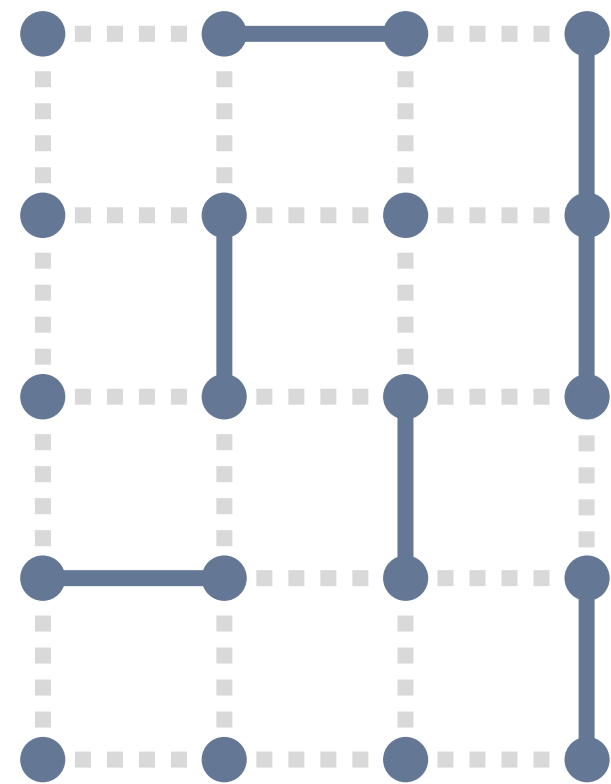
100 5 人

道路網の整備2 解説

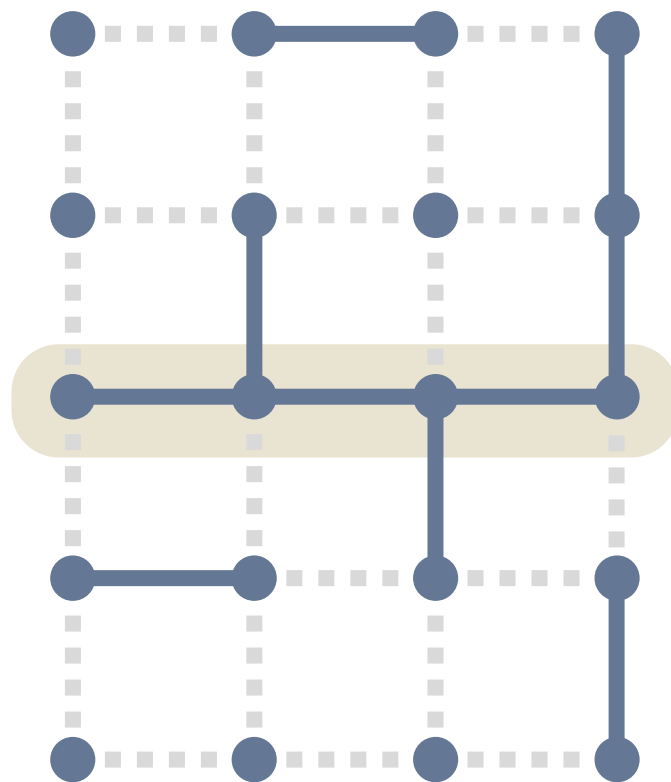
米田優峻 (E869120)

2024 年 2 月 4 日

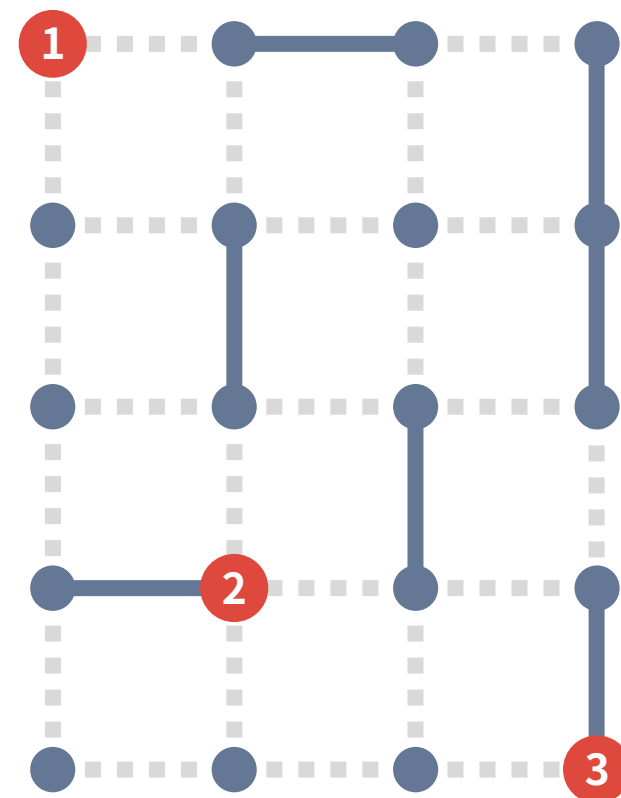
- H 行 W 列の碁盤目状道路がある
- 右図のように、道路には通れる部分と通れない部分がある



- H 行 W 列の碁盤目状道路がある
- 右図のように、道路には通れる部分と通れない部分がある
- あなたは c_i 日間の整備により、 i 行目の道路を全部「通れる状態」にすることができる

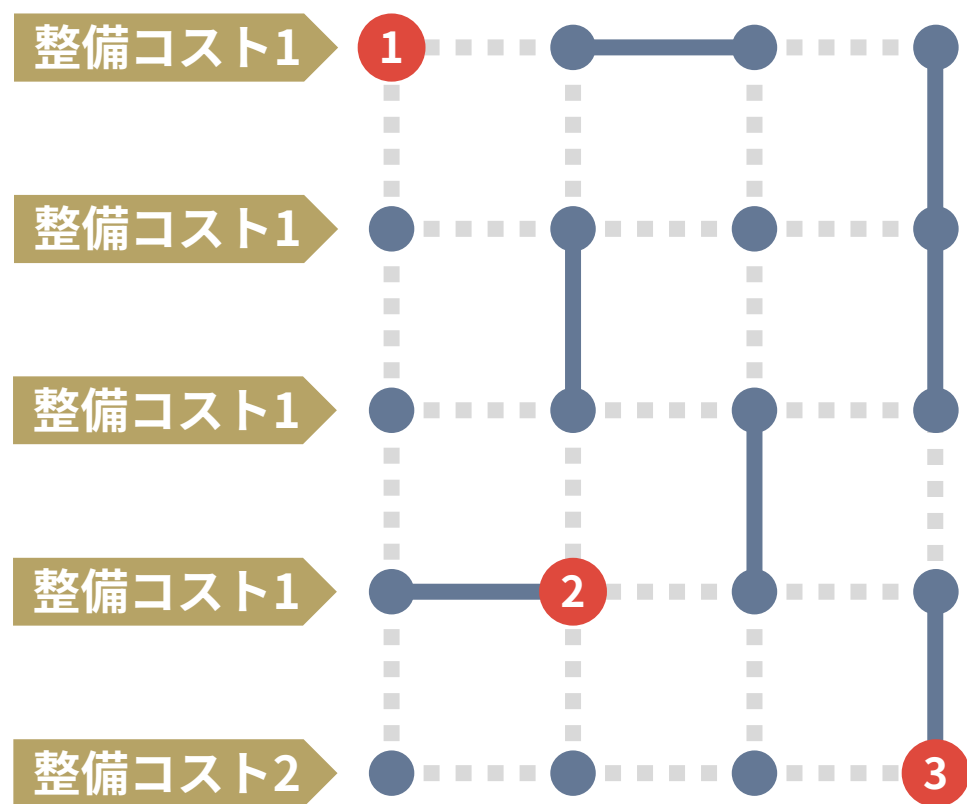


- H 行 W 列の碁盤目状道路がある
- 右図のように、道路には通れる部分と通れない部分がある
- あなたは C_i 日間の整備により、 i 行目の道路を全部「通れる状態」にすることができる
- 交差点 $(X_1, Y_1), (X_2, Y_2), \dots, (X_T, Y_T)$ を連結にするには何日かかるか？



問題概要: 具体例

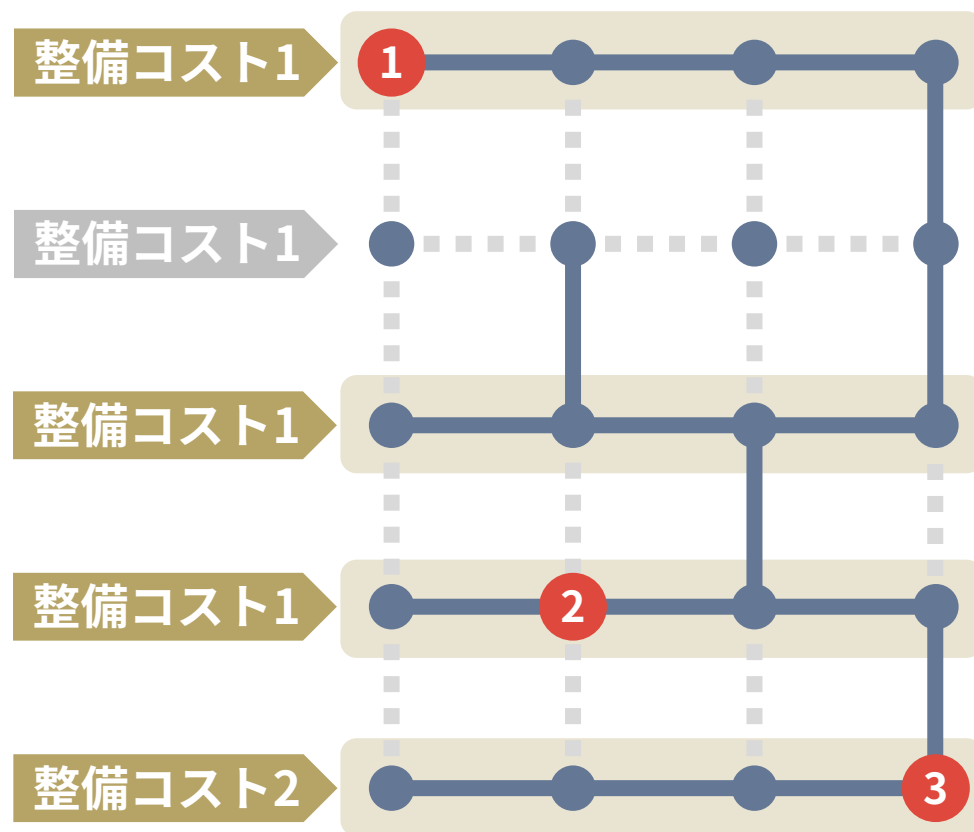
5



- たとえば、左図の道路の地点 1, 2, 3 を連結するにはコストいくつ必要か？

問題概要: 具体例

6



- たとえば、左図の道路の地点 1, 2, 3 を連結するにはコストいくつ必要か？
- 1, 3, 4, 5 行目を整備するのが最適で、コスト 5 が必要

問題概要はわかりましたか？

- 碁盤目状道路の大きさ: $H \times W \leq 1\,000\,000$
- 整備にかかるコスト: $C_i = 1$ または 2
- 質問回数: $Q \leq 100\,000$ (T の値の合計は $200\,000$ 以下)

- 小課題 5 までは $c_i = 1$ が続きます
- しばらくの間は、**どの行の整備も 1 日で出来る**と思って解説を読んでください

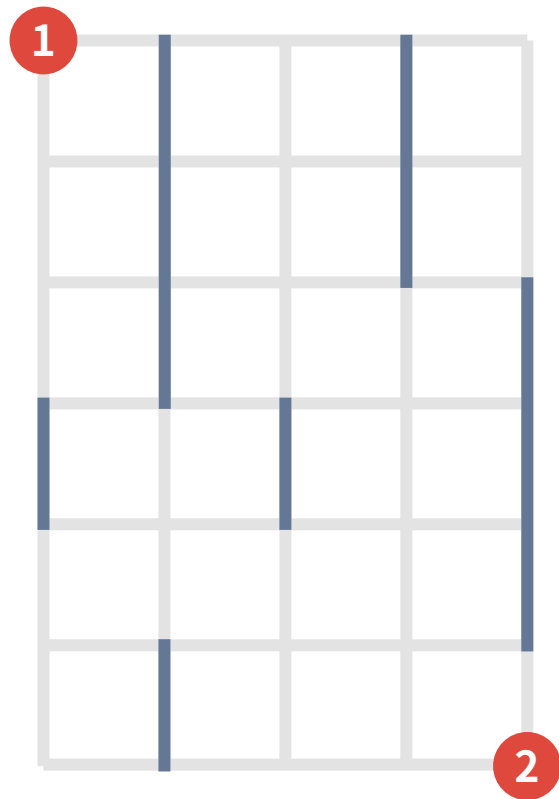
小課題 1, 2

$$C_i = 1, Q \leq 5, T = 2$$

基本方針：できるだけ下で整備する

小課題 1, 2: 具体例 A

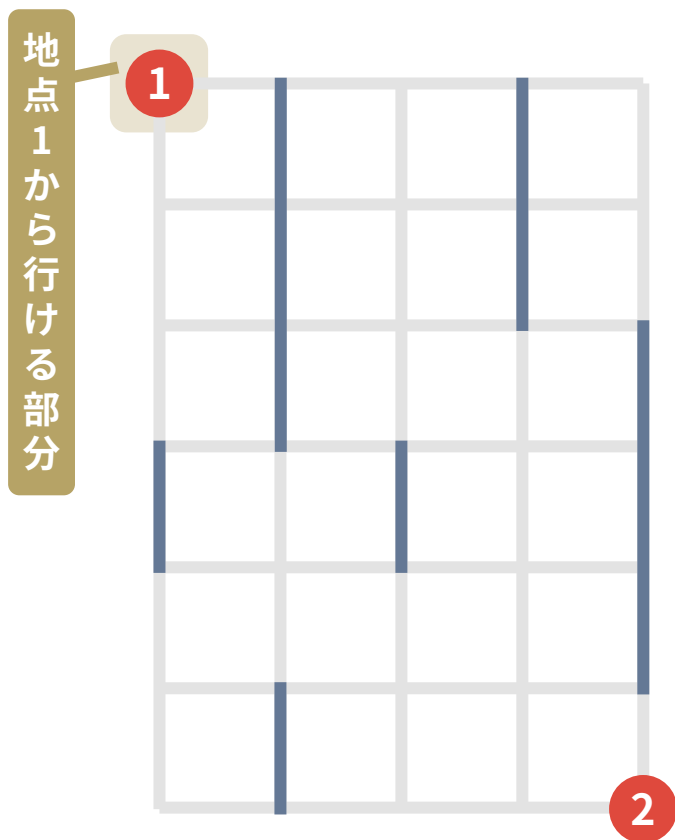
12



例: 下図の 1 と 2 を連結にすることを考える

小課題 1, 2: 具体例 A

13

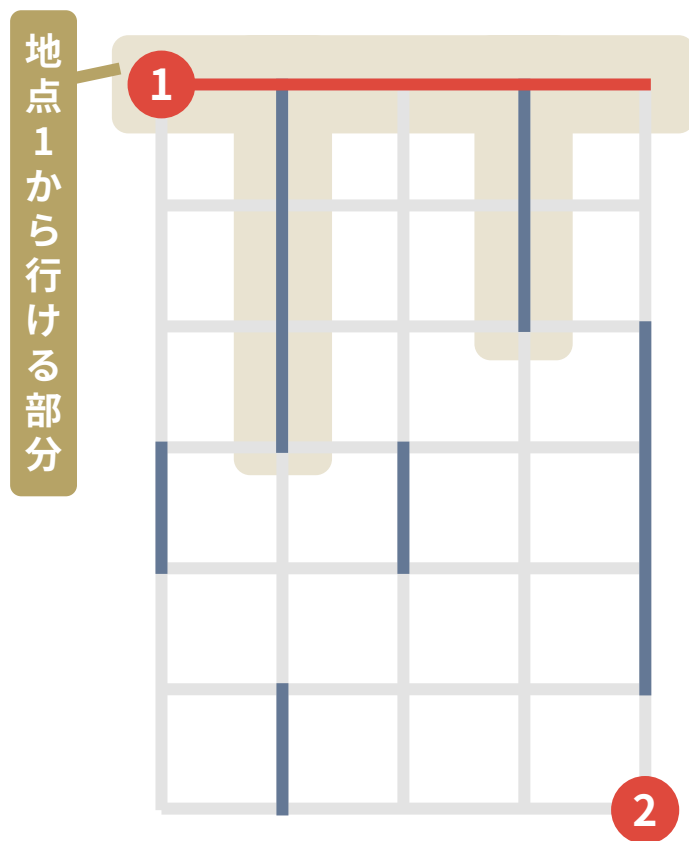


例: 下図の 1 と 2 を連結にすることを考える

- 現時点で地点 1 からは 1 行目まで到達可能 → 1 行目を整備

小課題 1, 2: 具体例 A

14

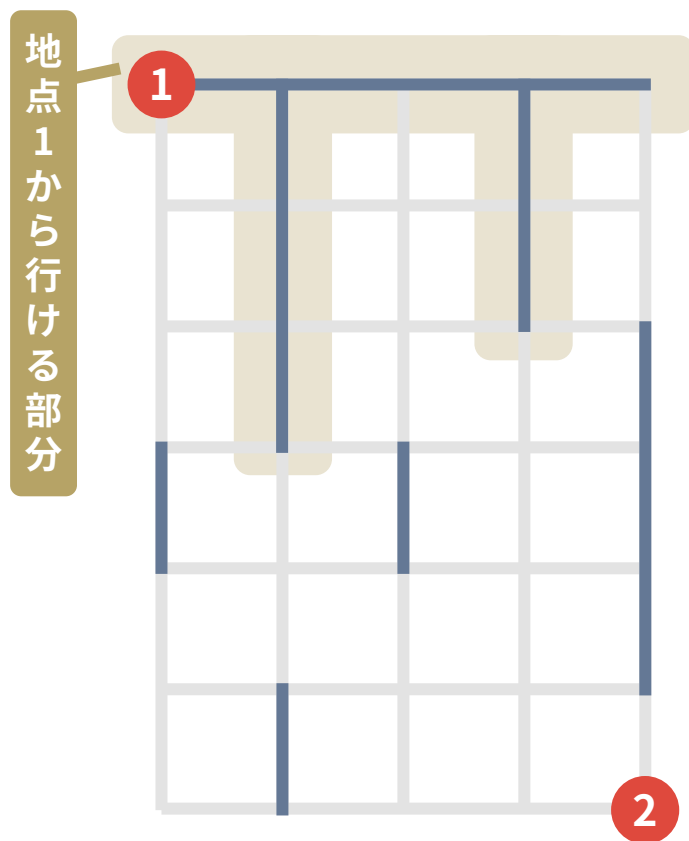


例: 下図の 1 と 2 を連結にすることを考える

- 現時点で地点 1 からは 1 行目まで到達可能 → 1 行目を整備

小課題 1, 2: 具体例 A

15

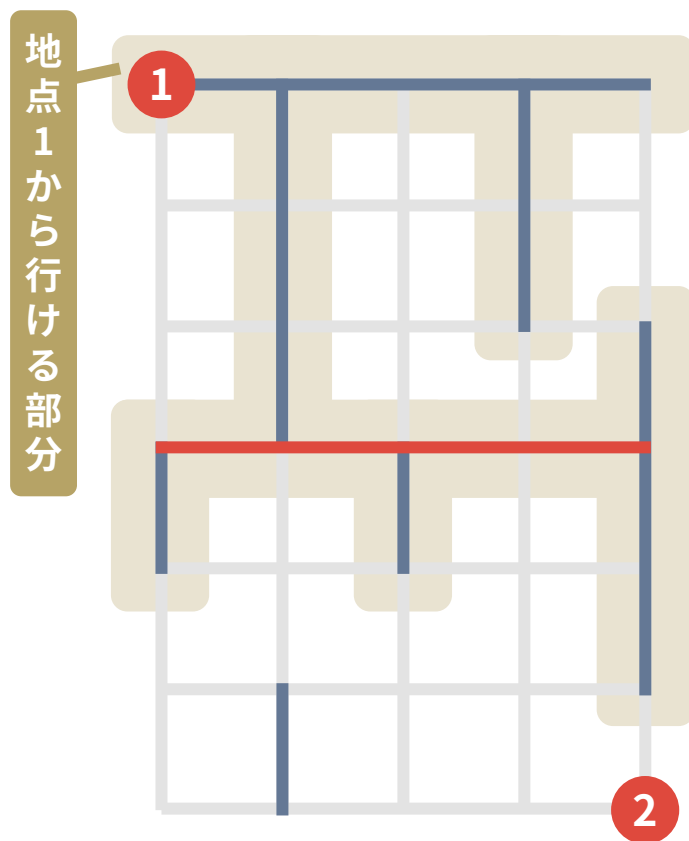


例: 下図の 1 と 2 を連結にすることを考える

- 現時点で地点 1 からは 1 行目まで到達可能 → 1 行目を整備
- 現時点で地点 1 からは 4 行目まで到達可能 → 4 行目を整備

小課題 1, 2: 具体例 A

16

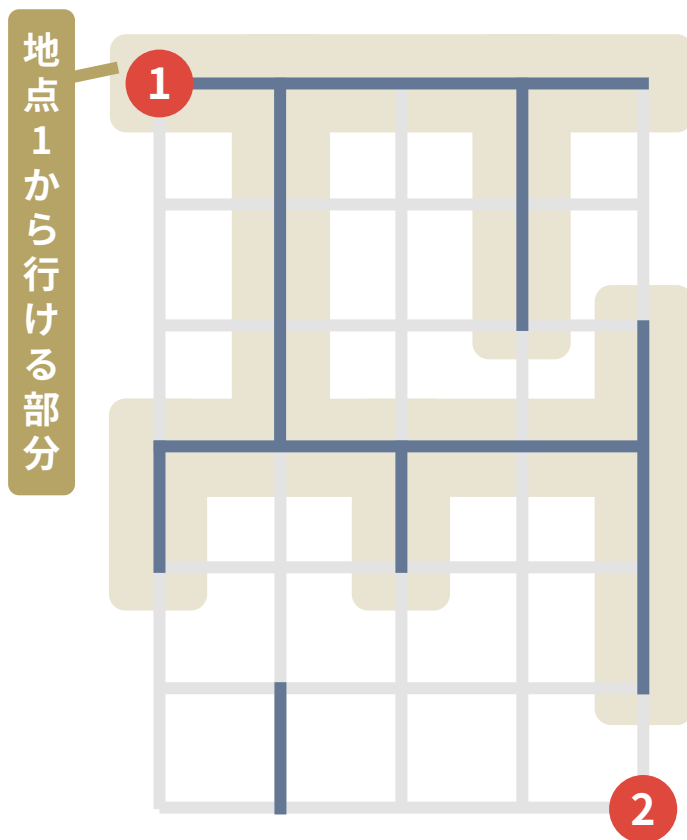


例: 下図の 1 と 2 を連結にすることを考える

- 現時点で地点 1 からは 1 行目まで到達可能 → 1 行目を整備
- 現時点で地点 1 からは 4 行目まで到達可能 → 4 行目を整備

小課題 1, 2: 具体例 A

17

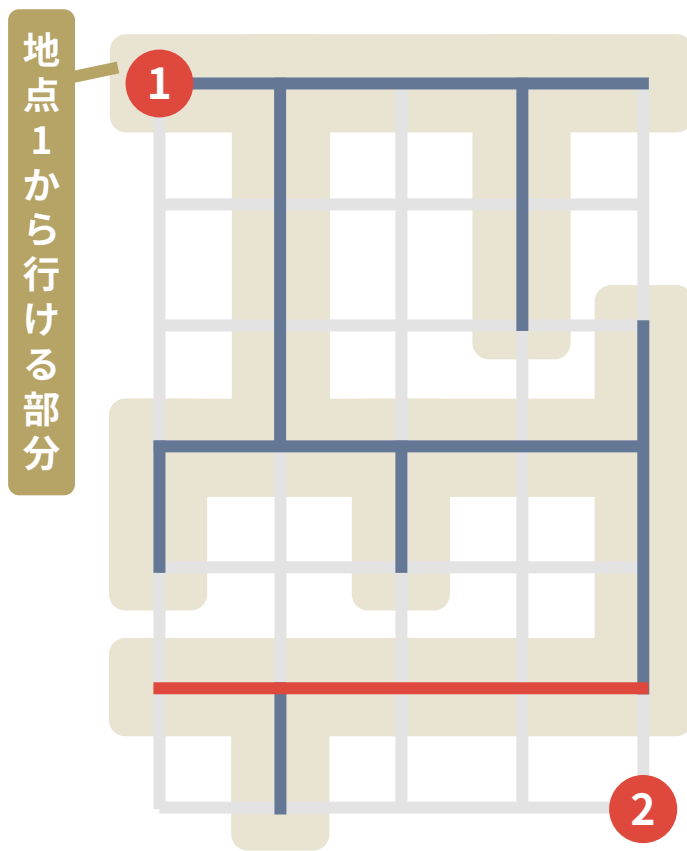


例: 下図の 1 と 2 を連結にすることを考える

- 現時点で地点 1 からは 1 行目まで到達可能 → 1 行目を整備
- 現時点で地点 1 からは 4 行目まで到達可能 → 4 行目を整備
- 現時点で地点 1 からは 6 行目まで到達可能 → 6 行目を整備

小課題 1, 2: 具体例 A

18

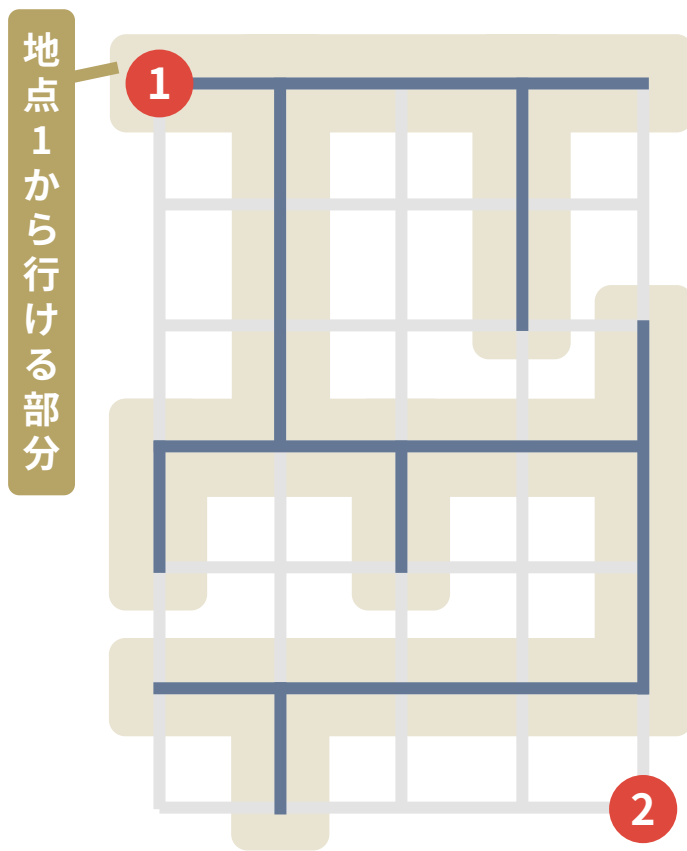


例: 下図の 1 と 2 を連結にすることを考える

- 現時点で地点 1 からは 1 行目まで到達可能 → 1 行目を整備
- 現時点で地点 1 からは 4 行目まで到達可能 → 4 行目を整備
- 現時点で地点 1 からは 6 行目まで到達可能 → 6 行目を整備

小課題 1, 2: 具体例 A

19

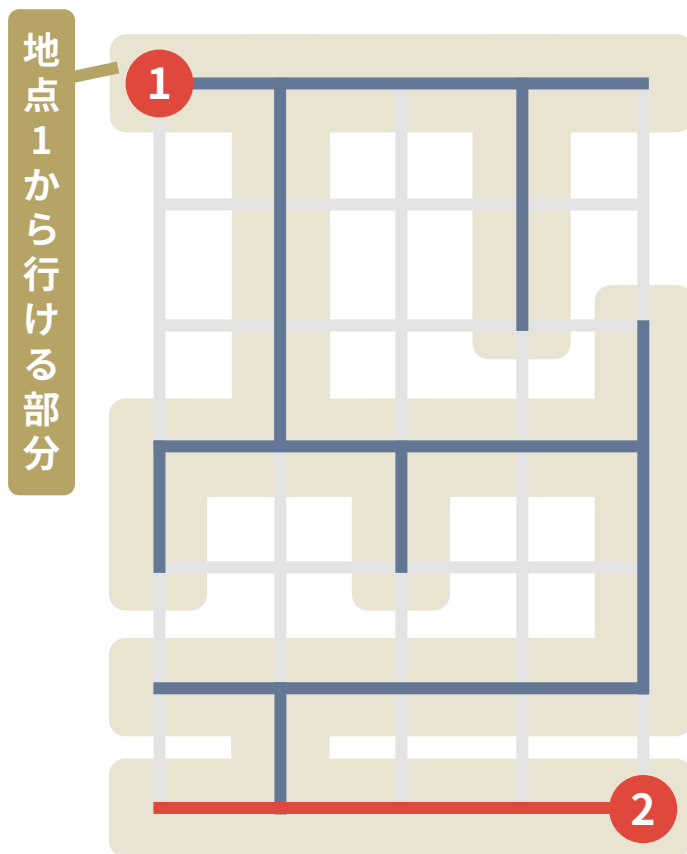


例: 下図の 1 と 2 を連結にすることを考える

- 現時点で地点 1 からは 1 行目まで到達可能 → 1 行目を整備
- 現時点で地点 1 からは 4 行目まで到達可能 → 4 行目を整備
- 現時点で地点 1 からは 6 行目まで到達可能 → 6 行目を整備
- 現時点で地点 1 からは 7 行目まで到達可能 → 7 行目を整備

小課題 1, 2: 具体例 A

20

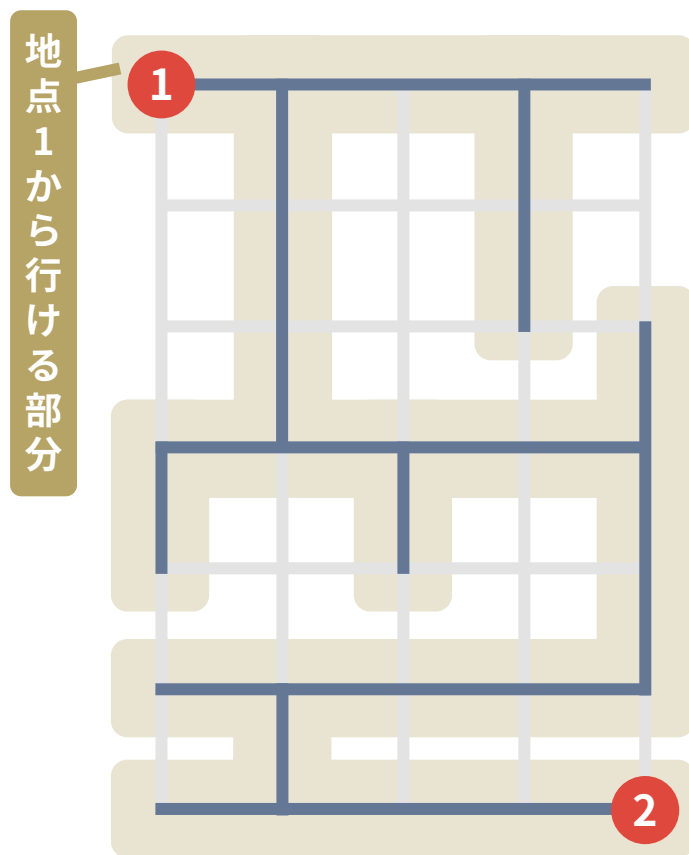


例: 下図の 1 と 2 を連結にすることを考える

- 現時点で地点 1 からは 1 行目まで到達可能 → 1 行目を整備
- 現時点で地点 1 からは 4 行目まで到達可能 → 4 行目を整備
- 現時点で地点 1 からは 6 行目まで到達可能 → 6 行目を整備
- 現時点で地点 1 からは 7 行目まで到達可能 → 7 行目を整備

小課題 1, 2: 具体例 A

21



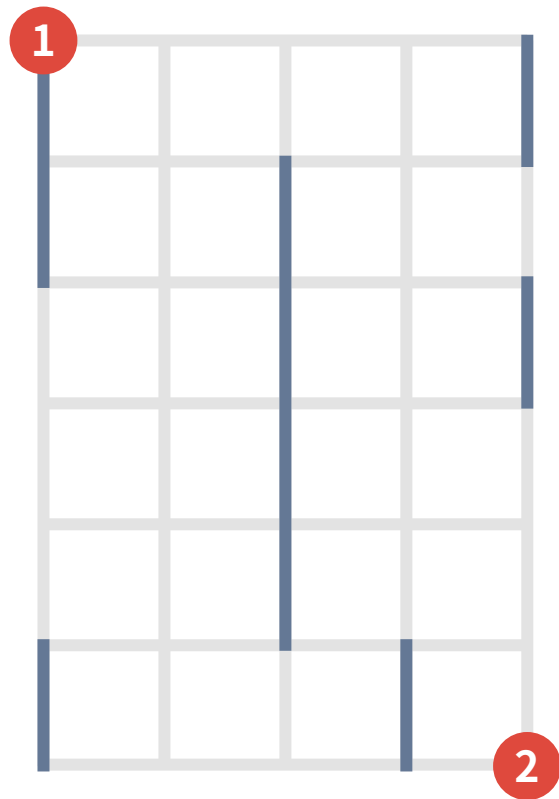
例: 下図の 1 と 2 を連結にすることを考える

- 現時点で地点 1 からは 1 行目まで到達可能 → 1 行目を整備
- 現時点で地点 1 からは 4 行目まで到達可能 → 4 行目を整備
- 現時点で地点 1 からは 6 行目まで到達可能 → 6 行目を整備
- 現時点で地点 1 からは 7 行目まで到達可能 → 7 行目を整備

➡ 4 回の整備で連結にできた！
(これが最適解)

小課題 1, 2: 具体例 B

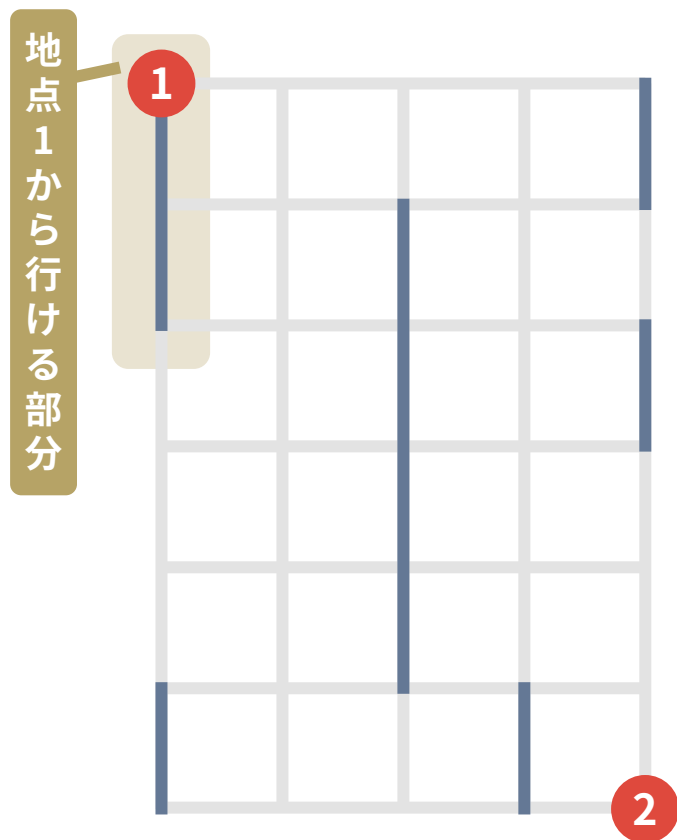
22



例: 下図の 1 と 2 を連結にすることを考える

小課題 1, 2: 具体例 B

23

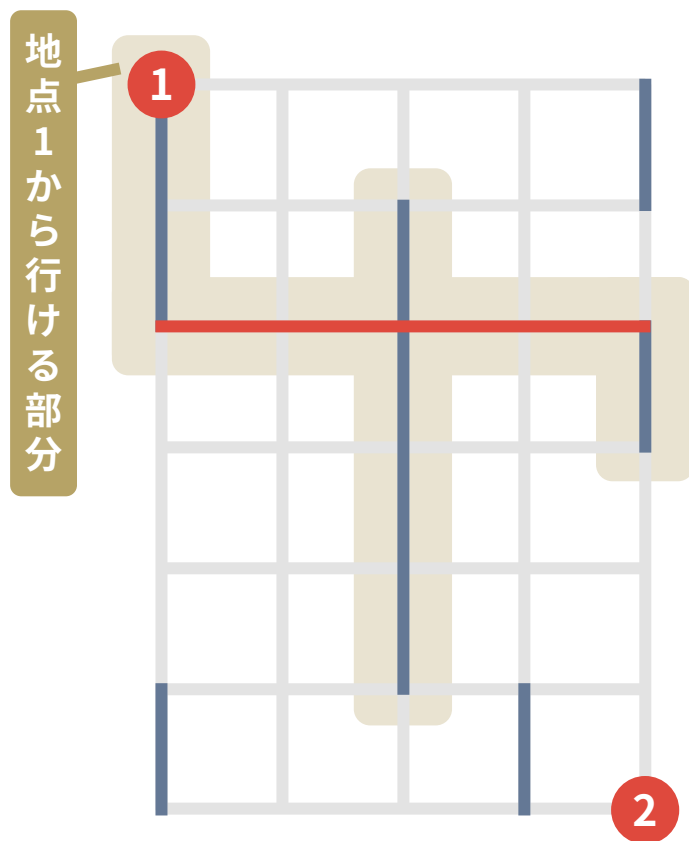


例: 下図の 1 と 2 を連結にすることを考える

- 現時点で地点 1 からは 3 行目まで到達可能 → 3 行目を整備

小課題 1, 2: 具体例 B

24

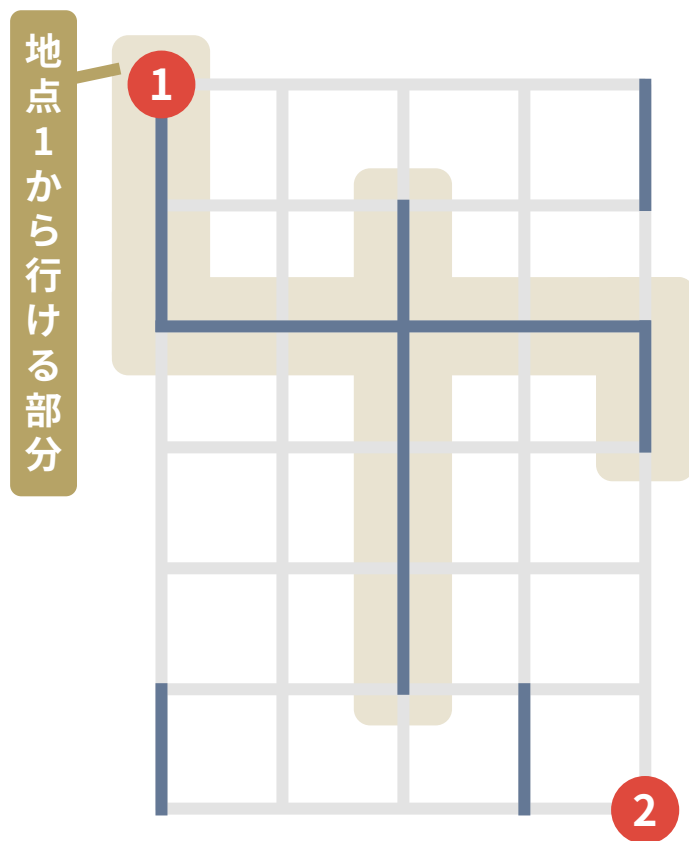


例: 下図の 1 と 2 を連結にすることを考える

- 現時点で地点 1 からは 3 行目まで到達可能 → 3 行目を整備

小課題 1, 2: 具体例 B

25

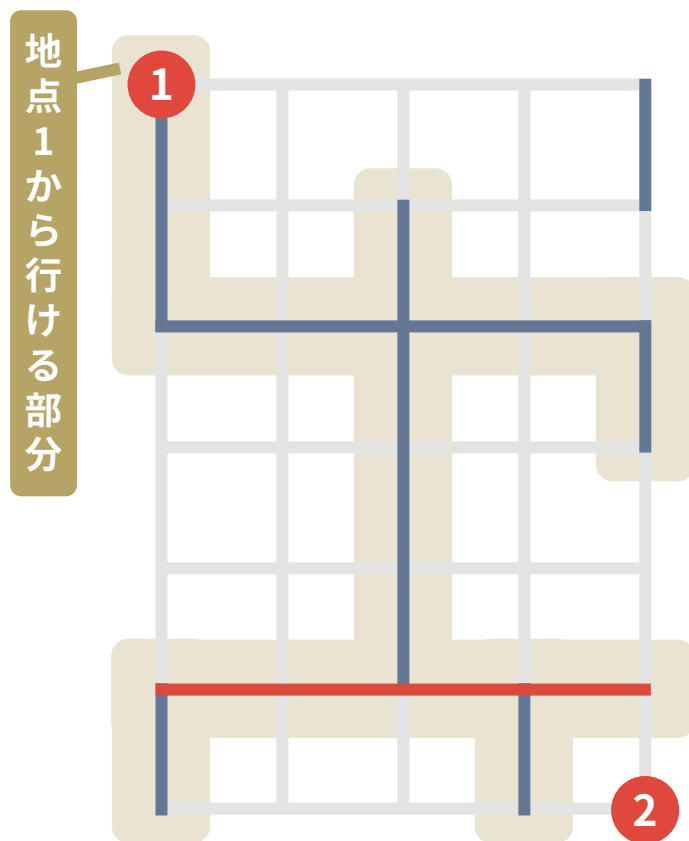


例: 下図の 1 と 2 を連結にすることを考える

- 現時点で地点 1 からは 3 行目まで到達可能 → 3 行目を整備
- 現時点で地点 1 からは 6 行目まで到達可能 → 6 行目を整備

小課題 1, 2: 具体例 B

26

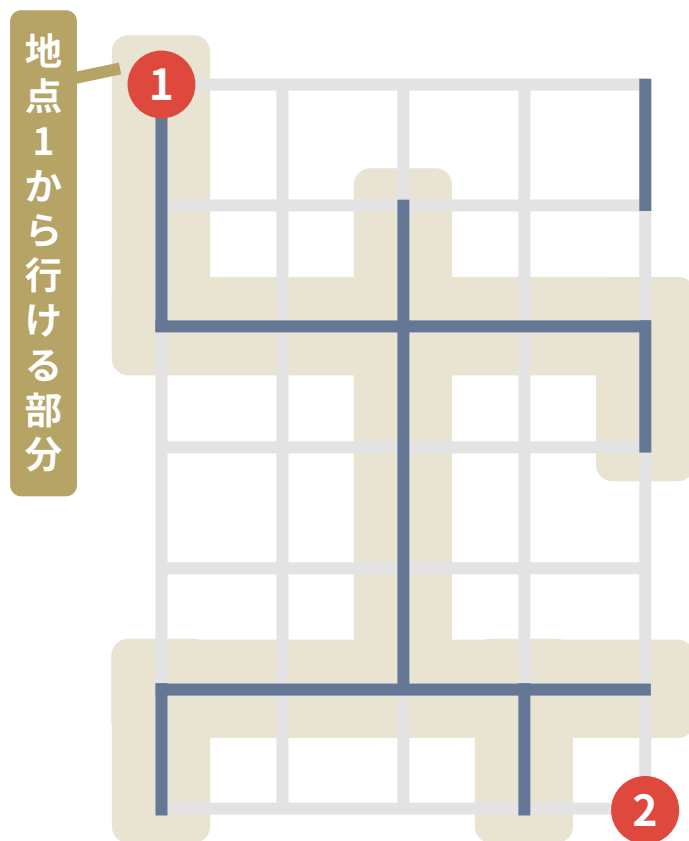


例: 下図の 1 と 2 を連結にすることを考える

- 現時点で地点 1 からは 3 行目まで到達可能 → 3 行目を整備
- 現時点で地点 1 からは 6 行目まで到達可能 → 6 行目を整備

小課題 1, 2: 具体例 B

27

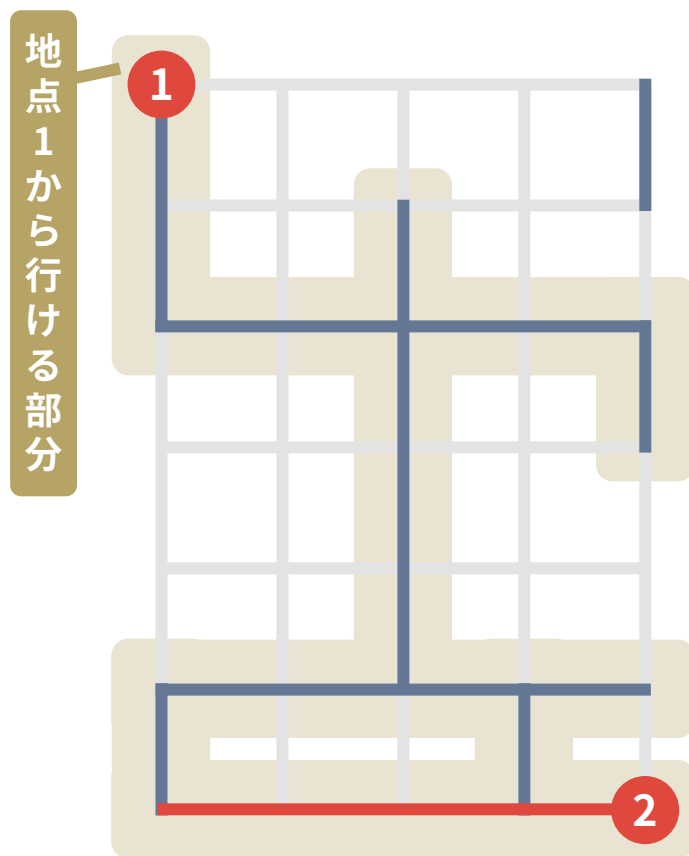


例: 下図の 1 と 2 を連結にすることを考える

- 現時点で地点 1 からは 3 行目まで到達可能 → 3 行目を整備
- 現時点で地点 1 からは 6 行目まで到達可能 → 6 行目を整備
- 現時点で地点 1 からは 7 行目まで到達可能 → 7 行目を整備

小課題 1, 2: 具体例 B

28

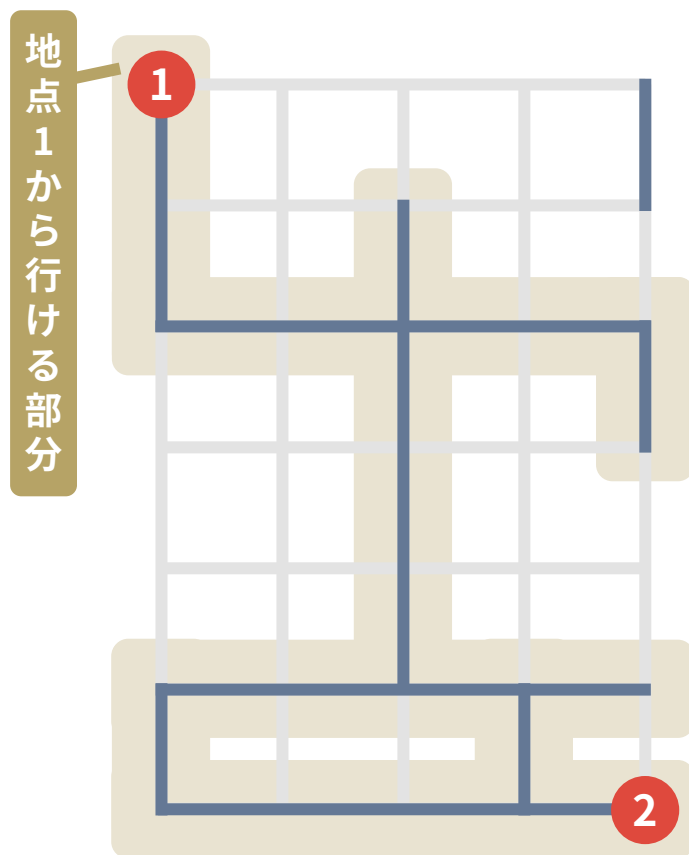


例: 下図の 1 と 2 を連結にすることを考える

- 現時点で地点 1 からは 3 行目まで到達可能 → 3 行目を整備
- 現時点で地点 1 からは 6 行目まで到達可能 → 6 行目を整備
- 現時点で地点 1 からは 7 行目まで到達可能 → 7 行目を整備

小課題 1, 2: 具体例 B

29



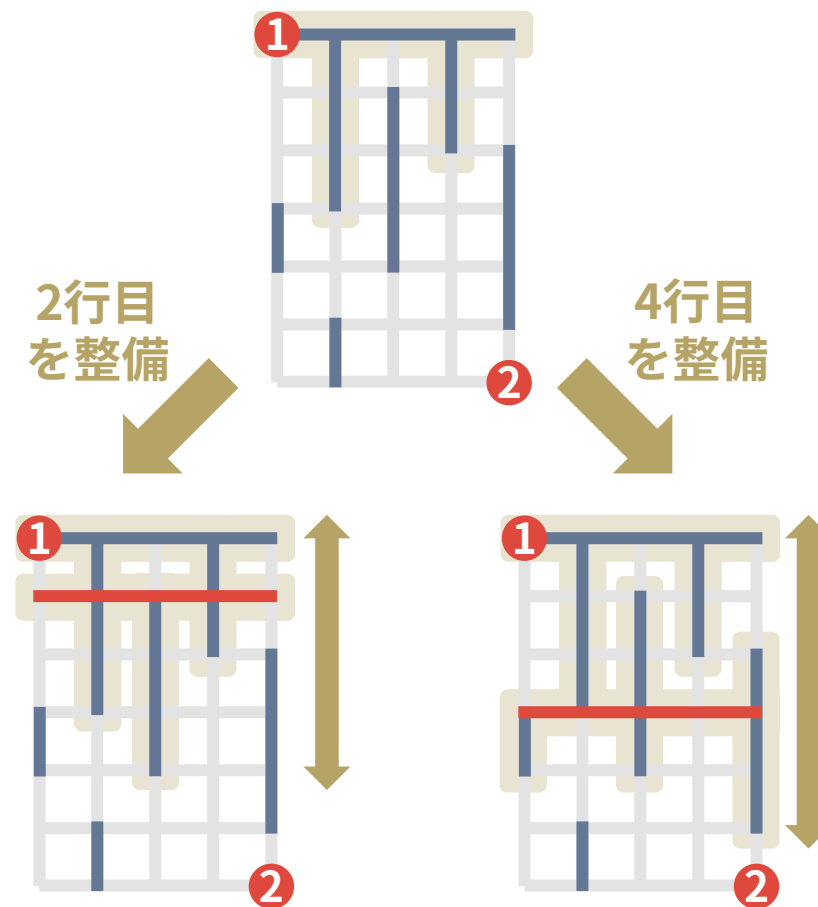
例: 下図の 1 と 2 を連結にすることを考える

- 現時点で地点 1 からは 3 行目まで到達可能 → 3 行目を整備
- 現時点で地点 1 からは 6 行目まで到達可能 → 6 行目を整備
- 現時点で地点 1 からは 7 行目まで到達可能 → 7 行目を整備

➡ 3 回の整備で連結にできた！
(これが最適解)

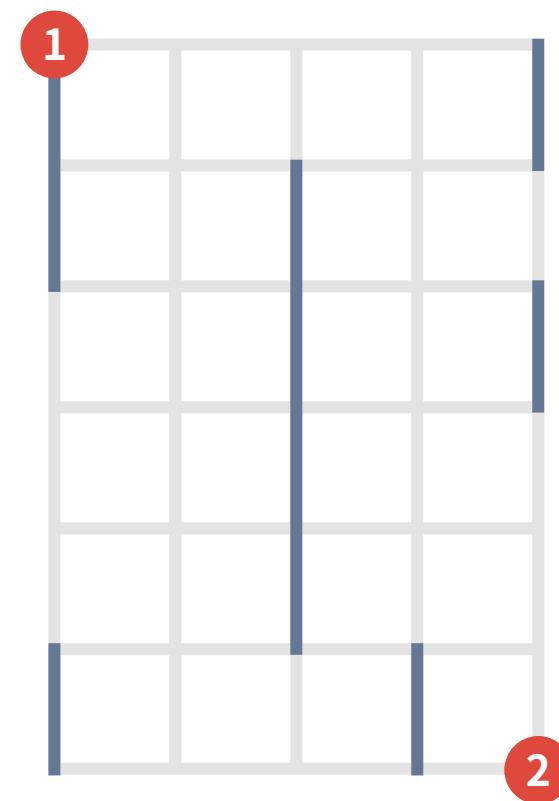
一番下を整備し続けるのが最適な理由

- 直感的には、下の行を整備したほうが、到達可能な行が広がるため
- たとえば右図で2行目を整備した場合と4行目を整備した場合では、後者の方が到達可能な行が広い

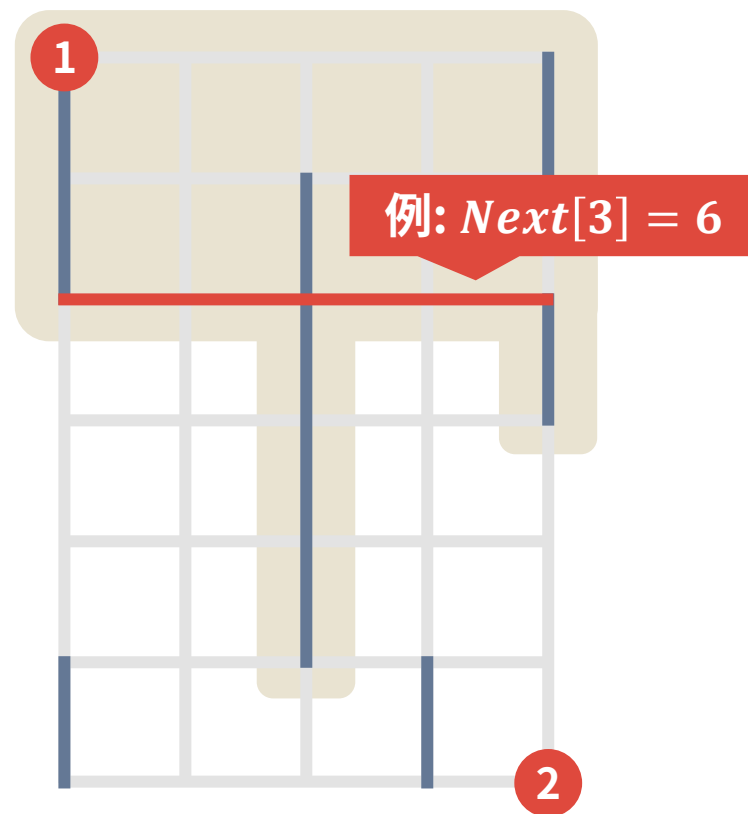


具体的にどう実装するか？

- i 行目まで到達可能なとき、 i 行目を整備した場合に到達できる一番下の行 $Next[i]$ が求まれば実装は簡単※

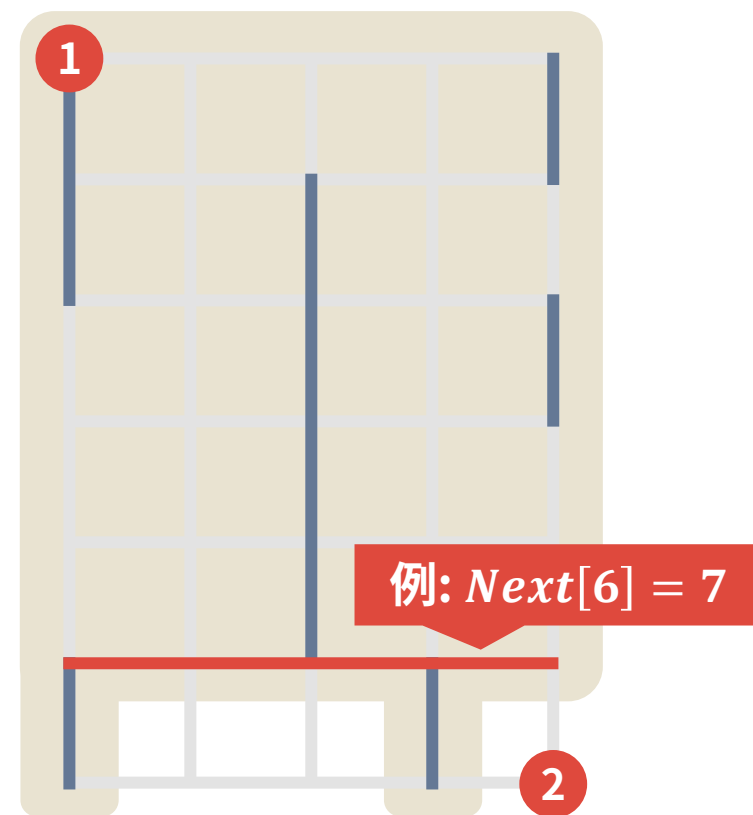


- i 行目まで到達可能なとき、 i 行目を整備した場合に到達できる一番下の行 $Next[i]$ が求まれば実装は簡単※



※具体的には、 i 行目に整備した後は $Next[i]$ 行目を整備すれば良い

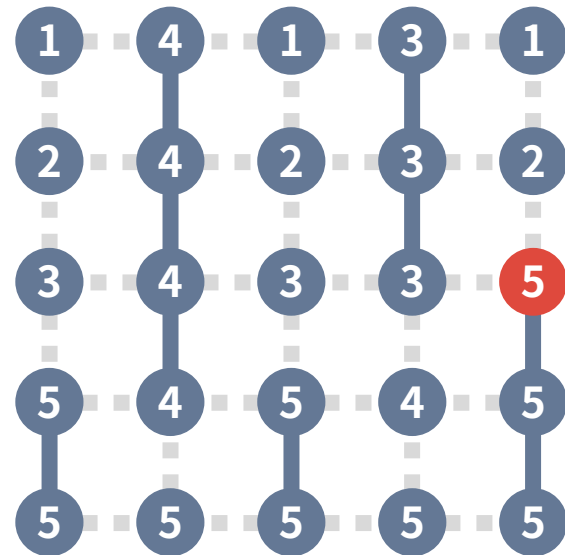
- i 行目まで到達可能なとき、 i 行目を整備した場合に到達できる一番下の行 $Next[i]$ が求まれば実装は簡単※



そこで $Next[i]$ は次のように計算できる

ステップ 1

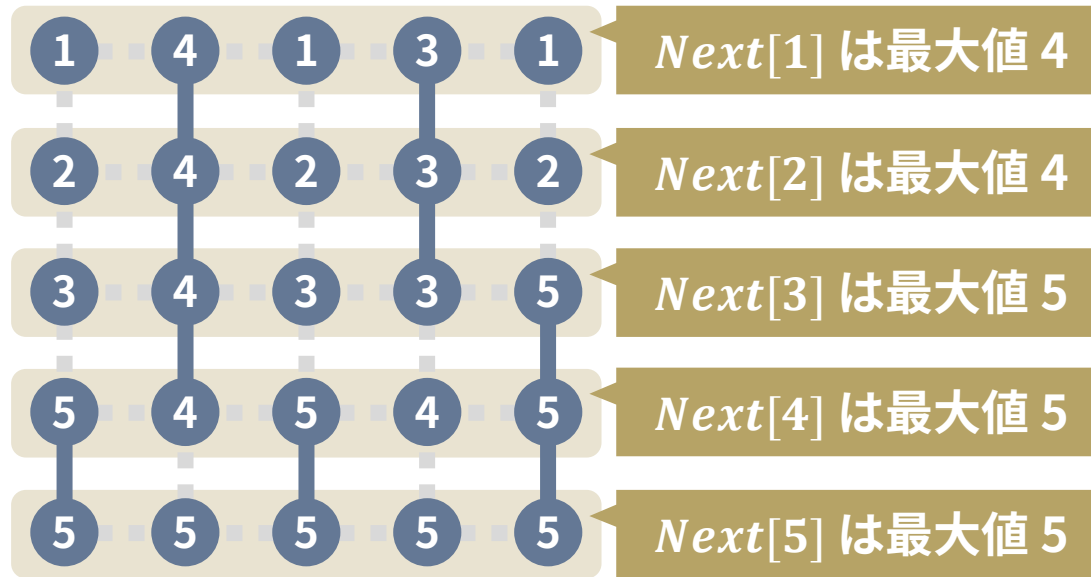
各交差点について、この場所から到達できる一番下の行を計算
幅優先探索などで $O(HW)$ で計算できる



例：このマスからは
5 行目まで到達可能

ステップ 2

そこで、 $Next[i]$ は i 行目の数字の最大値である
→ $O(HW)$ で全部計算できる



以下のような方針で小課題 1, 2 を解くことができた

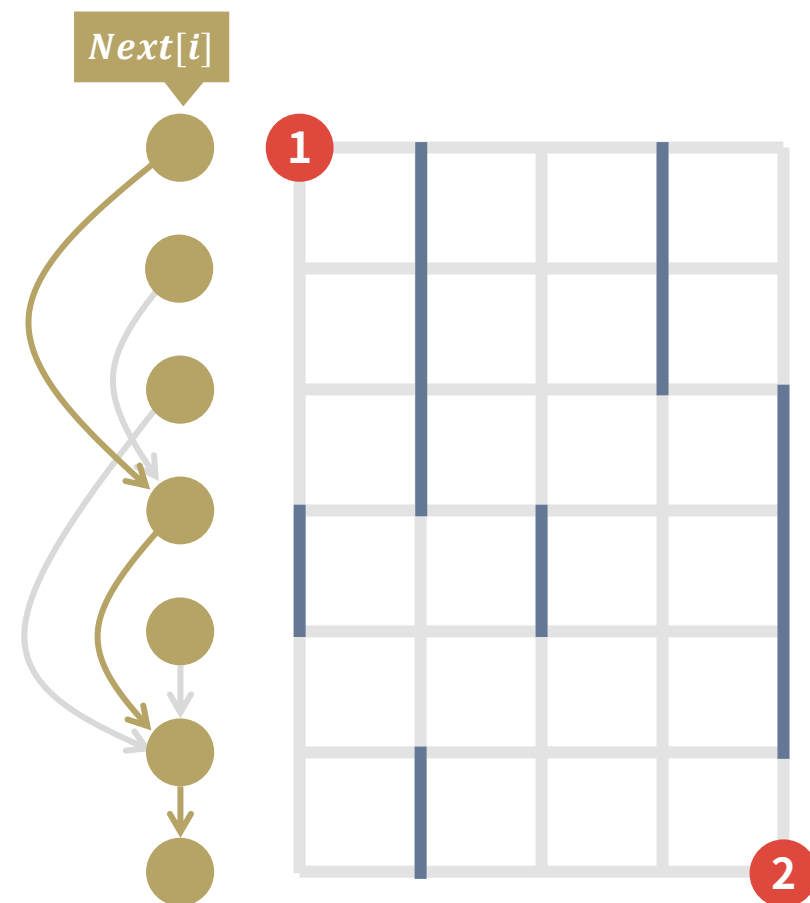
- i 行目を整備した場合に到達できる最大の行 $Next[i]$ を計算
- $Next[i]$ をもとに「一番下を整備する貪欲法」を適用

小課題 4

$$C_i = 1, T = 2$$

クエリはたくさん

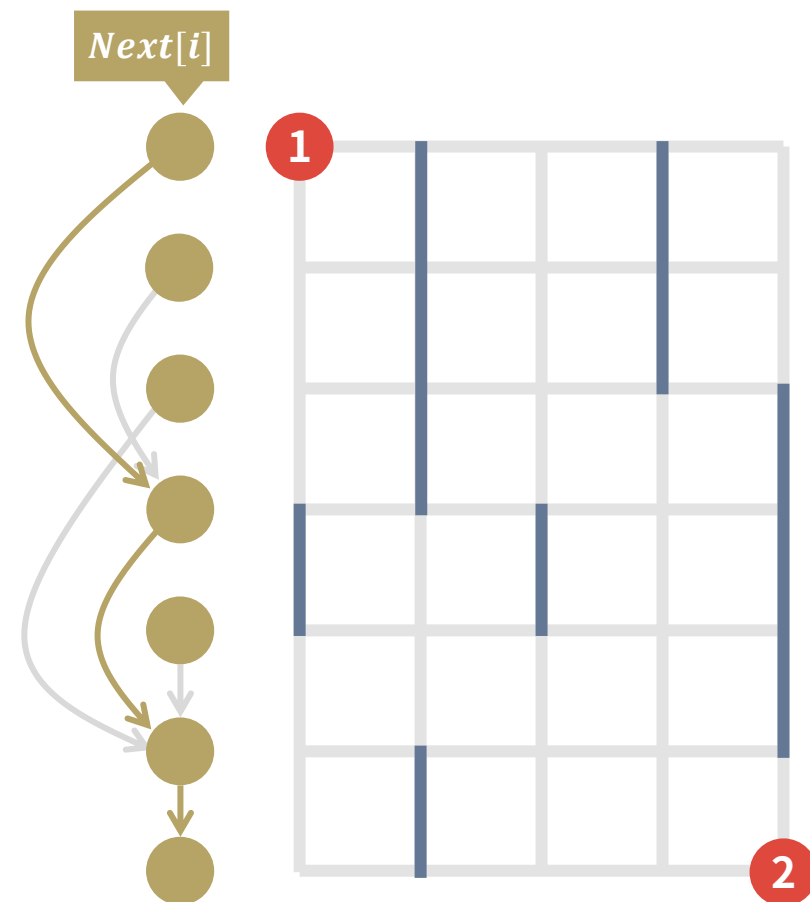
- 本問題の答えは、 i 行目から $Next[i]$ 行目まで 1 手で行けるとき、 a 行目から b 行目まで何手で行けるかとなる※
- たとえば右図の場合、1 行目から 7 行目まで何手で行けるかが答えとなる



※厳密には、この値に 1 を足した値 (最初と最後の整備を考える必要があるため)

- 本問題の答えは、 i 行目から $Next[i]$ 行目まで 1 手で行けるとき、 a 行目から b 行目まで何手で行けるかとなる※
- たとえば右図の場合、1 行目から 7 行目まで何手で行けるかが答えとなる

➡ しかし、 $Q \leq 100000$ なので
自然に実装すると
 $(H, W) = (500000, 2)$ などのケースで TLE



- 本問題の答えは、 i 行目から $Next[i]$ 行目まで 1 手で行けるとき、 a 行目から b 行目まで何手で行けるかとなる※

- たとえば右図の場合、1 行目から 7 行目まで何手で行けるかが答えとなる

ダブリングを使おう！

- ➡ しかし、自然に実装すると
(H, W) = (500000, 2) などのケースで TLE



- ダブリングとは、 $1, 2, 4, 8, 16, \dots$ 手先を前計算することで、 n 手先を高速に計算するテクニック

- ダブリングとは、**1, 2, 4, 8, 16, ...** 手先を前計算することで、 n 手先を高速に計算するテクニック
- 例として、以下のような場合について、地点 1 から地点 8 まで何手で行けるかを計算することを考える

i	1	2	3	4	5	6	7	8	9	10
1 手先 ($Next[i]$)	2	4	4	6	7	7	8	9	10	10

ステップ1 2手先を計算する

[illegible]

ダブリングとは: 具体例

46

ステップ 1 2 手先を計算する

i	1	2	3	4	5	6	7	8	9	10
1 手先 ($Next[i]$)	2	4	4	6	7	7	8	9	10	10
2 手先	4	??	??	??	??	??	??	??	??	??
4 手先	??	??	??	??	??	??	??	??	??	??

1 の 1 手先は 2
2 の 1 手先は 4

ダブリングとは: 具体例

47

ステップ 1 2 手先を計算する

i	1	2	3	4	5	6	7	8	9	10
1 手先 ($Next[i]$)	2	4	4	6	7	7	8	9	10	10
2 手先	4	6	??	??	??	??	??	??	??	??
4 手先	??	??	??	??	??	??	??	??	??	??

2 の 1 手先は 4
4 の 1 手先は 6

ダブリングとは: 具体例

48

ステップ 1 2 手先を計算する

i	1	2	3	4	5	6	7	8	9	10
1 手先 ($Next[i]$)	2	4	4	6	7	7	8	9	10	10
2 手先	4	6	6	??	??	??	??	??	??	??
4 手先	??	??	??	??	??	??	??	??	??	??

3 の 1 手先は 4
4 の 1 手先は 6

ダブリングとは: 具体例

49

ステップ 1 2 手先を計算する

i	1	2	3	4	5	6	7	8	9	10
1 手先 ($Next[i]$)	2	4	4	6	7	7	8	9	10	10
2 手先	4	6	6	7	??	??	??	??	??	??
4 手先	??	??	??	??	??	??	??	??	??	??

4 の 1 手先は 6
6 の 1 手先は 7

ダブリングとは: 具体例

50

ステップ 1 2 手先を計算する

i	1	2	3	4	5	6	7	8	9	10
1 手先 ($Next[i]$)	2	4	4	6	7	7	8	9	10	10
2 手先	4	6	6	7	8	??	??	??	??	??
4 手先	??	??	??	??	??	??	??	??	??	??

5 の 1 手先は 7
7 の 1 手先は 8

ダブリングとは: 具体例

51

ステップ 1 2 手先を計算する

i	1	2	3	4	5	6	7	8	9	10
1 手先 ($Next[i]$)	2	4	4	6	7	7	8	9	10	10
2 手先	4	6	6	7	8	8	??	??	??	??
4 手先	??	??	??	??	??	??	??	??	??	??

6 の 1 手先は 7
7 の 1 手先は 8

ダブリングとは: 具体例

52

ステップ 1 2 手先を計算する

i	1	2	3	4	5	6	7	8	9	10
1 手先 ($Next[i]$)	2	4	4	6	7	7	8	9	10	10
2 手先	4	6	6	7	8	8	9	??	??	??
4 手先	??	??	??	??	??	??	??	??	??	??

7 の 1 手先は 8
8 の 1 手先は 9

ダブリングとは: 具体例

53

ステップ 1 2 手先を計算する

i	1	2	3	4	5	6	7	8	9	10
1 手先 ($Next[i]$)	2	4	4	6	7	7	8	9	10	10
2 手先	4	6	6	7	8	8	9	10	??	??
4 手先	??	??	??	??	??	??	??	??	??	??

8 の 1 手先は 9
9 の 1 手先は 10

ダブリングとは: 具体例

54

ステップ 1 2 手先を計算する

i	1	2	3	4	5	6	7	8	9	10
1 手先 ($Next[i]$)	2	4	4	6	7	7	8	9	10	10
2 手先	4	6	6	7	8	8	9	10	10	??
4 手先	??	??	??	??	??	??	??	??	??	??

9 の 1 手先は 10
10 の 1 手先は 10

ダブリングとは: 具体例

55

ステップ 1 2 手先を計算する

i	1	2	3	4	5	6	7	8	9	10
1 手先 ($Next[i]$)	2	4	4	6	7	7	8	9	10	10
2 手先	4	6	6	7	8	8	9	10	10	10
4 手先	??	??	??	??	??	??	??	??	??	??

10 の 1 手先は 10
10 の 1 手先は 10

ステップ2 4 手先を計算する

[illegible]

ステップ2 4手先を計算する

i	1	2	3	4	5	6	7	8	9	10
1手先 ($Next[i]$)	2	4	4	6	7	7	8	9	10	10
2手先	4	6	6	7	8	8	9	10	10	10
4手先	7	??	??	??	??	??	??	??	??	??

1の2手先は4
4の2手先は7

ダブリングとは: 具体例

58

ステップ2 4手先を計算する

i	1	2	3	4	5	6	7	8	9	10
1手先 ($Next[i]$)	2	4	4	6	7	7	8	9	10	10
2手先	4	6	6	7	8	8	9	10	10	10
4手先	7	8	??	??	??	??	??	??	??	??

2の2手先は6
6の2手先は8

ダブリングとは: 具体例

59

ステップ2 4手先を計算する

i	1	2	3	4	5	6	7	8	9	10
1手先 ($Next[i]$)	2	4	4	6	7	7	8	9	10	10
2手先	4	6	6	7	8	8	9	10	10	10
4手先	7	8	8	??	??	??	??	??	??	??

3の2手先は6
6の2手先は8

ダブリングとは: 具体例

60

ステップ2 4手先を計算する

i	1	2	3	4	5	6	7	8	9	10
1手先 ($Next[i]$)	2	4	4	6	7	7	8	9	10	10
2手先	4	6	6	7	8	8	9	10	10	10
4手先	7	8	8	9	??	??	??	??	??	??

4の2手先は7
7の2手先は9

ダブリングとは: 具体例

61

ステップ2 4手先を計算する

i	1	2	3	4	5	6	7	8	9	10
1手先 ($Next[i]$)	2	4	4	6	7	7	8	9	10	10
2手先	4	6	6	7	8	8	9	10	10	10
4手先	7	8	8	9	10	??	??	??	??	??

5の2手先は8
8の2手先は10

ダブリングとは: 具体例

62

ステップ2 4手先を計算する

i	1	2	3	4	5	6	7	8	9	10
1手先 ($Next[i]$)	2	4	4	6	7	7	8	9	10	10
2手先	4	6	6	7	8	8	9	10	10	10
4手先	7	8	8	9	10	10	??	??	??	??

6の2手先は8
8の2手先は10

ダブリングとは: 具体例

63

ステップ2 4手先を計算する

i	1	2	3	4	5	6	7	8	9	10
1手先 ($Next[i]$)	2	4	4	6	7	7	8	9	10	10
2手先	4	6	6	7	8	8	9	10	10	10
4手先	7	8	8	9	10	10	10	??	??	??

7の2手先は9
9の2手先は10

ダブリングとは: 具体例

64

ステップ2 4手先を計算する

i	1	2	3	4	5	6	7	8	9	10
1手先 ($Next[i]$)	2	4	4	6	7	7	8	9	10	10
2手先	4	6	6	7	8	8	9	10	10	10
4手先	7	8	8	9	10	10	10	10	??	??

8の2手先は10
10の2手先は10

ダブリングとは: 具体例

65

ステップ2 4手先を計算する

i	1	2	3	4	5	6	7	8	9	10
1手先 ($Next[i]$)	2	4	4	6	7	7	8	9	10	10
2手先	4	6	6	7	8	8	9	10	10	10
4手先	7	8	8	9	10	10	10	10	10	??

9の2手先は10
10の2手先は10

ダブリングとは: 具体例

66

ステップ2 4手先を計算する

i	1	2	3	4	5	6	7	8	9	10
1手先 ($Next[i]$)	2	4	4	6	7	7	8	9	10	10
2手先	4	6	6	7	8	8	9	10	10	10
4手先	7	8	8	9	10	10	10	10	10	10

9の2手先は10
10の2手先は10

ステップ 3 地点 1 から地点 8 に行くのに何手かかるかを二分探索で計算

i	1	2	3	4	5	6	7	8	9	10
1 手先 ($Next[i]$)	2	4	4	6	7	7	8	9	10	10
2 手先	4	6	6	7	8	8	9	10	10	10
4 手先	7	8	8	9	10	10	10	10	10	10

ダブリングとは: 具体例

68

ステップ 3 地点 1 から地点 8 に行くのに何手かかるかを二分探索で計算

i	1	2	3	4	5	6	7	8	9	10
1 手先 ($Next[i]$)	2	4	4	6	7	7	8	9	10	10
2 手先	4	6	6	7	8	8	9	10	10	10
4 手先	7	8	8	9	10	10	10	10	10	10

地点 1 から 4 手で地点 7
→ 答えは 4 以上

ダブリングとは: 具体例

69

ステップ 3 地点 1 から地点 8 に行くのに何手かかるかを二分探索で計算

i	1	2	3	4	5	6	7	8	9	10
1 手先 ($Next[i]$)	2	4	4	6	7	7	8	9	10	10
2 手先	4	6	6	7	8	8	9	10	10	10
4 手先	7	8	8	9	10	10	10	10	10	10

地点 7 に進む

地点 1 から 4 手で地点 7
→ 答えは 4 以上

ダブリングとは: 具体例

70

ステップ 3 地点 1 から地点 8 に行くのに何手かかるかを二分探索で計算

i	1	2	3	4	5	6	7	8	9	10
1 手先 ($Next[i]$)	2	4	4	6	7	7	8	9	10	10
2 手先	4	6	6	7	8	8	9	10	10	10
4 手先	7	8	8	9	10	10	10	10	10	10

地点 7 に進む

地点 1 から 4 手で地点 7
→答えは 4 以上

地点 7 から 2 手で地点 9
→答えは 4+2 未満

ダブリングとは: 具体例

71

ステップ 3 地点 1 から地点 8 に行くのに何手かかるかを二分探索で計算

i	1	2	3	4	5	6	7	8	9	10
1 手先 ($Next[i]$)	2	4	4	6	7	7	8	9	10	10
2 手先	4	6	6	7	8	8	9	10	10	10
4 手先	7	8	8	9	10	10	10	10	10	10

地点 1 から 4 手で地点 7
→ 答えは 4 以上

地点 7 に進む

地点 7 から 2 手で地点 9
→ 答えは 4+2 未満

ダブリングとは: 具体例

72

ステップ 3 地点 1 から地点 8 に行くのに何手かかるかを二分探索で計算

i	1	2	3	4	5	6	7	8	9	10
1 手先 ($Next[i]$)	2	4	4	6	7	7	8			
2 手先	4	6	6	7	8	9	9	10	10	10
4 手先	7	8	8	9	10	10	10	10	10	10

地点 7 から 1 手で地点 8
→答えは 4+1 以上

地点 7 から 2 手で地点 9
→答えは 4+2 未満

地点 1 から 4 手で地点 7
→答えは 4 以上

地点 7 に進む

ダブリングとは: 具体例

73

ステップ3 地点1から地点8に行くのに何手かかるかを二分探索で計算

答えは5だと分かった!

2手先

4

6

6

7

8

9

10

10

10

4手先

7

8

8

9

10

10

10

10

10

10

地点1から4手で地点7
→答えは4以上

地点7に進む

地点7から2手で地点9
→答えは4+2未満

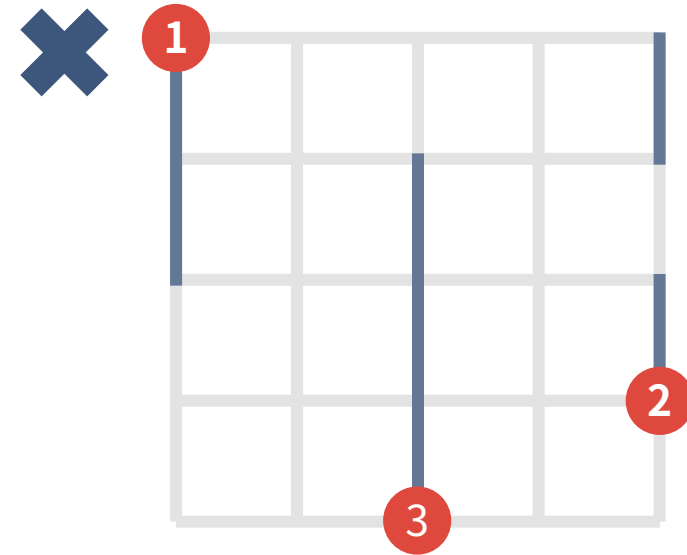
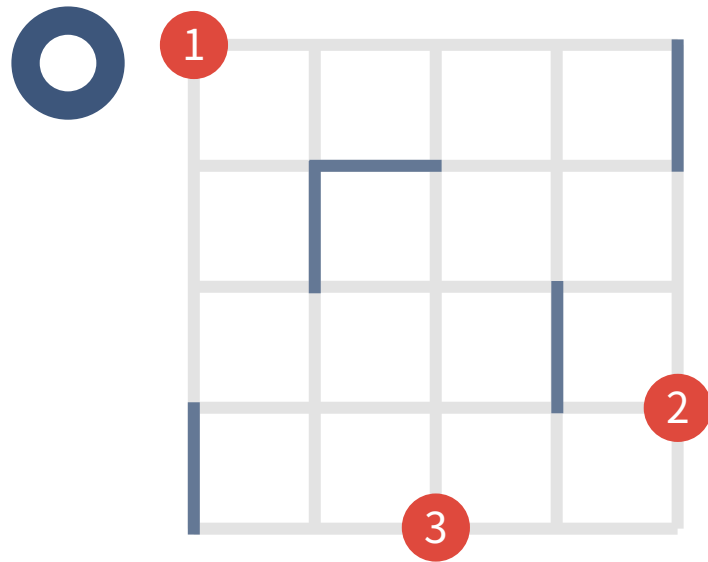
地点7から1手で地点8
→答えは4+1以上

- このように、**ダブリング**の技法を使うと各クエリにつき $O(\log H)$ で解くことができる
- 全体の計算量は $O((H + Q) \log H)$ となり、小課題 3 に通る

小課題 3, 5

$$C_i = 1$$

T が 3 以上になることもある



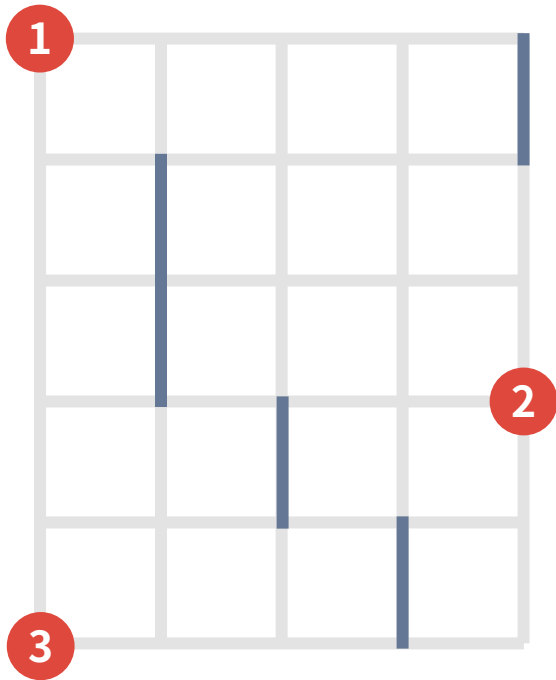
いきなり一般の場合を考えるのは難しいので
連結にすべき地点が道と繋がっていない場合を考えよう

- 座標 $(X_1, Y_1), \dots, (X_T, Y_T)$ (上から順) を連結にしたいが、これらの座標は道と繋がっていないとする
- このとき、 $X_1, X_2, \dots, X_T$ 行目は明らかに整備しなければならない

- 座標 $(X_1, Y_1), \dots, (X_T, Y_T)$ (上から順) を連結にしたいが、これらの座標は道と繋がっていないとする
- このとき、 $X_1, X_2, \dots, X_T$ 行目は明らかに整備しなければならない
- また、 X_i 行目と X_{i+1} 行目の間の部分は独立に考えられるので、単純な足し算で答えが求められる

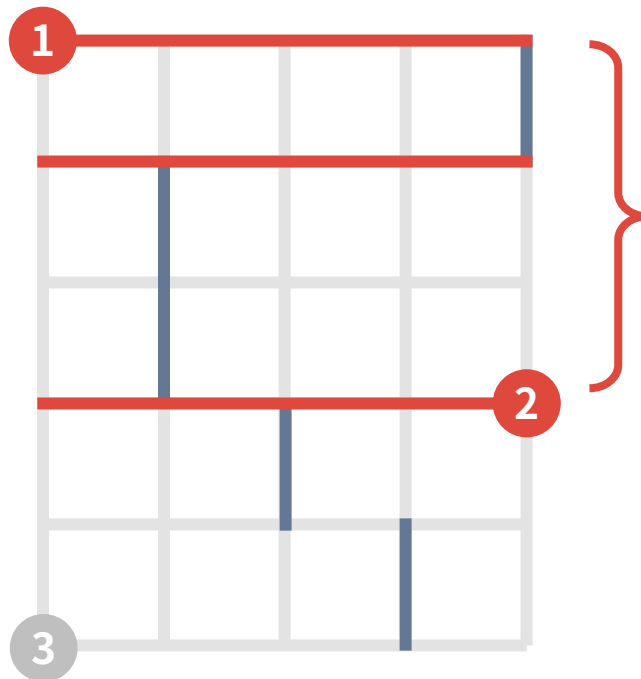
具体例

下図の 1～3 を連結にすることを考える



具体例

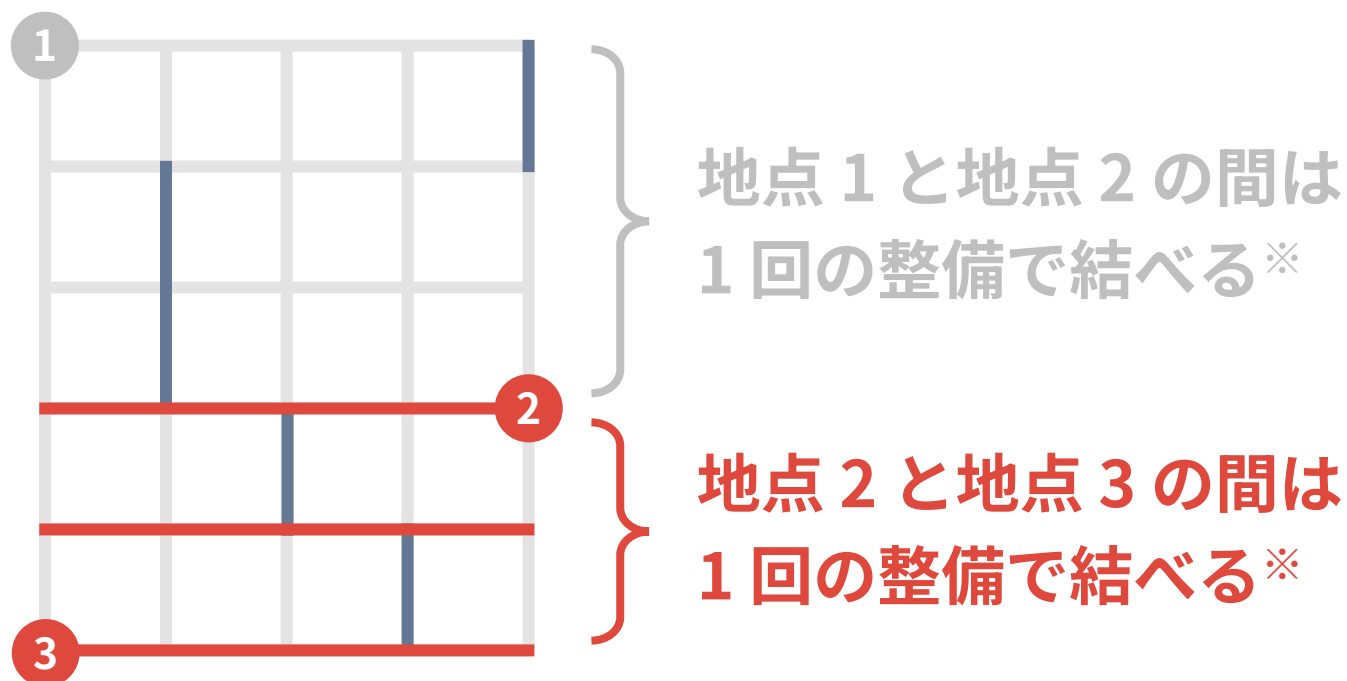
下図の 1～3 を連結にすることを考える



地点 1 と地点 2 の間は
1 回の整備で結べる※

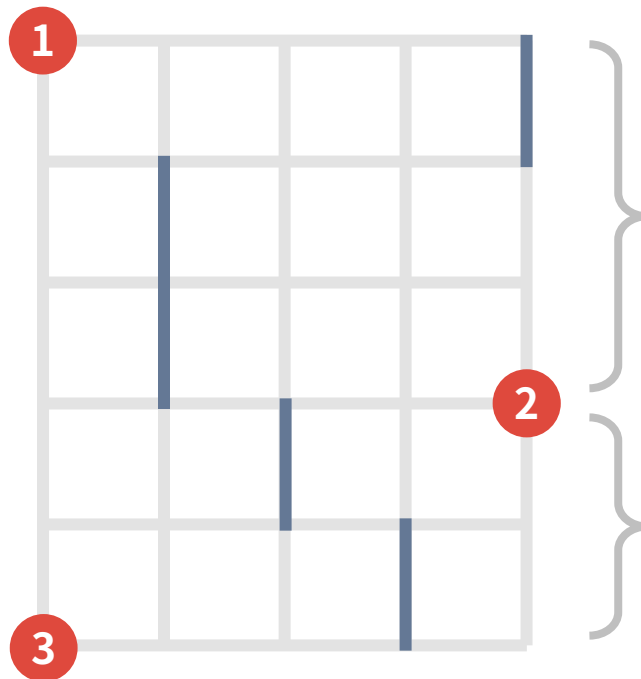
具体例

下図の 1～3 を連結にすることを考える



具体例

下図の 1~3 を連結にすることを考える



地点 1 と地点 2 の間は
1 回の整備で結べる※

地点 2 と地点 3 の間は
1 回の整備で結べる※



全体では
 $3+1+1=5$ 回の整備

このように、道が繋がっていない場合は
単純な足し算で解くことができる

しかし、現実はそうではない

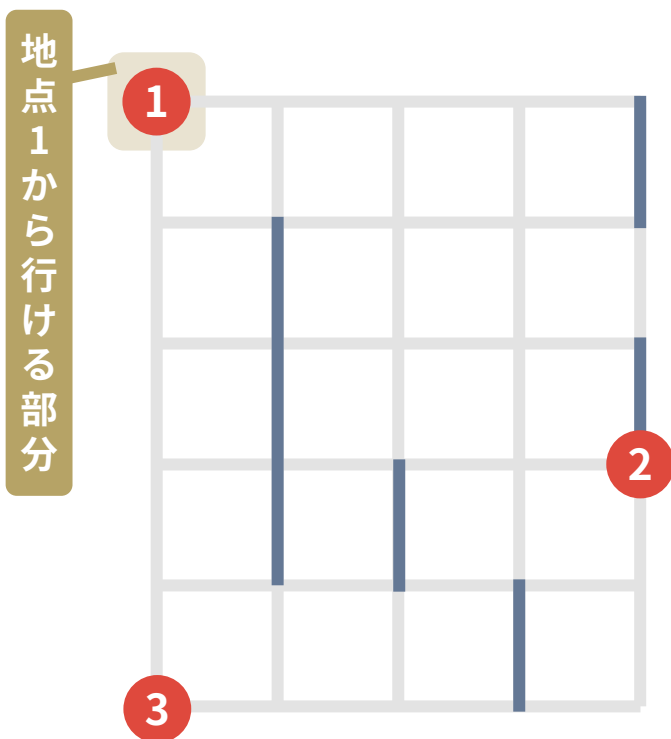
- そこでクエリの連結成分が $[l_1, r_1], \dots, [l_T, r_T]$ 行目を占めているとする
- このとき、次のような貪欲法で最適解を出すことができる
 - $\min(r_1, r_2, \dots, r_T)$ 行目と $\max(l_1, l_2, \dots, l_T)$ 行目を連結にすることを考える
 - 連結にするにあたっては小課題 3 と同様、できるだけ下を整備する貪欲法を使う。
ただし $[l_1, r_1], [l_2, r_2], \dots, [l_T, r_T]$ を一気に飛び越えないようにする

小課題 3, 5: 具体例

85

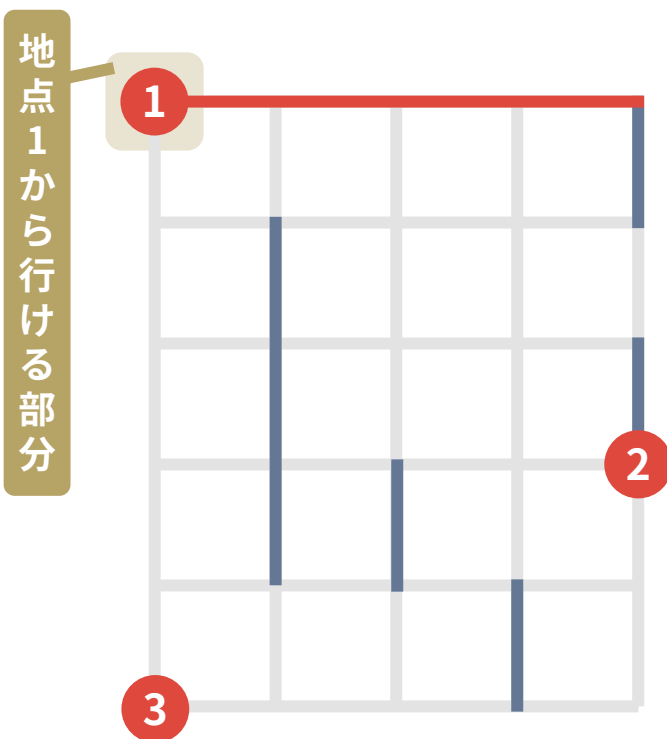
具体例

下図の 1~3 を連結にする (つまり 1 行目から 6 行目に行きたい)



具体例

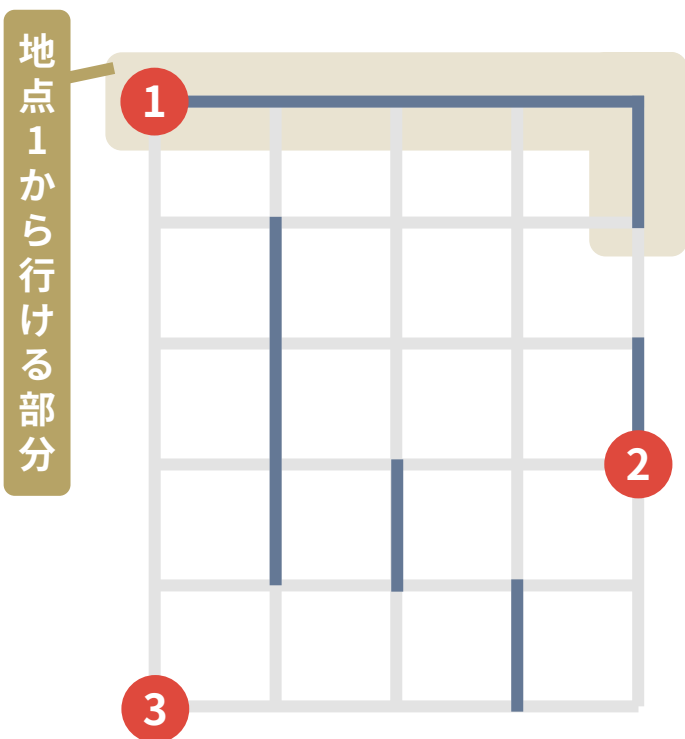
下図の 1~3 を連結にする (つまり 1 行目から 6 行目に行きたい)



- 最初はスタート地点の 1 行目を整備する

具体例

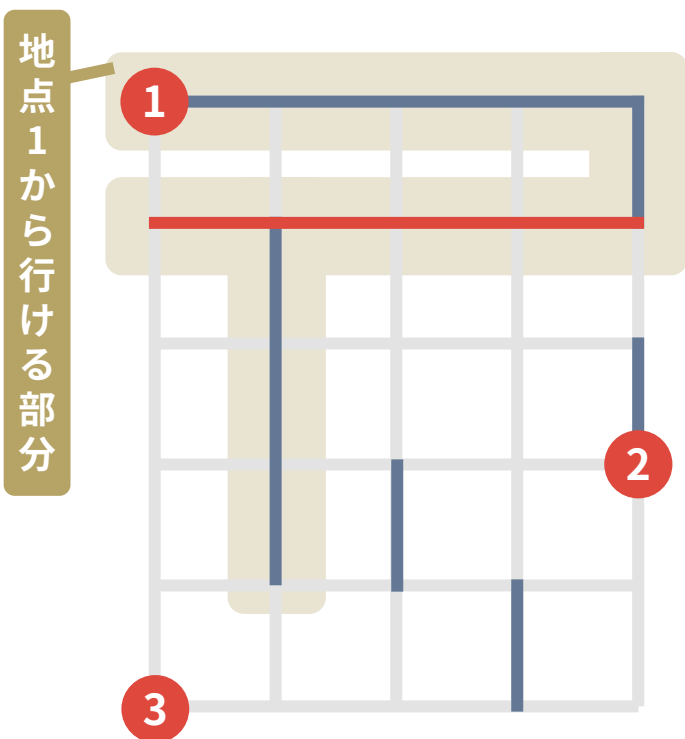
下図の 1~3 を連結にする (つまり 1 行目から 6 行目に行きたい)



- 最初はスタート地点の 1 行目を整備する
- 現時点では 2 行目まで移動可能 → 2 行目を整備

具体例

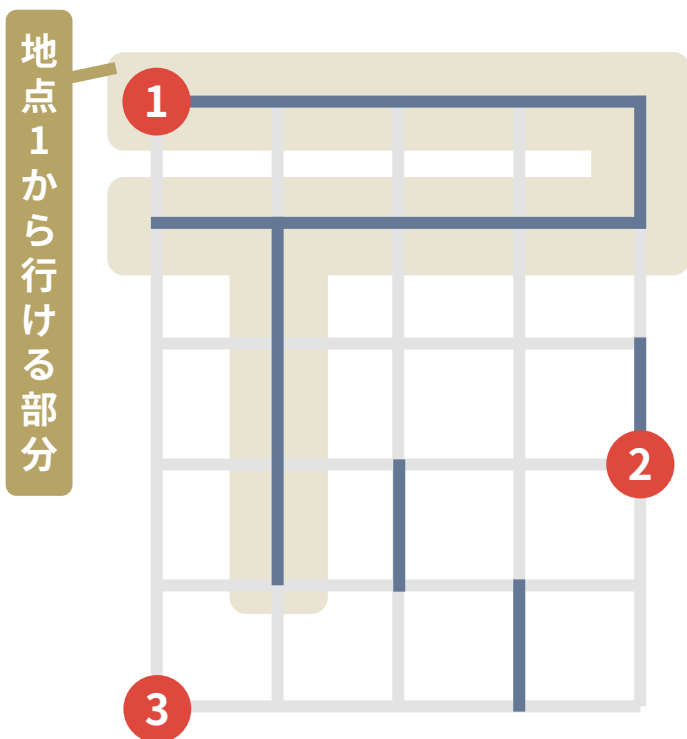
下図の 1~3 を連結にする (つまり 1 行目から 6 行目に行きたい)



- 最初はスタート地点の 1 行目を整備する
- 現時点では 2 行目まで移動可能 → 2 行目を整備
- ここまでは貪欲法と同じ

具体例

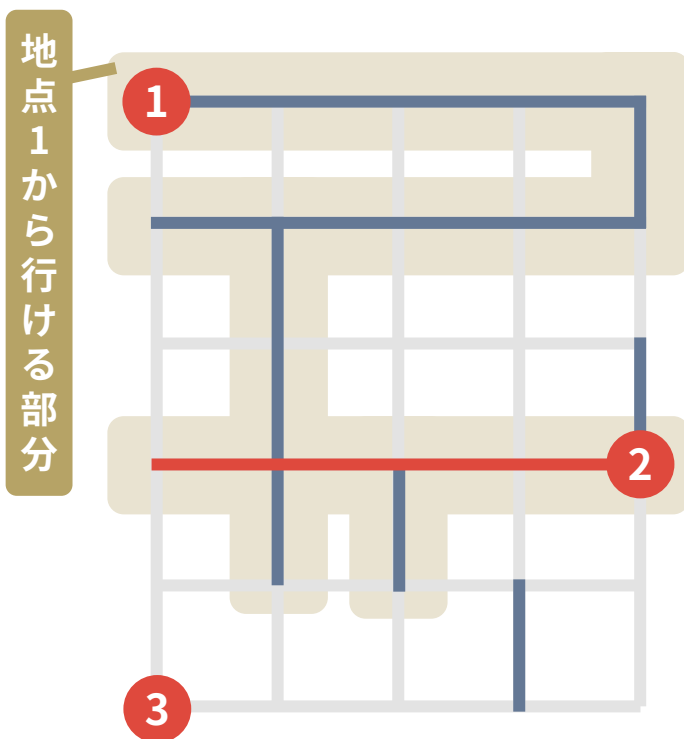
下図の 1~3 を連結にする (つまり 1 行目から 6 行目に行きたい)



- 最初はスタート地点の 1 行目を整備する
- 現時点では 2 行目まで移動可能 → 2 行目を整備
- ここまでは貪欲法と同じ
- 現時点で 5 行目まで行けるが、5 行目を整備すると地点 2 の連結成分 (3~4 行目) を飛び越えてしまう → 4 行目を整備

具体例

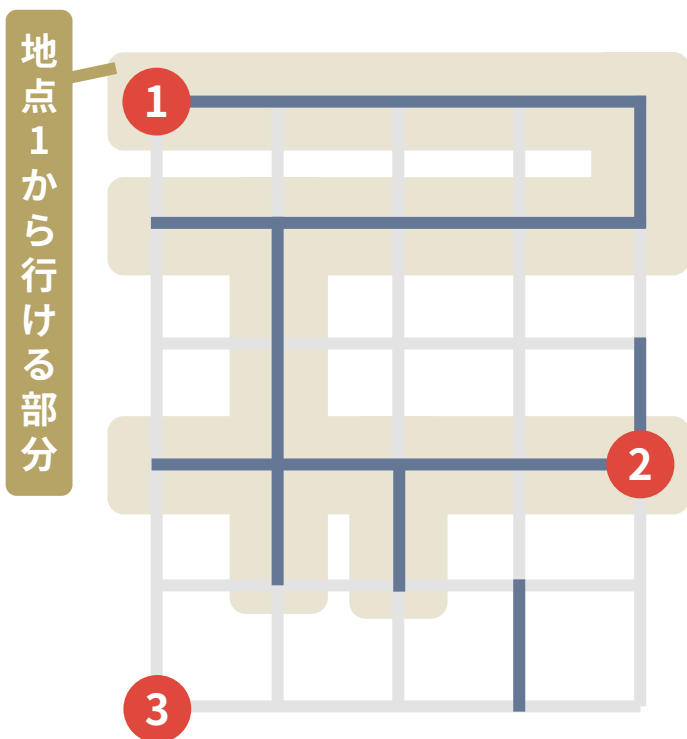
下図の 1~3 を連結にする (つまり 1 行目から 6 行目に行きたい)



- 最初はスタート地点の 1 行目を整備する
- 現時点では 2 行目まで移動可能 → 2 行目を整備
- ここまでは貪欲法と同じ
- 現時点で 5 行目まで行けるが、5 行目を整備すると地点 2 の連結成分 (3~4 行目) を飛び越えてしまう → 4 行目を整備

具体例

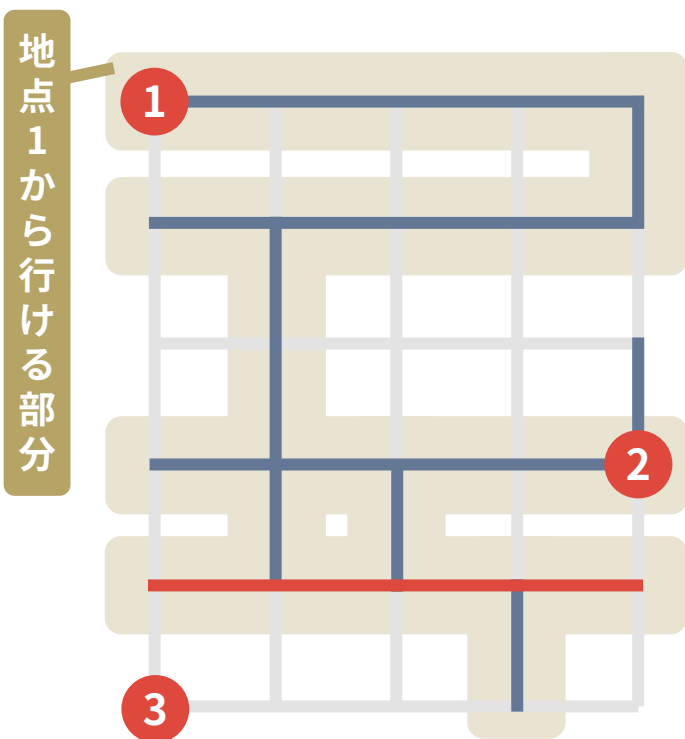
下図の 1~3 を連結にする (つまり 1 行目から 6 行目に行きたい)



- 最初はスタート地点の 1 行目を整備する
- 現時点では 2 行目まで移動可能 → 2 行目を整備
- ここまでは貪欲法と同じ
- 現時点で 5 行目まで行けるが、5 行目を整備すると地点 2 の連結成分 (3~4 行目) を飛び越えてしまう → 4 行目を整備
- 現時点で 5 行目まで行け、地点 2 の連結成分も一気には飛び越えない → 5 行目を整備

具体例

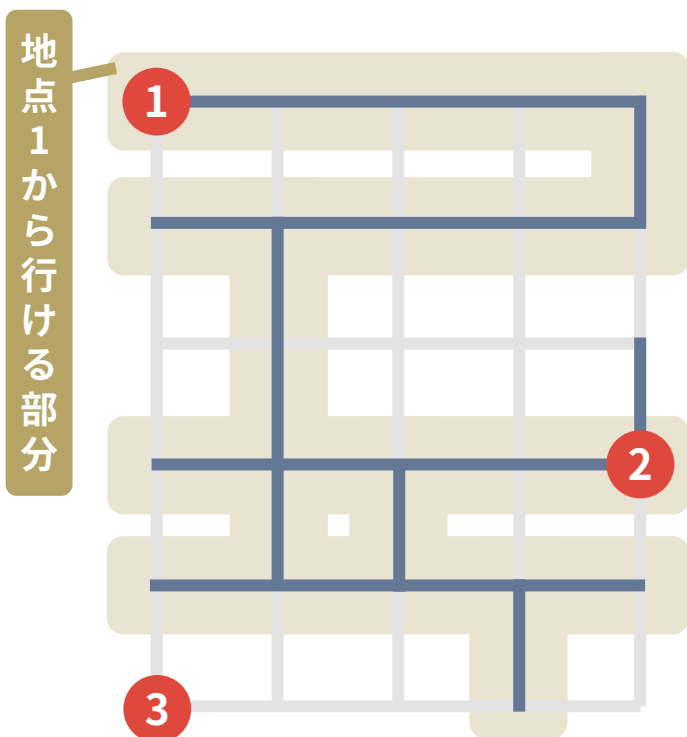
下図の 1~3 を連結にする (つまり 1 行目から 6 行目に行きたい)



- 最初はスタート地点の 1 行目を整備する
- 現時点では 2 行目まで移動可能 → 2 行目を整備
- ここまでは貪欲法と同じ
- 現時点で 5 行目まで行けるが、5 行目を整備すると地点 2 の連結成分 (3~4 行目) を飛び越えてしまう → 4 行目を整備
- 現時点で 5 行目まで行け、地点 2 の連結成分も一気には飛び越えない → 5 行目を整備

具体例

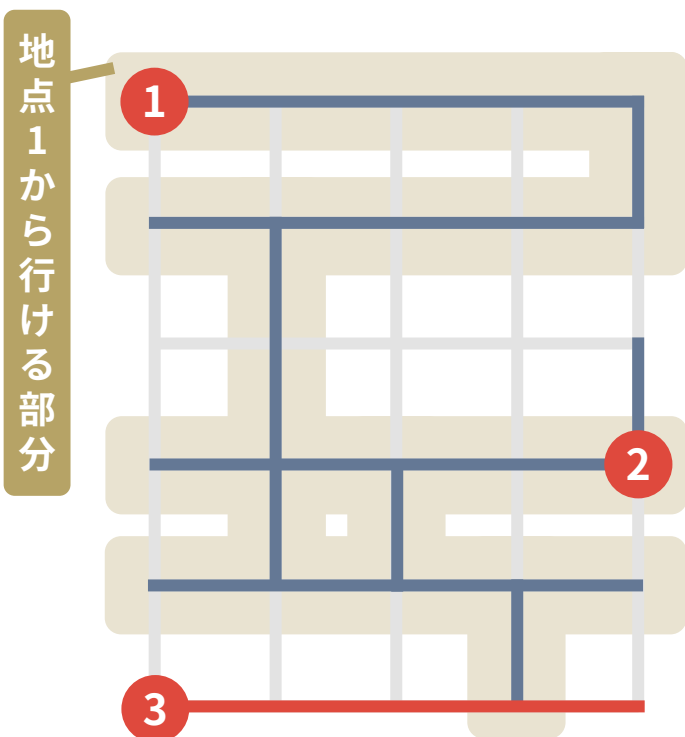
下図の 1~3 を連結にする (つまり 1 行目から 6 行目に行きたい)



- 最初はスタート地点の 1 行目を整備する
- 現時点では 2 行目まで移動可能 → 2 行目を整備
- ここまでは貪欲法と同じ
- 現時点で 5 行目まで行けるが、5 行目を整備すると地点 2 の連結成分 (3~4 行目) を飛び越えてしまう → 4 行目を整備
- 現時点で 5 行目まで行け、地点 2 の連結成分も一気には飛び越えない → 5 行目を整備
- 現時点で 6 行目まで行ける → 6 行目を整備

具体例

下図の 1~3 を連結にする (つまり 1 行目から 6 行目に行きたい)



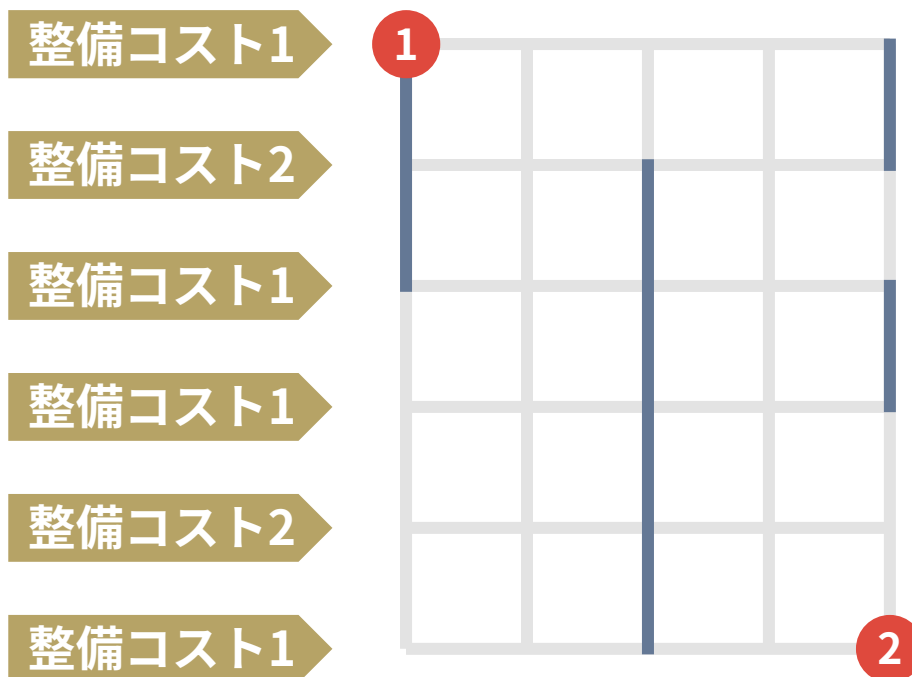
- 最初はスタート地点の 1 行目を整備する
- 現時点では 2 行目まで移動可能 → 2 行目を整備
- ここまでは貪欲法と同じ
- 現時点で 5 行目まで行けるが、5 行目を整備すると地点 2 の連結成分 (3~4 行目) を飛び越えてしまう → 4 行目を整備
- 現時点で 5 行目まで行け、地点 2 の連結成分も一気には飛び越えない → 5 行目を整備
- 現時点で 6 行目まで行ける → 6 行目を整備

5 回で連結にできた！

小課題 6

$$Q \leq 5$$

$$C_i = 1 \text{ または } C_i = 2$$



以降の小課題は、 c_i の値が 2 になることもある

- $T = 2$ の場合を考える
- $PreCost1[i]$ を i 行目の直前の「コスト 1 の行」とする
- $PreCost2[i]$ を i 行目の直前の「コスト 2 の行」とする
- このとき、 i 行目を整備したとき、次に整備すべき場所は以下の二択※
 - $PreCost1[Next[i]]$
 - $PreCost2[Next[i]]$

したがって、以下の形の動的計画法をすると解ける

- $dp[i]$: 一番上の地点と i 行目が連結であり、 i 行目を整備したときのコストの最小値

状態遷移

- $dp[PreCost1[Next[i]]] = \min(dp[PreCost1[Next[i]], dp[i] + 1)$
- $dp[PreCost2[Next[i]]] = \min(dp[PreCost2[Next[i]], dp[i] + 2)$

したがって、以下の形の動的計画法をすると解ける

- $dp[i]$: 一番上の地点と i 行目が連結であり、 i 行目を整備したときのコストの最小値

状態遷移

- $dp[PreCost1[Next[i]]] = \min(dp[PreCost1[Next[i]], dp[i] + 1)$
- $dp[PreCost2[Next[i]]] = \min(dp[PreCost2[Next[i]], dp[i] + 2)$

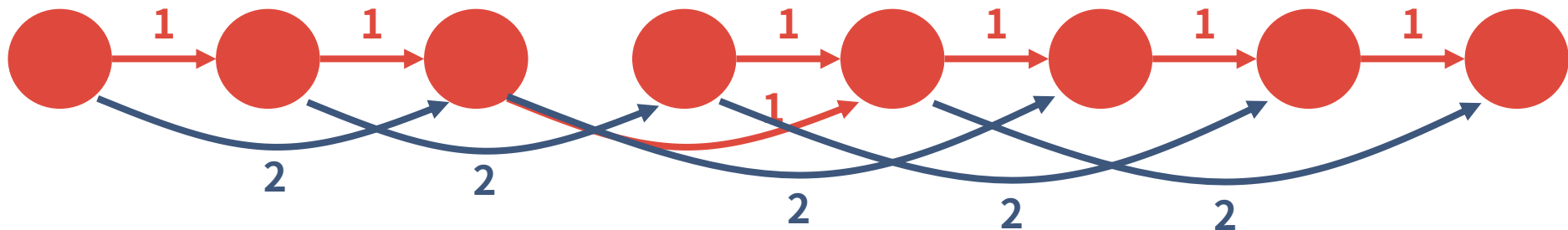
$T \geq 3$ の場合も、クエリの区間 $[l_i, r_i]$ を一気に飛び越えないように注意して DP をすると解ける

小課題 7

$$T = 2$$

クエリの制限は無し

- この小課題を解くには、以下のような問題を解く必要がある
 - 地点 i から $PreCost1[Next[i]]$ に行くのにコスト 1 かかる
 - 地点 i から $PreCost2[Next[i]]$ に行くのにコスト 2 かかる
 - 地点 a から地点 b 以上に行くのに最小でコストいくつ必要か？
 - $O(\log N)$ のオーダーで求めよ



- この小課題を解くには、以下のような問題を解く必要がある

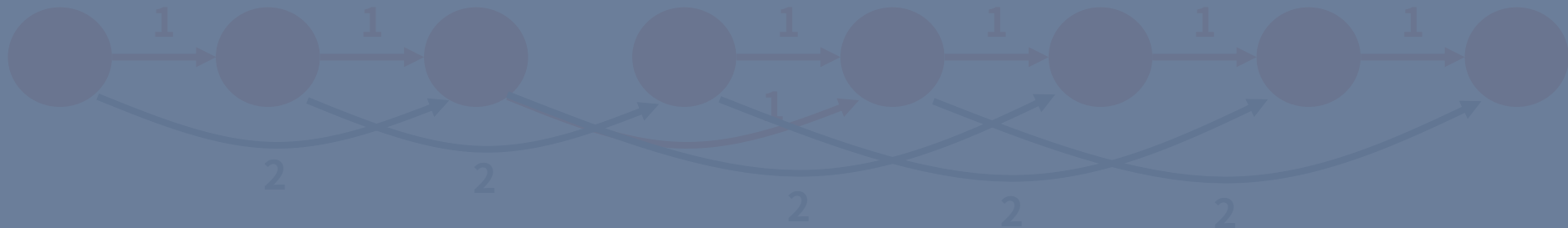
- 地点 i から $PreCost1[Next[i]]$ に行くのにコスト 1 かかる

- 地点 i から $PreCost2[Next[i]]$ に行くのにコスト 2 かかる

コスト 1 だとダブリングで解けるが...

- $O(\log N)$ のオーダーで求めよ

コスト 2 ではどうするのか？



- 以下のようなダブリングを考える
 - 各地点 i から、コスト $2^{k-1}, 2^k, 2^{k+1}$ で行ける最大の場所を計算

- 以下のようなダブリングを考える
 - 各地点 i から、コスト 2^{k-1} , 2^k , 2^{k+1} で行ける最大の場所を計算
- これらの場所は、次のようにして計算できる
 - 2^{k-1} : (コスト $2^{k-1}-1$ + コスト $2^{k-1}+0$) または (コスト $2^{k-1}+0$ + コスト $2^{k-1}-1$)
 - 2^k : (コスト $2^{k-1}-1$ + コスト $2^{k-1}+1$) または (コスト $2^{k-1}+0$ + コスト $2^{k-1}+0$)
 - 2^{k+1} : (コスト $2^{k-1}+0$ + コスト $2^{k-1}+1$) または (コスト $2^{k-1}+1$ + コスト $2^{k-1}+0$)

- 以下のようなダブリングを考える

- 各地点 i から、コスト $2^{k-1}, 2^k, 2^{k+1}$ で行ける最大の場所を計算

一般の x についてコスト x で行ける場所は

- 2^{k-1} : (コスト $2^{k-1-1} + \text{コスト } 2^{k-1+0}$) または (コスト $2^{k-1+0} + \text{コスト } 2^{k-1-1}$)

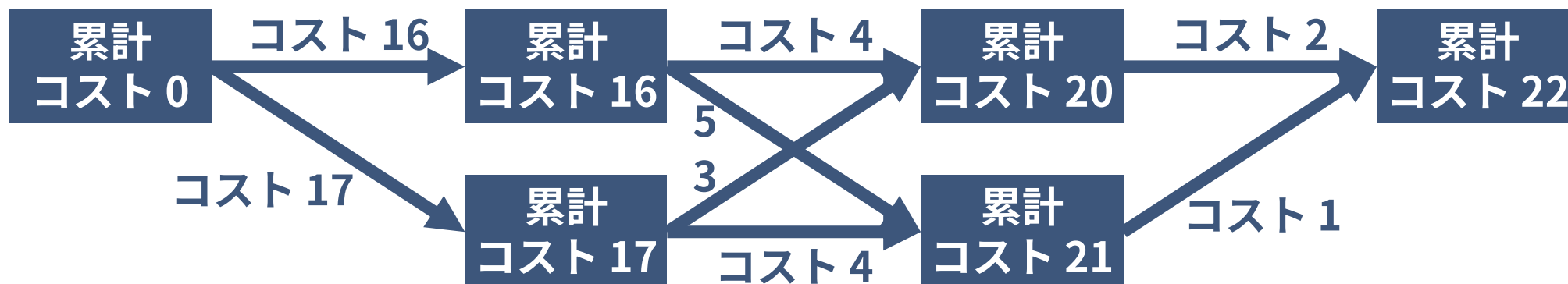
- 2^k : (コスト $2^{k-1} + \text{コスト } 2^{k-1}$) または (コスト $2^{k-1} + \text{コスト } 2^{k-1}$)

- 2^{k+1} : (コスト $2^{k+0} + \text{コスト } 2^{k+1}$) または (コスト $2^{k+1} + \text{コスト } 2^{k+0}$)

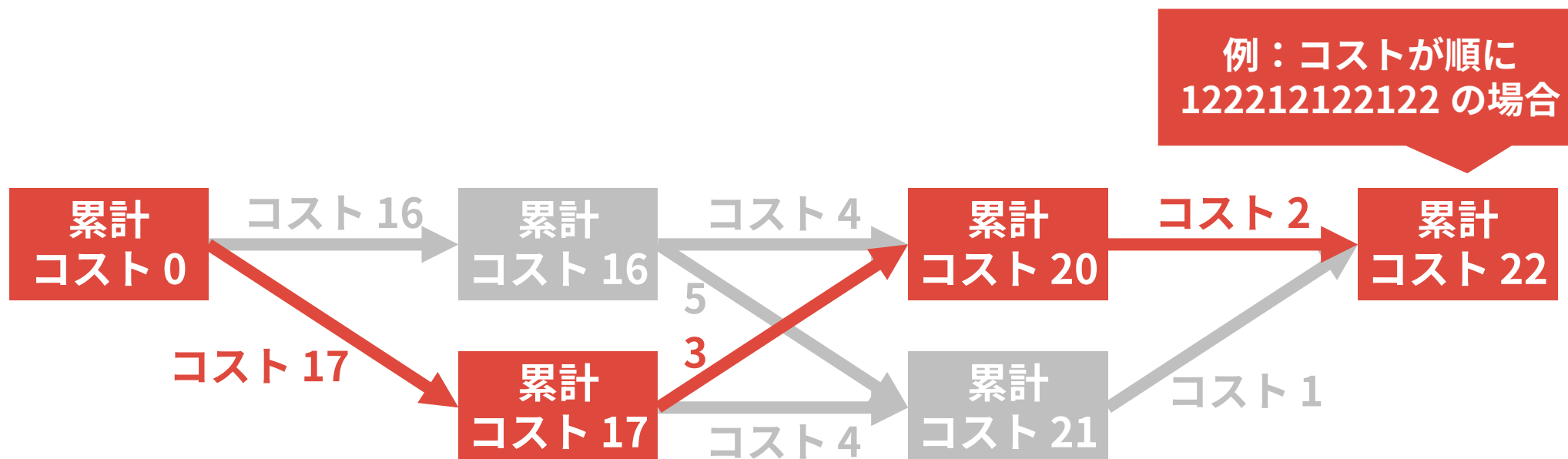
どうやって計算すれば良いか？

- まずは例として、コスト 22 を計算したいとする ($22=16+4+2$)

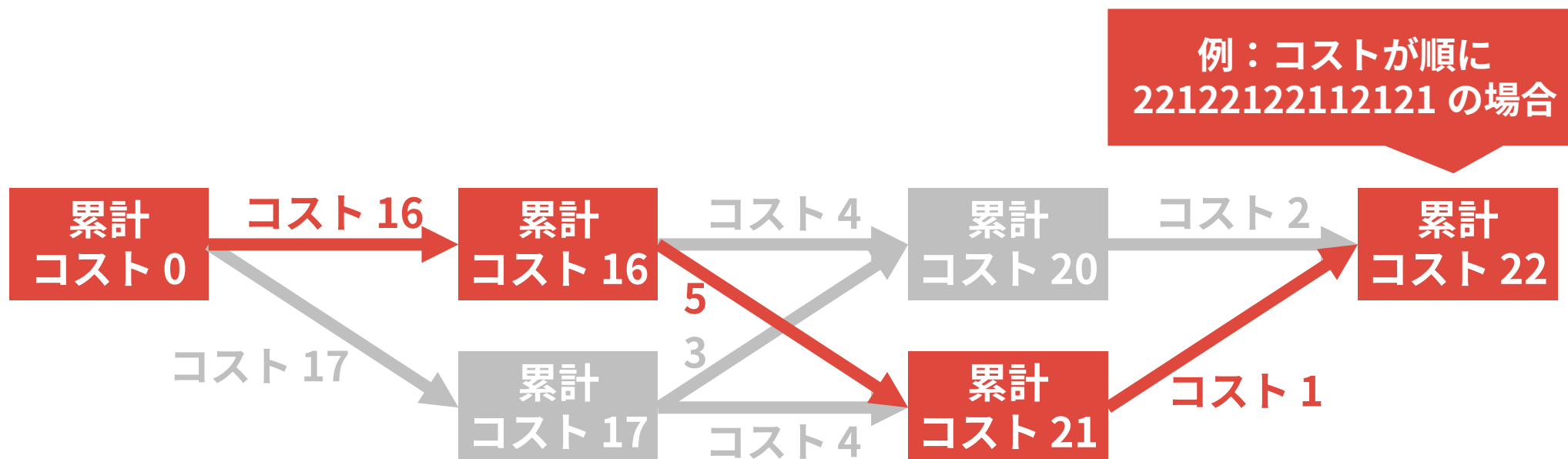
- まずは例として、コスト 22 を計算したいとする ($22=16+4+2$)
- このとき、どのようなコスト 22 の経路も以下の形で表せる



- まずは例として、コスト 22 を計算したいとする ($22=16+4+2$)
- このとき、どのようなコスト 22 の経路も以下の形で表せる



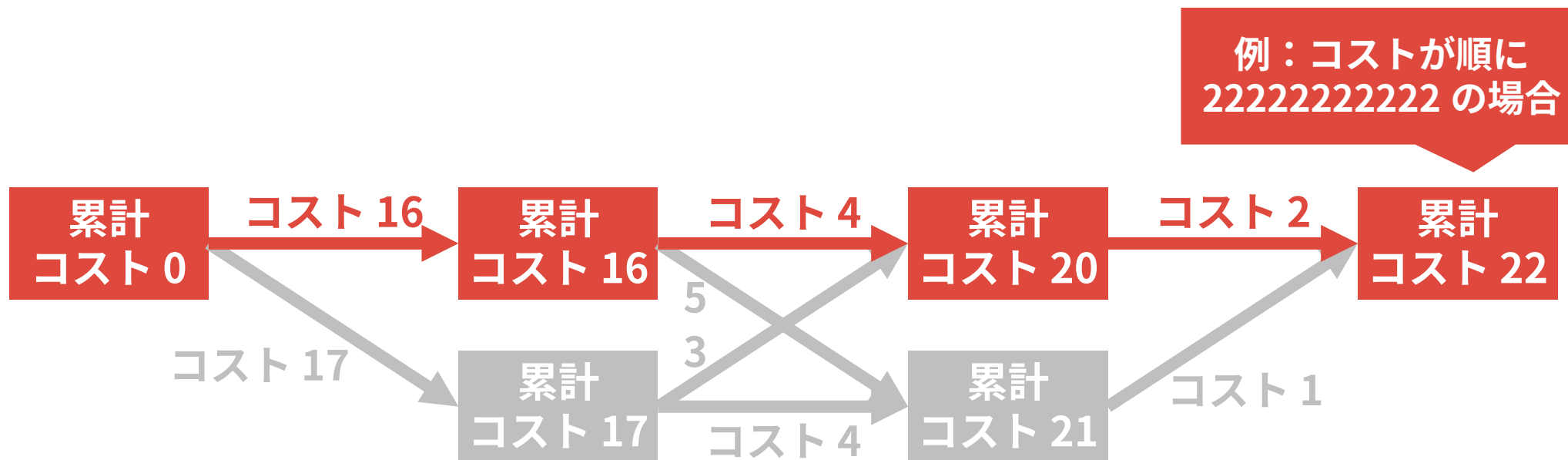
- まずは例として、コスト 22 を計算したいとする ($22=16+4+2$)
- このとき、どのようなコスト 22 の経路も以下の形で表せる



小課題 7: 具体例

110

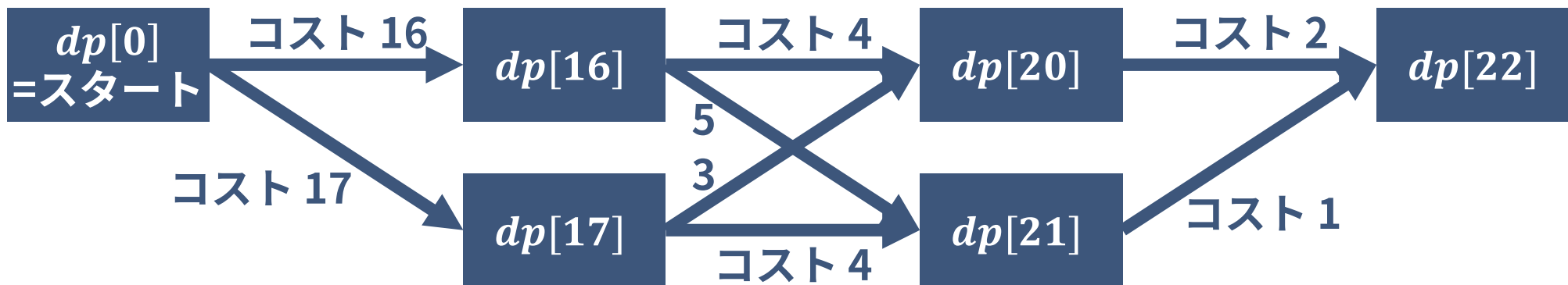
- まずは例として、コスト 22 を計算したいとする ($22=16+4+2$)
- このとき、どのようなコスト 22 の経路も以下の形で表せる



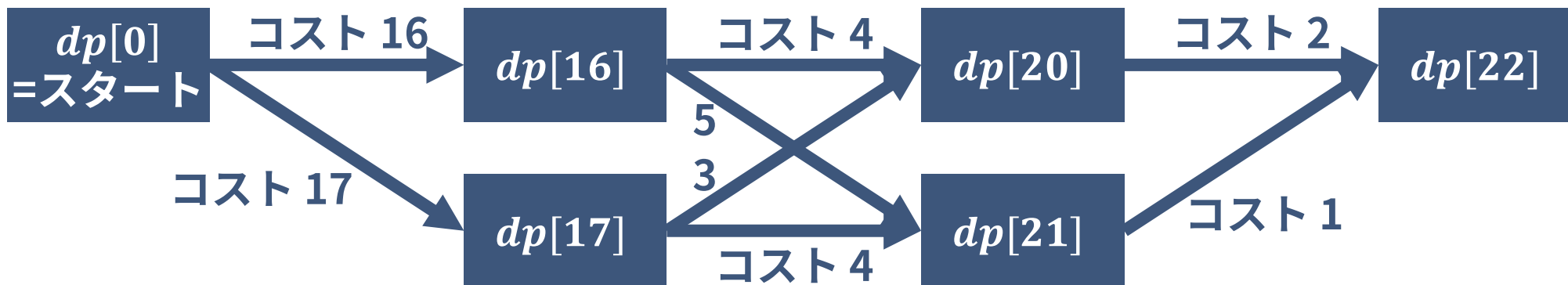
小課題 7: 具体例

111

- ここで、次のような DP を考える
 - $dp[i]$: コスト i で行ける位置の最大値 (最終的な答えは $dp[22]$)



- ここで、次のような DP を考える
 - $dp[i]$: コスト i で行ける位置の最大値 (最終的な答えは $dp[22]$)
- コスト $2^k - 1, 2^k, 2^k + 1$ が前計算されているので、各 i について $dp[i]$ を $O(1)$ で計算できる！



- ここで、次のような DP を考える

- $dp[i]$: コスト i で行ける位置の最大値 (最終的な答えは $dp[22]$)

- コスト $2^k - 1, 2^k, 2^k + 1$ が前計算されているので、各 i について $dp[i]$ を

これでコスト 22 の場合が
 $O(1)$ で計算できる!

計算量 $O(\log 22)$ で解けた



一般の場合でも、以下のようにしてコスト x で行ける最大の位置がわかる

- コスト x で行ける最大の位置を計算したいとする
- $x = 2^{n_1} + 2^{n_2} + 2^{n_3} + \dots + 2^{n_t}$ とする
- このとき、 $i = 1, 2, \dots, t$ の順に以下を計算する ($dp0[t]$ が答え)
 - $dp0[i]$: コスト $2^{n_1} + \dots + 2^{n_i} + 0$ で行ける最大の位置
 - $dp1[i]$: コスト $2^{n_1} + \dots + 2^{n_i} + 1$ で行ける最大の位置

- さて、小課題 7 では以下のような問題を解く必要があった
 - 地点 a から地点 b まで行くのに最小でコストいくつ必要か？

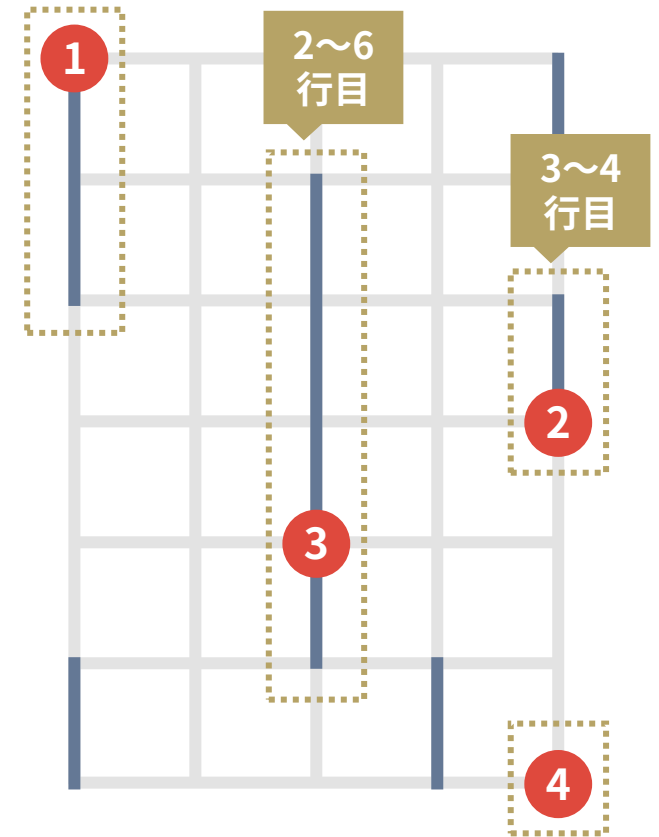
- さて、小課題 7 では以下のような問題を解く必要があった
 - 地点 a から地点 b まで行くのに最小でコストいくつ必要か？
- 前述の問題 (地点 a からコスト x で行ける最大の位置) とは少し違う
- しかし x を二分探索すると、 $O(\log H)$ で解ける※

86点

小課題 8

追加の制約はない

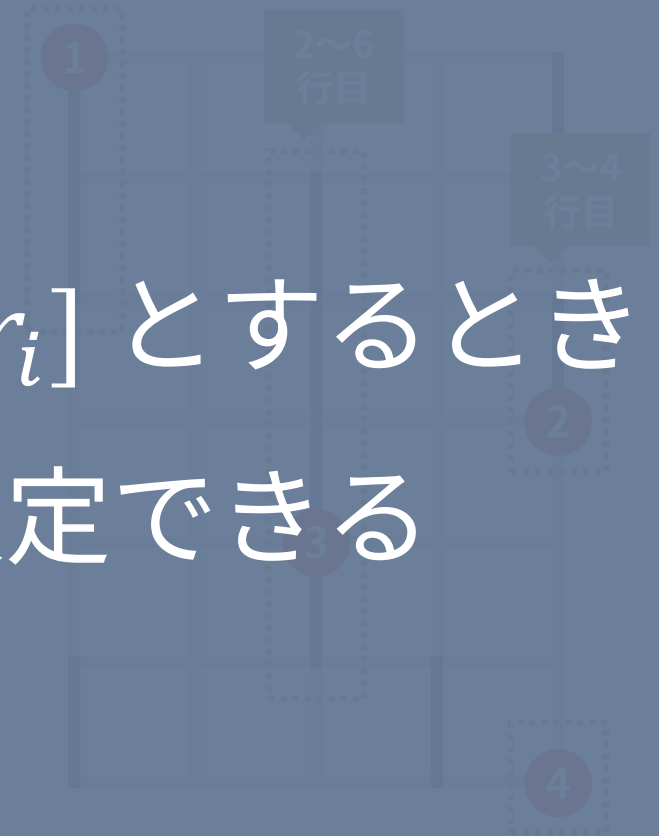
- クエリの連結成分の区間を $[l_1, r_1], \dots, [l_T, r_T]$ 行目とする
- このとき、 $[l_i, r_i]$ が $[l_j, r_j]$ に完全に含まれている場合、区間 $[l_j, r_j]$ は無視してよい
- たとえば下図の場合、地点 3 は無視して、地点 1, 2, 4 のみを連結にする問題だと考えて良い



- クエリの連結成分の区間を $[l_1, r_1], \dots, [l_T, r_T]$ 行目とする

したがって、クエリの区間を $[l_i, r_i]$ とするとき

- たとえば下図の場合、地点 3 は無視して、地点 1, 2, 4 のみを連結にする問題だと考えて良い



- そこで次のような動的計画法を考える※
 - $dp1[i]$: 地点 1 から地点 i までを連結にするのにかかる最小コスト
 - $dp2[i]$: 地点 1 から地点 i までを連結にする場合、コスト $dp1[i] + 1$ をかければ何行目まで行けるか

※ $l_1 < l_2 < \dots < l_T$ および $r_1 < r_2 < \dots < r_T$ が成り立つとする。つまり地点 1 は一番上

- そこで次のような動的計画法を考える※
 - $dp1[i]$: 地点 1 から地点 i までを連結にするのにかかる最小コスト
 - $dp2[i]$: 地点 1 から地点 i までを連結にする場合、コスト $dp1[i] + 1$ をかければ何行目まで行けるか

➡ 小課題 7 で解いた問題を使えば、 $dp1[i - 1], dp2[i - 1]$ から $dp1[i], dp2[i]$ を計算することができる

小課題7 の問題

- 地点 a からコスト x でどこまで行けるか？
- 地点 a から地点 b まで最小コストいくつで行けるか？

- そこで次のような動的計画法を考える
 - $dp1[i]$: 地点 1 から地点 i までを連結にするのにかかる最小コスト
 - $dp2[i]$: 地点 1 から地点 i までを連結にする場合、コスト $dp1[i] + 1$ をかければ何行目まで行けるか

これで計算量 $O(T \log H)$ で



小課題 7 で解いた問題を使えば、 $dp1[i-1], dp2[i-1]$ から

$dp1[i], dp2[i]$ を計算することができる

各クエリを解くことができた！

小課題7
の問題

- 地点 a からコスト x でどこまで行けるか？
- 地点 a から地点 b まで最小コストいくつで行けるか？

- この問題は実装と場合分けが非常に大変
- しかし、以下のようなテクニックを使うとバグりにくくなったり、バグ取りが速くなったりする
 - コードに適切なコメントを書く
 - 実装する前に紙に実装方針を書く
 - 自分なりのコーディングスタイルを身につける etc.

得点分布

得点分布

125

